

第5章 回溯与分支限界

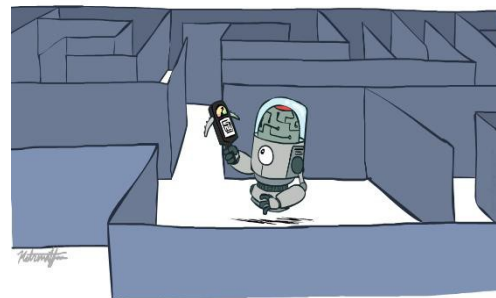
问题求解与搜索算法

问题求解涉及到两个方面：

- 1) **问题的表示**，比如枚举对象、状态表示
- 2) **求解的方法**，比如自顶向下的递归，自底向上的动态规划

□ 搜索算法是一种**通用的问题求解方法**：首先把问题表示转换为一个状态空间图，然后设计特定的**图遍历方法**在状态空间中搜索问题的答案。

□ 为了提高搜索的效率，在遍历状态空间时**需要添加优化技术**，比如**剪枝策略**用于尽可能避免无效搜索，**启发式信息**用来加速朝目标状态逼近的速度。



状态空间图

一个问题用搜索算法求解时，往往需要把问题描述为状态空间图，包括以下要素：

状态（State）：是为描述某类不同事物间的差别而引入的一组最少变量的有序集合，其矢量形式为： $X = [x_1, x_2, \dots, x_n]$ ，其中每个分量 $x_i (i=1, \dots, n)$ 称为状态分量。

操作符（运算符）：是指把一个状态转换为另外一个状态的操作或者运算。操作符可以是规划、数学运算和问题场景中行为等。

状态图：如果把状态定义为图的结点，操作符定义为图的边，一个问题的全部可能状态则可以表示为一个图，即状态图。

路径：通过操作符序列连接起来的状态图中的一个状态序列。

路径耗散函数：定义在路径上的一个数值函数，它反映了一条路径的性能度量或者求解问题的代价。在求解最优化问题时，路径耗散函数往往与优化目标相关联。

状态空间图

状态空间图可以形式化地定义为一个四元组 (S, A, G, F)

- S 表示问题的**初始状态**，它是搜索的起点。
- A 是采取的**操作符集合**，初始状态和操作符隐含地定义了问题的状态图。
- G 表示**目标测试**，它判断给定的状态是否为目标状态。它可以是表示目标状态的一个状态集合，也可以是一个判定函数。
- F 代表**路径耗散函数**，它的定义需要具体问题具体分析。

搜索就是在状态空间图中从初始状态出发，执行特定的操作，试探地寻找目标状态的过程。当然，也可以从目标结点到初始结点反向进行。状态空间图中从初始状态到目标状态的路径则代表**问题的解**。解的优劣由路径耗散函数量度，最优解就是路径耗散函数值最小的路径。

深度与复杂度权衡

深度对资源的影响

深度即从初始状态到目标状态的步数。深度增加虽然可能带来更优解，但会显著增加时间和空间复杂度，因为需要探索的状态数量呈指数级增长。

完整性的定义

完整性是指搜索算法保证能找到解（如果存在）的特性。一个完整的算法最终一定能到达目标状态，但可能需要大量时间和内存。

最优性的意义

最优性意味着搜索算法能找到成本最低或路径最短的解。在资源有限的情况下，找到最优解往往比找到任意解更具挑战性。

时间与空间复杂度的考量

时间复杂度和空间复杂度是衡量搜索效率的关键指标。高效的搜索算法需要在有限的时间和内存内找到满意的解，这对算法设计提出了很高的要求。

。

状态空间图-传教士问题

【例1】在河的左岸有三个传教士、一条船和三个野人，传教士们想用这条船将所有的成员都运过河去，但是受到以下条件的限制：

- ① 教士和野人都会划船，但船一次最多只能装运两个；
 - ② 在任何岸边野人数目都不得超过传教士，否则传教士会遭遇危险：被野人攻击甚至吃掉。
- 此外，假定野人会服从任何一种过河安排，试设计出一个确保全部成员安全过河的计划。

1) 状态表示 确定问题的状态表示，以及每一个状态变量的值域。

渡河问题包括三类对象：传教士，野人和渡船，得三元组 $S = (m, c, b)$ ，其中：

- ✓ m 为左岸传教士数，有 $m = \{0, 1, 2, 3\}$ ；对应右岸的传教士数为 $3 - m$ 。
- ✓ c 为左岸的野人数，有 $c = \{0, 1, 2, 3\}$ ；对应右岸野人数为 $3 - c$ 。
- ✓ b 为左岸渡船数，有 $b = \{0, 1\}$ ，右岸的船数为 $1 - b$ 。

初始状态只有一个，即 $S_0 = (3, 3, 1)$ ，表示全部成员在河的左岸；目标状态也只有一个，即 $S_g = (0, 0, 0)$ ，表示全部成员从河左岸渡河完毕。

状态空间图-传教士问题

状态	(m, c, b)	状态	(m, c, b)	状态	(m, c, b)	状态	(m, c, b)
S ₀	331	S ₈	131	S ₁₆	330	S ₂₄	130
S ₁	321	S ₉	121	S ₁₇	320	S ₂₅	120
S ₂	311	S ₁₀	111	S ₁₈	310	S ₂₆	110
S ₃	301	S ₁₁	101	S ₁₉	300	S ₂₇	100
S ₄	231	S ₁₂	031	S ₂₀	230	S ₂₈	030
S ₅	221	S ₁₃	021	S ₂₁	220	S ₂₉	020
S ₆	211	S ₁₄	011	S ₂₂	210	S ₃₀	010
S ₇	201	S ₁₅	001	S ₂₃	200	S ₃₁	000

表中的状态并不全都是合法的
状态，可以删除非法的状态，
从而加速搜索过程。

2) 操作符集合

- 把船从左岸划向右岸定义为 L_{ij} 操作，第一下标 i 表示船载的传教士数, 第二下标 j 表示船载的野人数。
- 从右岸将船划回左岸称之为 R_{ij} 操作，下标的定义同前。
- 则共有10种操作，操作集为

$$F = \{L_{01}, L_{10}, L_{11}, L_{02}, L_{20}, R_{01}, R_{10}, R_{11}, R_{02}, R_{20}\}$$

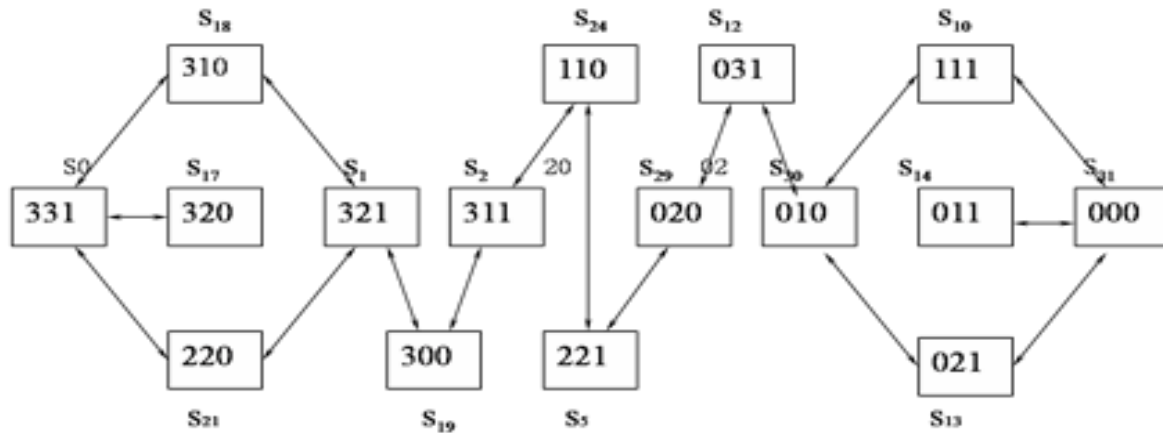
状态空间图-传教士问题

1) 状态表示: $S = (m, c, b)$

初始状态 $S_0 = (3, 3, 1)$, 目标状态 $S_g = (0, 0, 0)$

2) 操作符集合

$F = \{L_{01}, L_{10}, L_{11}, L_{02}, L_{20}, R_{01}, R_{10}, R_{11}, R_{02}, R_{20}\}$



状态空间图-8 数码问题

【例2】把左图的8数码排列通过空格的上下左右移动，转换为右图的8数码排列

□ 状态表示

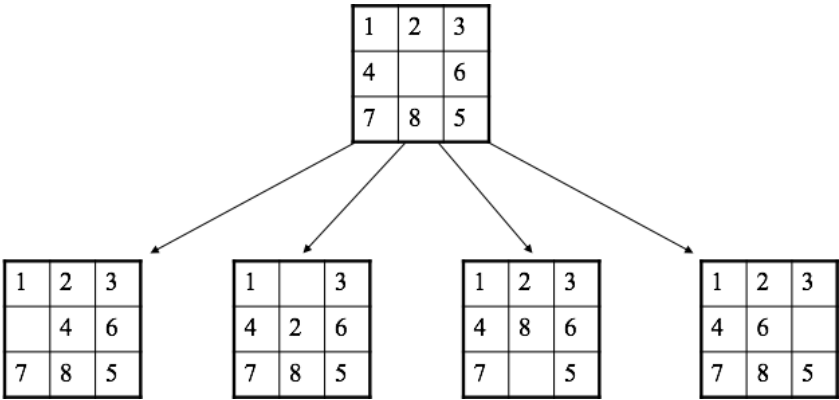
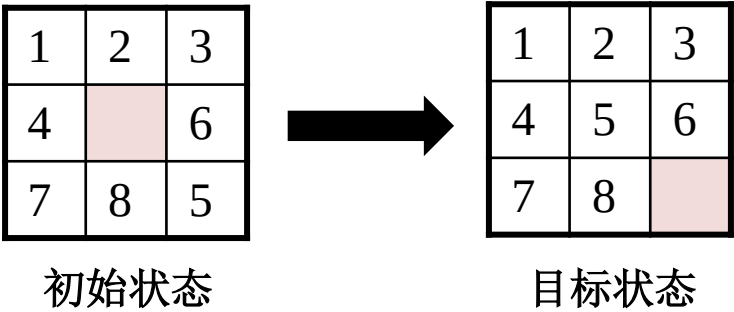
数字格与空格组成的排列

□ 操作符集合

空格向上、下、左和右移动

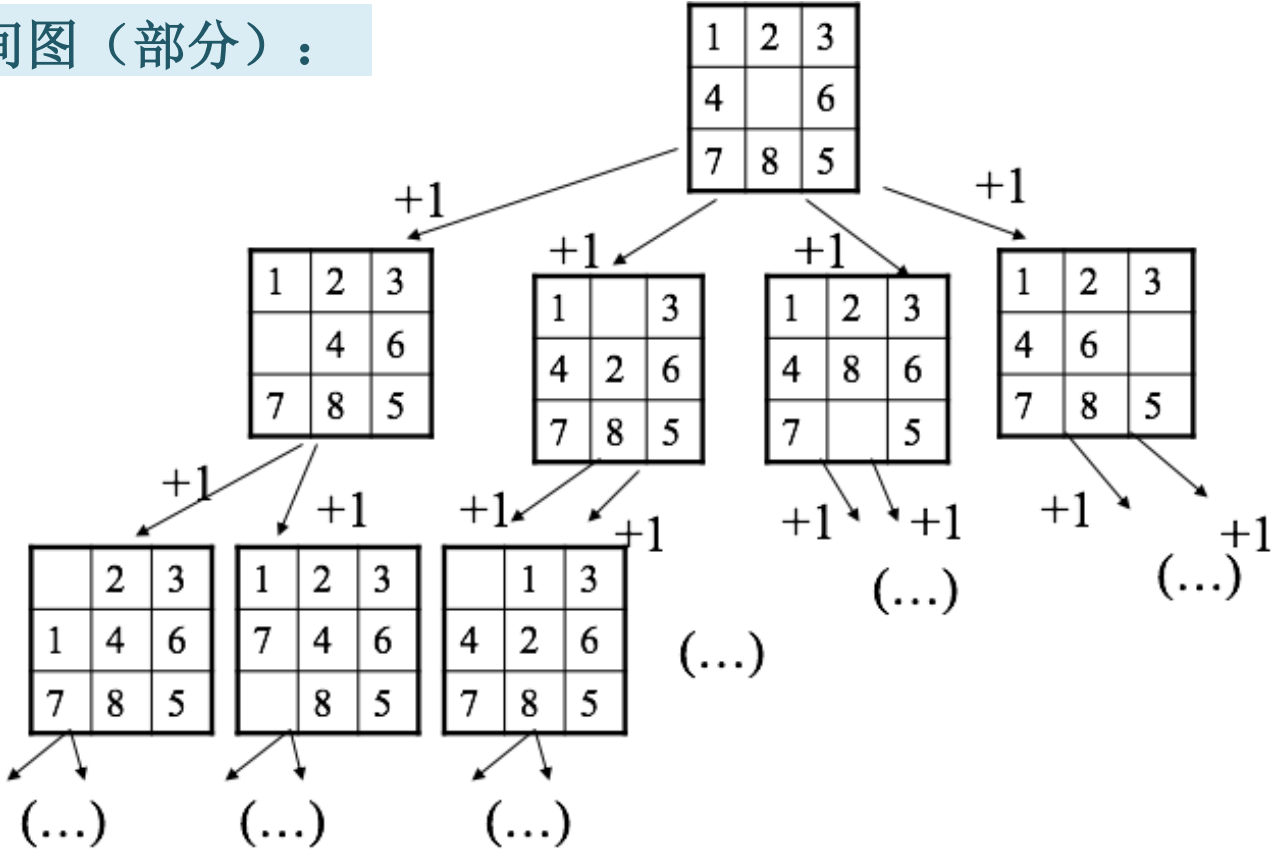
□ 路径耗散函数

每一次移动代价为 1，路径耗散等于移动次数



状态空间图-8 数码问题

状态空间图（部分）：



搜索算法类型

- 基于枚举策略的通用搜索
 - ✓ 深度优先搜索
 - ✓ 广度优先搜索
- 枚举 + 优化的通用搜索
 - ✓ 回溯算法 = 深度优先搜索 + 剪枝策略
 - ✓ 分支限界算法 = 广度优先搜索 + 剪枝策略
- 启发式搜索，启发式搜索是一种基于规则的优化搜索算法。
- 对抗搜索，DeepBlue，AlphaGO中的搜索



无 信 息 搜 索 算 法

(U n i n f o r m e d S e a r c h A l g o r i t h m s)

深度优先搜索

Depth First Search

01

DFS概述



DFS定义

深度优先搜索 (DFS) 是一种遍历树和图结构的算法，它从根节点开始，沿着一条路径尽可能深入地探索，直到遇到没有后继节点的叶子节点，然后回溯。



DFS的优势

DFS实现简单，内存占用低，适用于深度未知或状态爆炸的问题。



AI中的作用

在人工智能中，DFS被广泛应用于解决各种问题，如路径规划、迷宫求解、游戏树搜索等，它能够快速找到解决方案，尽管不一定是成本最优的。



与其它方法交叉

DFS可以与启发式搜索、并行计算、强化学习等技术结合，弥补其局限，提升性能。

DFS流程

一

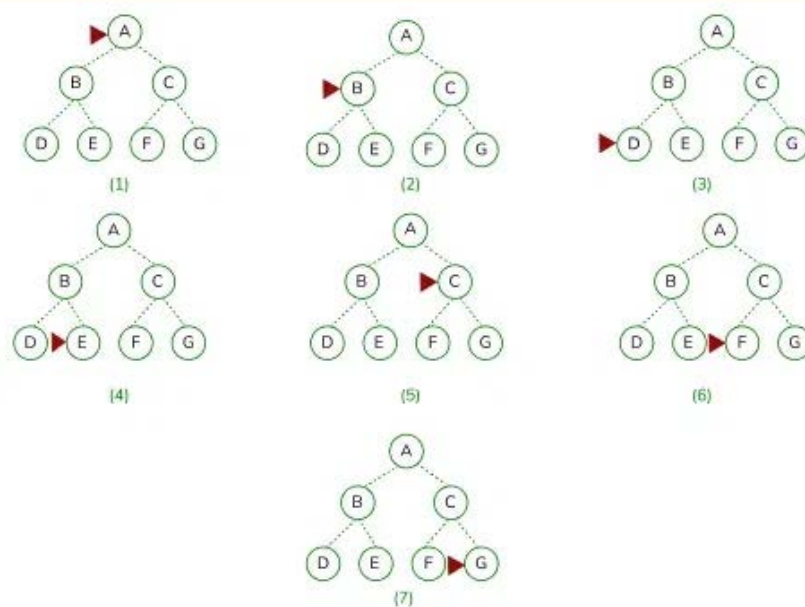
从根节点出发：DFS从根节点开始，选择一个分支进行深入探索，直到遇到死胡同。

二

回溯到未探索节点：当遇到死胡同时，DFS会回溯到最近的未探索节点，继续探索其他路径。

三

重复直到所有节点访问：这个过程会一直重复，直到所有节点都被访问过，或者找到满足条件的节点为止。



DFS四大核心特征

不是成本最优

DFS不保证找到最短路径，但它可能快速找到一个可行解，这在某些场景下非常有用。

回溯搜索变体

回溯搜索是DFS的一种变体，它通过一次生成一个子节点来节省内存。

基于堆栈的搜索

DFS使用栈来记录待访问的节点，通过栈的后进先出特性实现回溯。

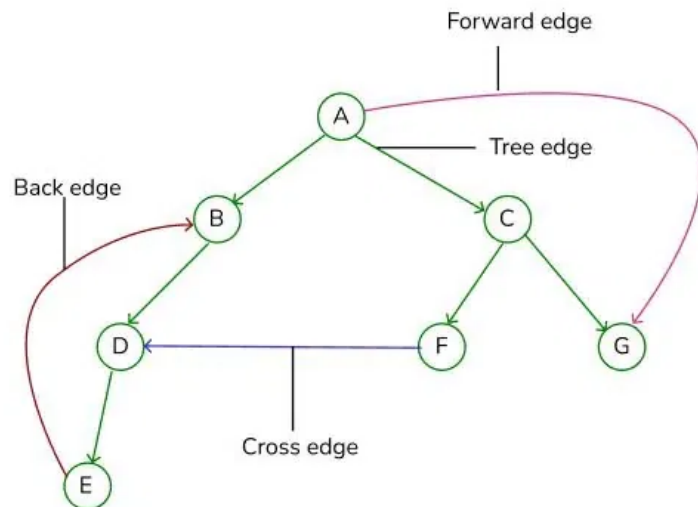
内存占用低

DFS的空间复杂度相对较低，因为它只需要存储当前路径上的节点。

DFS生成树边分类

树边

后向边



前向边

交叉边

时间、空间复杂度分析

显式时间复杂度

在显式图中，DFS的时间复杂度为 $O(|V| + |E|)$ ，其中 $|V|$ 是顶点数， $|E|$ 是边数。

隐式时间复杂度

在隐式图中，DFS的时间复杂度为 $O(b^d)$ ，其中 b 是分支因子， d 是搜索深度。

显式空间复杂度

在显式图中，DFS的空间复杂度为 $O(|V|)$ ，因为需要存储所有顶点的访问状态。

隐式空间复杂度

在隐式图中，DFS的空间复杂度为 $O(bd)$ ，主要取决于当前路径上的节点数。

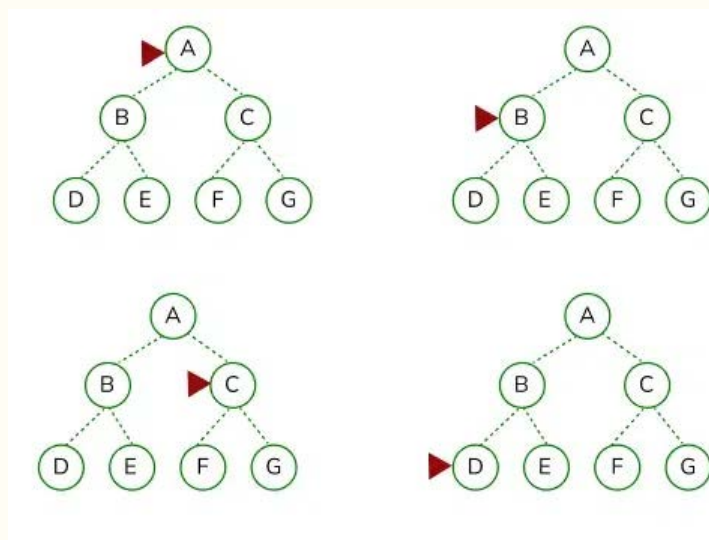
广度优先搜索

Depth First Search

01

BFS定义与作用

广度优先搜索（BFS）是一种按层级逐层向外扩散的遍历策略，在人工智能中，它为树或图结构提供最短路径保证与系统性状态空间探测手段。它从根节点出发，先访问所有邻居再深入下一层，确保路径探索的全面性。



BFS流程

一

从根节点出发：BFS从根节点开始，首先将根节点加入队列，标记为已访问。这是搜索的起点，所有后续的节点都将从这里开始逐层展开。

二

逐层访问节点：BFS按照队列的顺序，逐层访问每个节点的所有邻居节点，并将未访问的邻居节点加入队列。这一过程确保了搜索的广度优先性。

三

终止条件：BFS的搜索过程会一直持续，直到队列为空或找到目标节点为止。在找到目标节点时，搜索路径即为最短路径。

BFS三大特征

特征一

FIFO队列驱动：BFS使用先进先出（FIFO）队列来管理待访问的节点。新节点被加入队列的尾部，而访问节点则从队列的头部移除，确保了节点的访问顺序。

特征二

早期目标测试：在传统的广度优先搜索（BFS）中，算法会跟踪搜索过程中达到的状态。它不会存储所有状态，而是仅保留早期目标测试所需的状态，以便在新生成的节点创建后立即检查其是否满足目标。

特征三

成本最优保证：在未加权图中，BFS保证找到从起点到终点的最短路径。这是因为BFS会按照路径长度的顺序逐层扩展节点，首次到达目标节点的路径即为最短路径。



时间、空间复杂度分析

时间复杂度

BFS的时间复杂度为 $O(|V|+|E|)$ ，其中 $|V|$ 表示图中顶点的数量， $|E|$ 表示边的数量。这是因为BFS需要遍历每个顶点和每条边一次。

空间复杂度

BFS的空间复杂度为 $O(|V|)$ ，这是因为在最坏情况下，队列中可能需要存储所有顶点。这使得BFS在空间占用上相对较高。

与DFS对比

与深度优先搜索（DFS）相比，BFS的空间复杂度更高，因为DFS只需要存储当前路径上的节点，而BFS需要存储整层的节点。

BFS在人工智能中的应用

寻路算法

BFS通常用于在无权图中查找最短路径。通过逐层探索节点，BFS可以保证首次到达目的地时找到最短路径。示例：BFS在迷宫求解、机器人运动和路线规划中非常有用。

网络爬虫

BFS是网页爬取的基础，它根据页面的链接深度系统地进行访问。从根URL开始，它会先探索页面上的所有超链接，然后再深入到链接页面。此策略可确保数据收集有组织、逐级，尤其适用于构建搜索引擎索引或从网站提取结构化数据。

社交网络分析

在LinkedIn等平台中，BFS用于分析关系，例如识别分离度、寻找共同好友或追踪用户之间的连接路径。它还用于定位网络中的关键影响者或集群，通过广度遍历用户连接，洞察复杂的社会结构。

游戏中的人工智能

BFS常用于游戏人工智能，探索可能的状态或确定最佳移动路径。例如，在像吃豆人这样的游戏中，广度优先搜索（BFS）有助于计算鬼魂追逐吃豆人或吃豆人逃脱的最短路径。

BFS优势与趋势

BFS的核心优势

BFS在未加权图中保证找到最短路径，实现简单，适用于多种场景。

局限性

BFS的局限性在于高内存占用，深层节点搜索效率较低，且无法直接处理加权图。

未来发展方向

BFS未来可与双向搜索结合降低内存占用，与启发式剪枝融合提升效率。

与并行计算协同

BFS还可以与并行计算协同，加速大规模图的遍历，拓展其应用边界。

深度限制搜索

Depth Limited Search

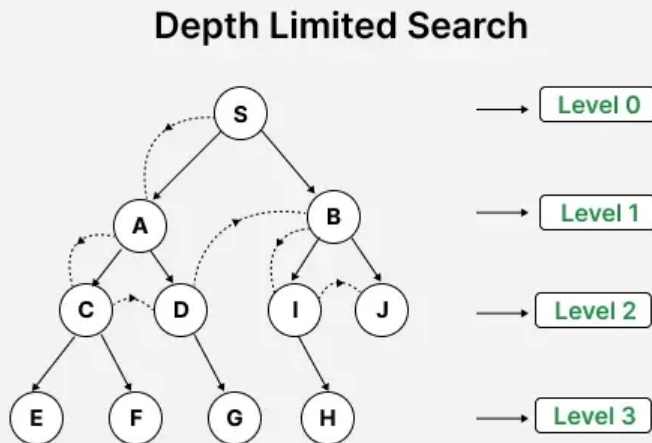
01

DFS困境与DLS诞生

01

DFS的局限性

深度优先搜索（DFS）在面对无限或极深的搜索空间时，容易陷入无意义的长路径，浪费大量计算资源。例如在迷宫寻路中，DFS可能会沿着一条死胡同一直探索到底，导致搜索效率低下。



02

DLS的改进思路

深度有限搜索（DLS）通过人为设定一个深度上限，解决了DFS的这一问题。它在DFS的基础上加入了一个深度计数器，当搜索深度达到预设的限制时，DLS会强制停止探索当前路径并回溯，从而避免了无谓的深入，提高了搜索效率。

DLS核心思想

深度限制的作用

DLS的核心在于深度限制，它像一条安全绳，防止搜索掉入过深的分支。通过设定深度上限，DLS可以有效地控制搜索范围，避免在无限或极深的搜索空间中迷失方向，确保算法在有限时间内给出结果。

保留DFS优势

DLS保留了DFS的内存优势，其内存占用仅为 $O(d)$ ，其中 d 是深度限制。这使得DLS在处理大规模搜索空间时，仍然能够保持较低的内存消耗，适用于资源受限的环境。

强制回溯机制

当搜索深度达到预设的限制时，DLS会强制回溯，而不是继续深入探索。这种机制确保了DLS不会陷入无解的路径，提高了搜索的效率和可靠性。

DLS执行流程

DLS的执行流程包括五步：初始化根节点与深度上限、逐层扩展子节点、实时检查深度是否超限、在限界内验证目标、超限或叶节点则回溯。通过这种方式，DLS能够在有限的深度内高效地搜索目标。

优势

内存效率

DLS的内存占用与DFS相同，仅为 $O(d)$ ，其中 d 是深度限制。这使得DLS在资源受限的环境中非常实用，能够高效地利用有限的内存资源。

防止无限循环

DLS通过深度限制防止了无限循环的发生，尤其适用于无限或循环的图结构。这确保了算法能够在有限的时间内完成搜索，避免了陷入无解的困境。

实现简单

DLS的实现非常简单，可以通过递归轻松实现。这种简洁的实现方式使得DLS易于理解和应用，适合在各种场景中快速部署。

迭代加深的基础

DLS是迭代加深搜索的基础组件。通过反复调用DLS并逐步增加深度限制，可以实现既完备又高效的搜索，兼顾了搜索的深度和效率。

局限性



不完备性

如果深度限制设置得过浅，DLS可能无法找到目标，导致搜索不完备。因此，深度限制的选择需要根据具体问题进行合理调整。



非最优解

DLS不保证找到最短路径。如果存在多个目标状态，DLS可能会找到一个较远的目标，而不是最近的目标。



深度选择的挑战

选择合适的深度限制并不总是容易的。如果深度限制设置得过高，可能会导致不必要的计算；如果设置得过低，则可能错过目标。

优化方向

1

与启发式结合

DLS可以与启发式函数结合，形成深度受限A*算法，进一步提高搜索效率，适用于更复杂的搜索问题。

2

动态深度调整

在动态环境中，DLS可以根据实时反馈自适应调整深度限制，提高搜索的灵活性和适应性。

3

并行探索

利用并行硬件，DLS可以同时探索多条深度受限的路径，显著提高搜索速度，适用于大规模搜索任务。

迭代加深搜索

Iterative Deepening Search

01

IDS诞生的背景与动机

BFS与DFS的局限

BFS在未知深度的复杂图中需存储整层节点导致内存爆炸，DFS虽省内存却无法保证最短路径。

IDS的动机

IDS通过逐轮加深深度限制，既保留DFS的线性内存优势，又兼具BFS的最优性。IDS继承了BFS的完备与最短路径保证，同时采用DFS的栈式遍历策略，每一轮仅保存当前路径，内存占用始终为 $O(bd)$ 。

问题背景与改进思路

在资源受限场景下，IDS能够有效平衡内存与最优性，为复杂搜索问题提供高效解决方案。

循环流程

01

循环流程一：初始化

从深度0开始，调用深度受限DFS，为搜索过程设定初始深度限制。

02

循环流程二：目标检测

若本轮找到目标立即返回，确保算法在找到解决方案时能够及时终止。

03

循环流程三：深度增加

否则深度加1进入下一轮，逐步扩大搜索范围。

04

循环流程四：重复执行

重复直至成功或穷尽搜索空间，确保算法的完备性。

优势

内存优势

IDS内存需求仅为 $O(bd)$ ，远小于BFS，适用于内存受限的场景。

最优性保证

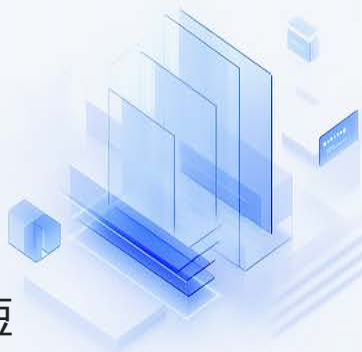
在无权图中，IDS能够保证找到最短路径，具备完备性。

完全搜索

与 BFS 类似，只要搜索空间有限，IDS 就能确保如果有解决方案，最终一定会找到它。

对深树有效

在目标节点位于大型搜索树深处的情况下，IDS 可以有效地找到解决方案，而无需过多的内存开销。



局限性

加权图局限

重复探索

IDS 会随着深度限制的增加而反复重新探索相同的节点。例如，随着深度限制的增加，根节点会被多次探索。这会导致冗余计算。

IDS 对于遍历边的成本不同的加权图表现不佳，因为它关注的是均匀深度而不是路径成本。

时间复杂度较高

虽然空间复杂度较低，但时间复杂度高于 DFS 或 BFS。IDS 的时间复杂度为 $O(b^d)$ ，其中 b 为分支因子， d 为解的深度。重复探索会增加总计算时间的开销。

均匀成本搜索

Uniform Cost Search

01

UCS诞生的背景与动机



01

传统算法局限

在加权图中，传统BFS只能保证步数最短，无法找到最低成本路径。Dijkstra算法虽能解决，但需预先知道全图且常实现为贪心，灵活性受限。

UCS弥补空白

UCS通过优先扩展累计成本最小的节点，确保在任意正权图中找到从起点到终点的最低成本路径，弥补了BFS与Dijkstra之间的空白。



02



03

动态适应性

UCS在动态或部分未知图中表现出色，能实时根据已知信息调整搜索方向，灵活适应环境变化，适用于多种场景。

BFS仅适用于单位权图，Dijkstra需预先知道全图且常实现为贪心，UCS则通过优先队列实时维护最小累积成本，适用于动态或部分未知图。

优先级队列与累积成本

优先级队列

UCS使用最小堆存储节点，键值为从起点到该节点的累积成本。每次弹出成本最低的节点进行扩展，确保已扩展节点即为当前已知最短路径。

数据结构基础

优先级队列天然实现“先优后劣”的搜索顺序，是UCS高效性的数据结构基础，确保算法能快速找到最低成本路径。

流程

初始化

从根节点开始，将其添加到优先级队列中，累积成本为零，为搜索过程奠定基础。

节点扩展

从优先级队列中移除路径成本最低的节点，然后扩展该节点，探索其邻居节点，为寻找最低成本路径做准备。

探索邻居

对于扩展节点的每个邻居，计算从起始节点经过当前节点到邻居的总成本，若邻居节点不在优先级队列中，则将其添加到已计算成本的队列中。

目标检查

扩展节点后，检查是否已到达目标节点。若已到达，返回路径和总成本，终止搜索过程。



应用场景



地图导航

GPS系统利用UCS在加权道路网中寻找最快路线，为用户提供最优出行方案。



网络路由

通信网络通过UCS选择延迟最低的路由，优化数据传输效率。



解谜

滑动方块谜题将每步代价设为1，UCS自动给出最少移动方案，解决智力挑战。

优势与局限



优势

UCS保证最低成本路径，适用于任意正权图，实现简单易于调试，在需要严格最优解且资源允许的场景中是首选。



空间复杂度

空间复杂度随节点数线性增长，优先级队列可能占用大量内存，需优化存储结构。



时间复杂度

时间复杂度在最坏情况下为 $O(E \log V)$ ，在大规模图中搜索耗时显著，需探索加速策略。

优化方向

1

与A*结合

引入启发式以减少扩展节点，提高搜索效率。

2

并行优先级队列

利用并行优先级队列加速大规模图搜索，提升算法性能。

3

动态权重环境

在动态权重环境中增量更新路径，适应实时变化。

双向搜索

Bidirectional Search

01

双向搜索定义与动机

01

双向搜索的定义

双向搜索是一种高效的人工智能搜索算法，它通过同时从初始状态和目标状态进行搜索，并在中间相遇来缩短搜索时间。这种策略特别适合于大型搜索空间，能够显著减少搜索所需的节点数量。

02

双向搜索的动机

在传统的单向搜索中，搜索空间可能呈指数级增长，导致效率低下。双向搜索通过两端同时搜索，将复杂度从 $O(b^d)$ 降低到 $O(b^{\{d/2\}})$ ，从而显著提高了搜索效率。

双向搜索核心思想

1

两端夹击策略

双向搜索的核心在于两端夹击，即从起点和终点同时启动搜索。这种策略可以将原本可能呈指数增长的搜索树压缩成两个较小的子树，从而减少搜索空间。

2

搜索树的压缩

通过同时从起点和终点进行搜索，双向搜索可以将搜索树的深度减半，从而显著减少需要访问的节点数量。这种策略在大型搜索空间中尤为有效。

3

效率提升的关键

双向搜索的效率提升主要来自于减少的节点访问数量。通过两端同时搜索，可以在更短的时间内找到目标节点，从而提高搜索效率。

双向搜索四步流程

初始化搜索

双向搜索的第一步是初始化两个搜索队列，一个从初始状态开始，另一个从目标状态开始。同时，初始化两个已访问集合和父节点映射。

检查交点

每次扩展后，检查新生成的节点是否已经出现在对面搜索的已访问集合中。如果找到交点，则搜索成功。

交替扩展节点

在双向搜索中，两个搜索交替进行，每次从队列中取出一个节点，生成其所有可能的后继节点或前驱节点，并将其加入队列和已访问集合。

路径拼接

一旦找到交点，通过回溯父节点映射，从起点到交点和从交点到终点的路径可以被拼接起来，形成完整的解路径。

双向搜索的关键在于相遇点的判定。每次扩展后，检查新生成的节点是否已出现在对面搜索的已访问集合中。一旦找到交点，立即拼接前向路径与反向路径，形成从起点到终点的完整解。

优势与局限

效率提升

双向搜索将搜索深度减半，显著减少节点访问数量，从而提高搜索效率。

最优性保证

当与统一代价搜索策略结合时，双向搜索可以保证找到最优解，适用于需要精确路径的场景。

内存优化

双向搜索通常比单向广度优先搜索占用更少内存，尤其在密集连接的图或大型搜索空间中表现突出。

实施复杂性

双向搜索需要同时维护两套队列、已访问集合与父映射，代码复杂度高于单向搜索。

边界管理

必须精细管理两端边界，防止重复扩展。相遇判定需高效哈希或集合操作，否则可能成为性能瓶颈。

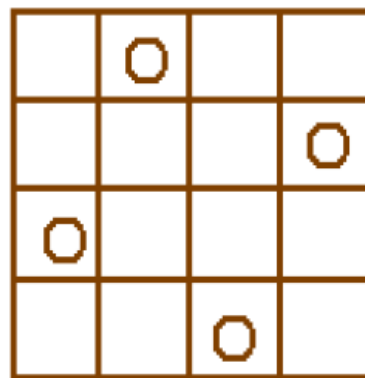
回溯与分支限界

4后问题

4后问题：在 4×4 的方格棋盘上放置4个皇后，使得没有两个皇后在同一行、同一列、也不在同一条45度的斜线上。问有多少种可能的布局？

解是 4 维向量 $\langle x_1, x_2, x_3, x_4 \rangle$

解： $\langle 2, 4, 1, 3 \rangle$, $\langle 3, 1, 4, 2 \rangle$

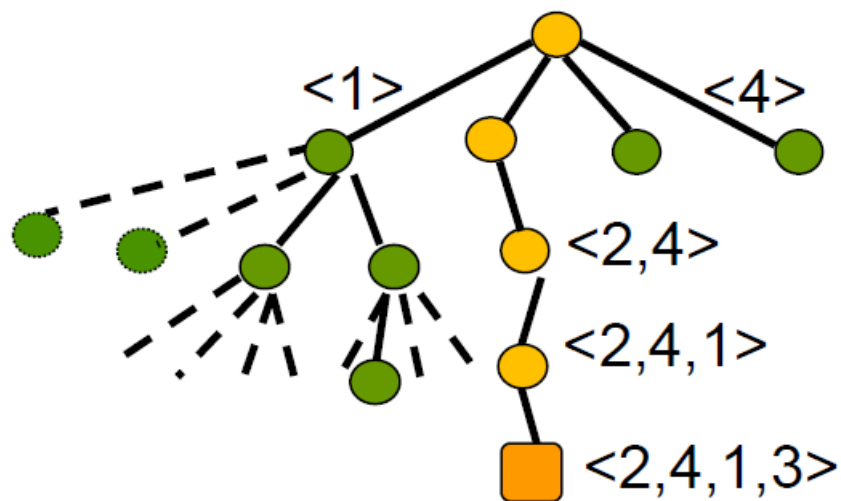


推广到8后问题

解：8维向量，有 92个。

例如： $\langle 1, 5, 8, 6, 3, 7, 2, 4 \rangle$ 是解。

搜索空间：4叉树



每个结点有4个儿子，分别代表选择
1,2,3,4列位置

第 i 层选择解向量中第 i 个分量的值

最深层的树叶是解

按深度优先次序遍历树，找到所有解

以 4 皇后问题为例,上述算法执行的部分过程如下:

初始: $X_1 = X_2 = X_3 = X_4 = \{1, 2, 3, 4\}$.

$k=1, S_1 = \{1, 2, 3, 4\}$, 取 $x_1 = 1$, 修改 $S_1 = \{2, 3, 4\}$.

$k=2, S_2 = \{3, 4\}$, 取 $x_2 = 3$, 修改 $S_2 = \{4\}$.

$k=3, S_3 = \emptyset$, 回溯.

$k=2, S_2 = \{4\}$, 取 $x_2 = 4$, 修改 $S_2 = \emptyset$.

$k=3, S_3 = \{2\}$, 取 $x_3 = 2$, 修改 $S_3 = \emptyset$.

$k=4, S_4 = \emptyset$, 回溯.

$k=3, S_3 = \emptyset$, 回溯.

$k=2, S_2 = \emptyset$, 回溯.

$k=1, S_1 = \{2, 3, 4\}$, 取 $x_1 = 2$, 修改 $S_1 = \{3, 4\}$

$k=2, S_2 = \{4\}$, 取 $x_2 = 4$, 修改 $S_2 = \emptyset$.

$k=3, S_3 = \{1\}$, 取 $x_3 = 1$, 修改 $S_3 = \emptyset$.

$k=4, S_4 = \{3\}$, 取 $x_4 = 3$, 修改 $S_4 = \emptyset$, 得到解 $\langle 2, 4, 1, 3 \rangle$.

$k=3, S_3 = \emptyset$, 回溯.

$k=2, S_2 = \emptyset$, 回溯.

$k=1, S_1 = \{3, 4\}$, 取 $x_1 = 3$, 修改 $S_1 = \{4\}$.

回溯算法递归实现

算法 **ReBack** (k)

1. if $k > n$ then $\langle x_1, x_2, \dots, x_n \rangle$ 是解
2. else while $S_k \neq \emptyset$ do
3. $x_k \leftarrow S_k$ 中最小值
4. $S_k \leftarrow S_k - \{x_k\}$
5. 计算 S_{k+1}
6. **ReBack** ($k+1$)

算法 **ReBacktrack** (n)

输入: n

输出: 所有的解

1. for $k \leftarrow 1$ to n 计算 X_k 且 $S_k \leftarrow X_k$
2. **ReBack** (1)

迭代实现

迭代算法 Backtrack

输入: n

输出: 所有的解

1. 对于 $i=1, 2, \dots, n$ 确定 X_i

2. $k \leftarrow 1$

3. 计算 S_k

4. while $S_k \neq \emptyset$ do

5. $x_k \leftarrow S_k$ 中最小值; $S_k \leftarrow S_k - \{x_k\}$

6. if $k < n$ then

7. $k \leftarrow k+1$; 计算 S_k

8. else $\langle x_1, x_2, \dots, x_n \rangle$ 是解

9. if $k > 1$ then $k \leftarrow k-1$; goto 4

确定初
始取值

满足约束
分支搜索

回溯

问题分析

问题	解性质	解描述 向量	搜索 空间	搜索 方式	约束 条件
n 后	可行 解	$\langle x_1, x_2, \dots, x_n \rangle$ x_i : 第 i 行列号	n 叉 树	深度, 宽 度优先	彼此不 攻击
0-1 背包	最优 解	$\langle x_1, x_2, \dots, x_n \rangle$ $x_i = 0, 1,$ $x_i = 1 \Leftrightarrow$ 选 i	子集 树	深度, 宽 度优先	不超背包 重量
货郎	最优 解	$\langle i_1 = 1, i_2, \dots, i_n \rangle$ $1, 2, \dots, n$ 的排列	排列 树	深度, 宽 度优先	选没有经 过的城市
特点	搜索 解	向量, 不断扩 张部分向量	树	跳跃式 遍历	约束条件 回溯判定

装载问题

问题描述：有 n 个集装箱要装上2艘载重量分别为 C_1 和 C_2 的轮船，其中集装箱 i 的重量为 W_i ，且 $\sum W_i \leq C_1 + C_2$ 。请设计算法确定是否有一个合理的装载方案可将这些集装箱上这2艘轮船。

输入：多组测试数据。每组测试数据包括两行：第一行输入集装箱数目 n （ $n < 1000$ ），以及两艘轮船的载重 C_1 和 C_2 ；第二行输入 n 个整数，表示每个集装箱的重量。

输出：如果存在合理装载方案，输出第一艘轮船的最大装载重量；否则，输出“No”。

输入样例：

3 50 50

10 40 40

3 50 50

20 40 40

输出样例

50

No

把原问题简化为一艘轮船的装载问题。容易证明下述结论：如果一个给定装载问题实例有解，则采用下面的策略可得到最优装载方案：

(1) 首先将第一艘轮船**尽可能装满**；

(2) 将剩余的集装箱装上第二艘轮船。

问题分析

装载问题等价于以下特殊的0-1背包问题：

$$\begin{aligned} & \max \sum_{i=1}^n w_i x_i \\ & st. \quad \begin{cases} \sum_{i=1}^n w_i x_i \leq C_1 \\ x_i \in \{0,1\}, 1 \leq i \leq n \end{cases} \end{aligned}$$

1. 问题的状态表示

$$X = [x_1, x_2, \dots, x_k]$$

2. 约束条件

$$\sum_{i=1}^k x_i w_i \leq C_1, x_i \in \{0, 1\}$$

3. 操作符

给定状态 $X = [x_1, x_2, \dots, x_k]$ ，可能的操作有两种：

① 把第 $k+1$ 个集装箱装上轮船，有 $x_{k+1} = 1$

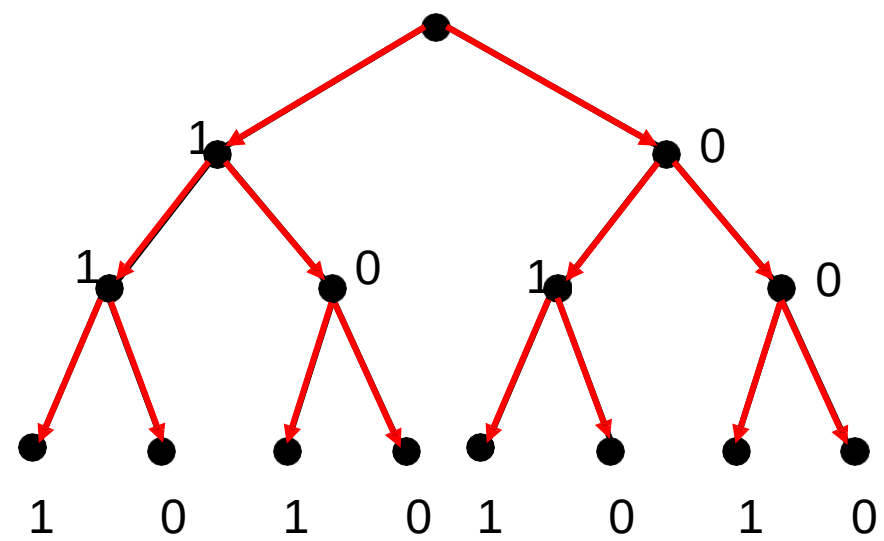
② 把第 $k+1$ 个集装箱不装轮船，有 $x_{k+1} = 0$

得到新的状态 $X' = [x_1, x_2, \dots, x_k, x_{k+1}]$

4. 问题解和解空间

满足约束条件的任何 n 维0-1向量 $[x_1, x_2, \dots, x_n]$ 都是一个可行解，解空间可以用一棵二叉树来表示

深度优先搜索演示



8个状态全部访问一遍，选择最大且小于**C1**的结果输出

伪码

Loading (W, c_1),

输入：集装箱重量 $W = \langle w_1, w_2, \dots, w_n \rangle$, c_1 是第一条船的载重

输出：使第一条船装载最大的方案 $\langle x_1, x_2, \dots, x_n \rangle$, 其中 $x_i = 0, 1$

1. Sort(W); //对 $w_1, w_2, \dots, w_n$ 按照从大到小排序
2. $B \leftarrow c_1$; $best \leftarrow c_1$; $i \leftarrow 1$; // B 为当前空隙, $best$ 最小空隙
3. while $i \leq n$ do
4. if 装入 i 后重量不超过 c_1
5. then $B \leftarrow B - w_i$; $x[i] \leftarrow 1$; $i \leftarrow i + 1$;
6. else $x[i] \leftarrow 0$; $i \leftarrow i + 1$;
7. if $B < best$ then 记录解; $Best \leftarrow B$; $i \leftarrow i - 1$;
8. Backtrack(i);
9. if $i = 1$ then return 最优解
10. else goto 3.

子过程 Backtrack

算法 Backtrack(i)

1. while $i > 1$ and $x[i] = 0$ do

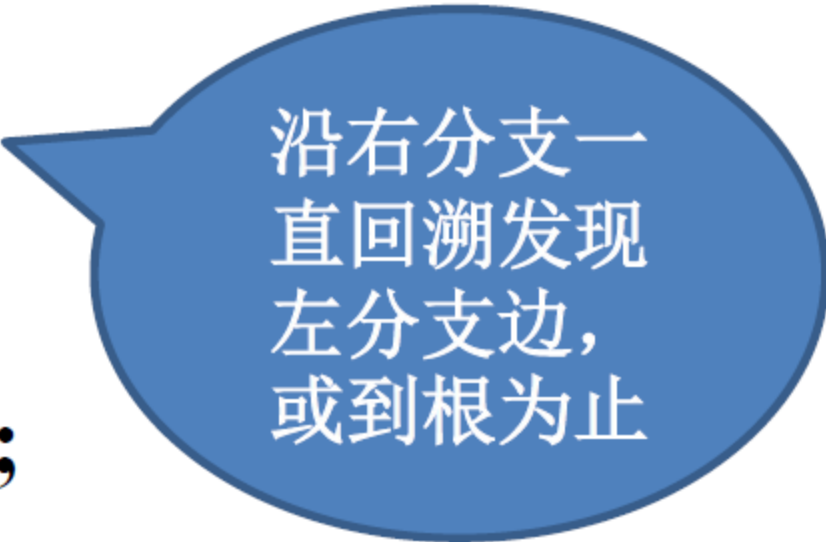
2. $i \leftarrow i - 1$;

3. if $x[i] = 1$

4. then $x[i] \leftarrow 0$;

5. $B \leftarrow B + w_i$;

6. $i \leftarrow i + 1$.



沿右分支一
直回溯发现
左分支边，
或到根为止

实例

$$W = \langle 90, 80, 40, 30, 20, 12, 10 \rangle$$

$$c_1 = 152, c_2 = 130$$

解:

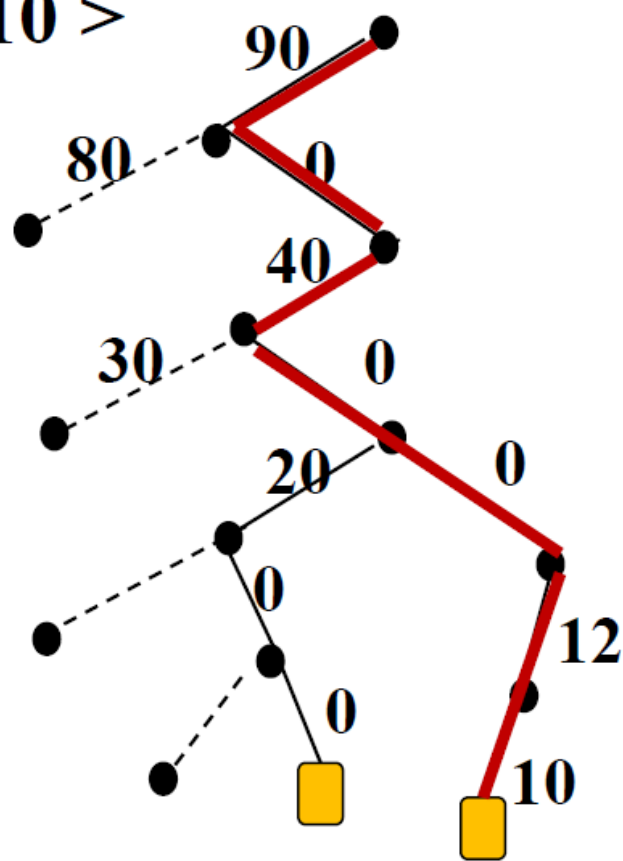
可以装, 方案如下:

1, 3, 6, 7 装第一艘船

2, 4, 5 装第二艘船

时间复杂性:

$$W(n) = O(2^n)$$



回溯算法基本思想

- (1) **适用**：求解搜索问题和优化问题.
- (2) **搜索空间**：树，结点对应部分解向量，可行解在树叶上.
- (3) **搜索过程**：采用系统的方法隐含遍历搜索树.
- (4) **搜索策略**：深度优先，宽度优先，函数优先，宽深结合等.

回溯算法基本思想(续)

(5) **结点分支判定条件:**

满足约束条件---分支扩张解向量
不满足约束条件, 回溯到该结点的父结点.

(6) **结点状态:** 动态生成

白结点(尚未访问)

灰结点(正在访问该结点为根的子树)

黑结点(该结点为根的子树遍历完成)

(7) **存储:** 当前路径

结点状态

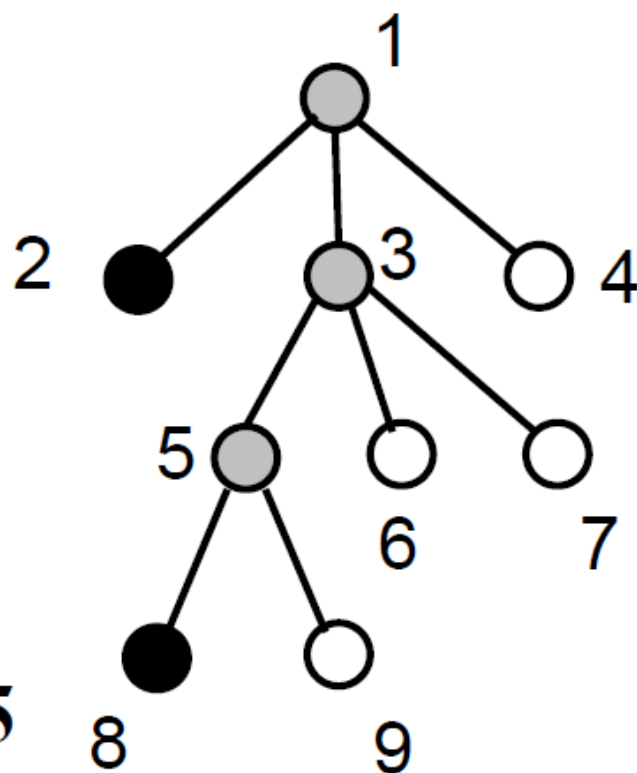
深度优先
访问次序:

1→2→3→5→8

已完成访问: 2, 8

已访问但未结束: 1, 3, 5

尚未访问: 9, 6, 7, 4



回溯算法的适用条件

在结点 $\langle x_1, x_2, \dots, x_k \rangle$ 处

$P(x_1, x_2, \dots, x_k)$ 为真

$\Leftrightarrow$ 向量 $\langle x_1, x_2, \dots, x_k \rangle$ 满足某个性质
(n 后中 k 个皇后放在彼此不攻击的位置)

多米诺性质

$$P(x_1, x_2, \dots, x_{k+1}) \rightarrow P(x_1, x_2, \dots, x_k) \quad 0 < k < n$$

$$\neg P(x_1, x_2, \dots, x_k) \rightarrow \neg P(x_1, x_2, \dots, x_{k+1}) \quad 0 < k < n$$

k 维向量不满足约束条件, 扩张向量到
 $k+1$ 维仍旧不满足, 可以回溯.

一个反例

例 求不等式的整数解

$$5x_1 + 4x_2 - x_3 \leq 10, \quad 1 \leq x_k \leq 3, \quad k=1,2,3$$

$P(x_1, \dots, x_k)$: 将 $x_1, x_2, \dots, x_k$ 代入原不等式的相应部分, 部分和小于等于10

不满足多米诺性质:

$$5x_1 + 4x_2 - x_3 \leq 10 \not\Rightarrow 5x_1 + 4x_2 \leq 10$$

变换使得问题满足多米诺性质:

令 $x_3 = 3 - x_3'$,

$$5x_1 + 4x_2 + x_3' \leq 13$$

$$1 \leq x_1, x_2 \leq 3, \quad 0 \leq x_3' \leq 2$$

小结

- 回溯算法的适用条件：
多米诺性质
- 回溯算法的设计步骤
 - (1) 定义解向量和每个分量的取值范围
解向量为 $\langle x_1, x_2, \dots, x_n \rangle$
确定 x_i 的取值集合为 $X_i, i=1, 2, \dots, n$.

小结（续）

- (2) 在 $\langle x_1, x_2, \dots, x_{k-1} \rangle$ 确定如何计算 x_k 取值集合 S_k , $S_k \subseteq X_k$
- (3) 确定结点儿子的排列规则
- (4) 判断是否满足多米诺性质
- (5) 确定每个结点分支的约束条件
- (6) 确定搜索策略: 深度优先, 宽度优先等
- (7) 确定存储搜索路径的数据结构

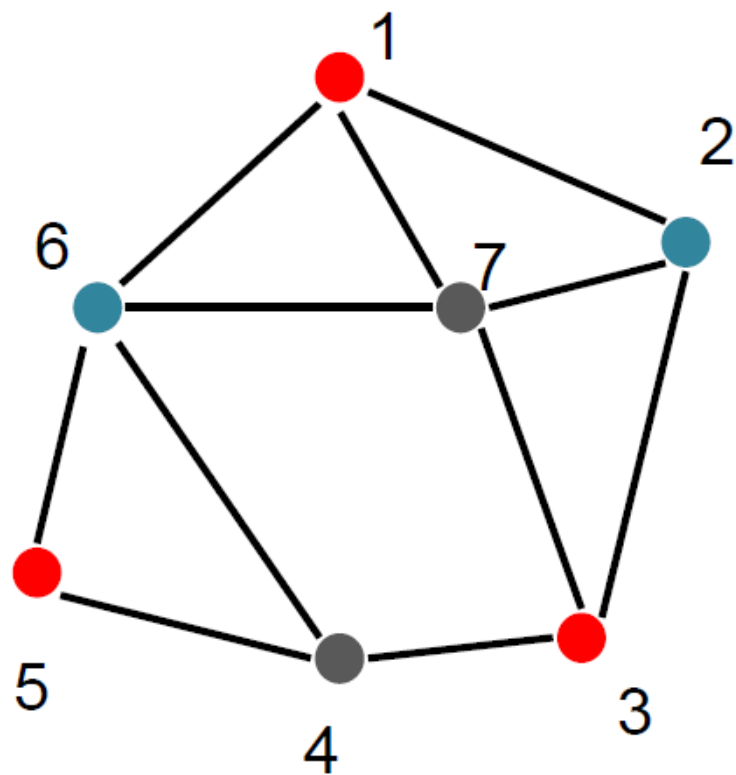
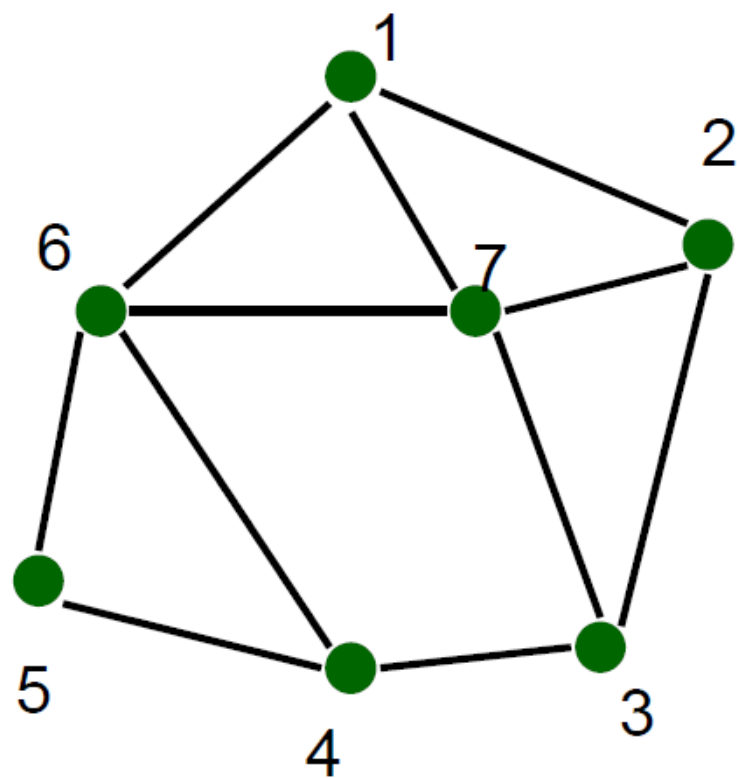
着色问题

输入:

无向连通图 G 和 m 种颜色的集合
用这些颜色给图的顶点着色, 每个
顶点一种颜色. 要求是: G 的每条
边的两个顶点着不同颜色.

输出: 所有可能的着色方案.
如果不存在着色方案, 回答 “No”.

实例



$$n=7, m=3$$

解向量

设 $G=(V,E)$, $V=\{1,2, \dots, n\}$

颜色编号: $1, 2, \dots, m$

解向量: $\langle x_1, x_2, \dots, x_n \rangle$,

$$x_1, x_2, \dots, x_n \in \{1, 2, \dots, m\}$$

结点的部分向量 $\langle x_1, x_2, \dots, x_k \rangle$

$$x_1, x_2, \dots, x_k, 1 \leq k \leq n,$$

表示只给顶点 $1, 2, \dots, k$ 着色的部分方案

算法设计

搜索树： m 叉树

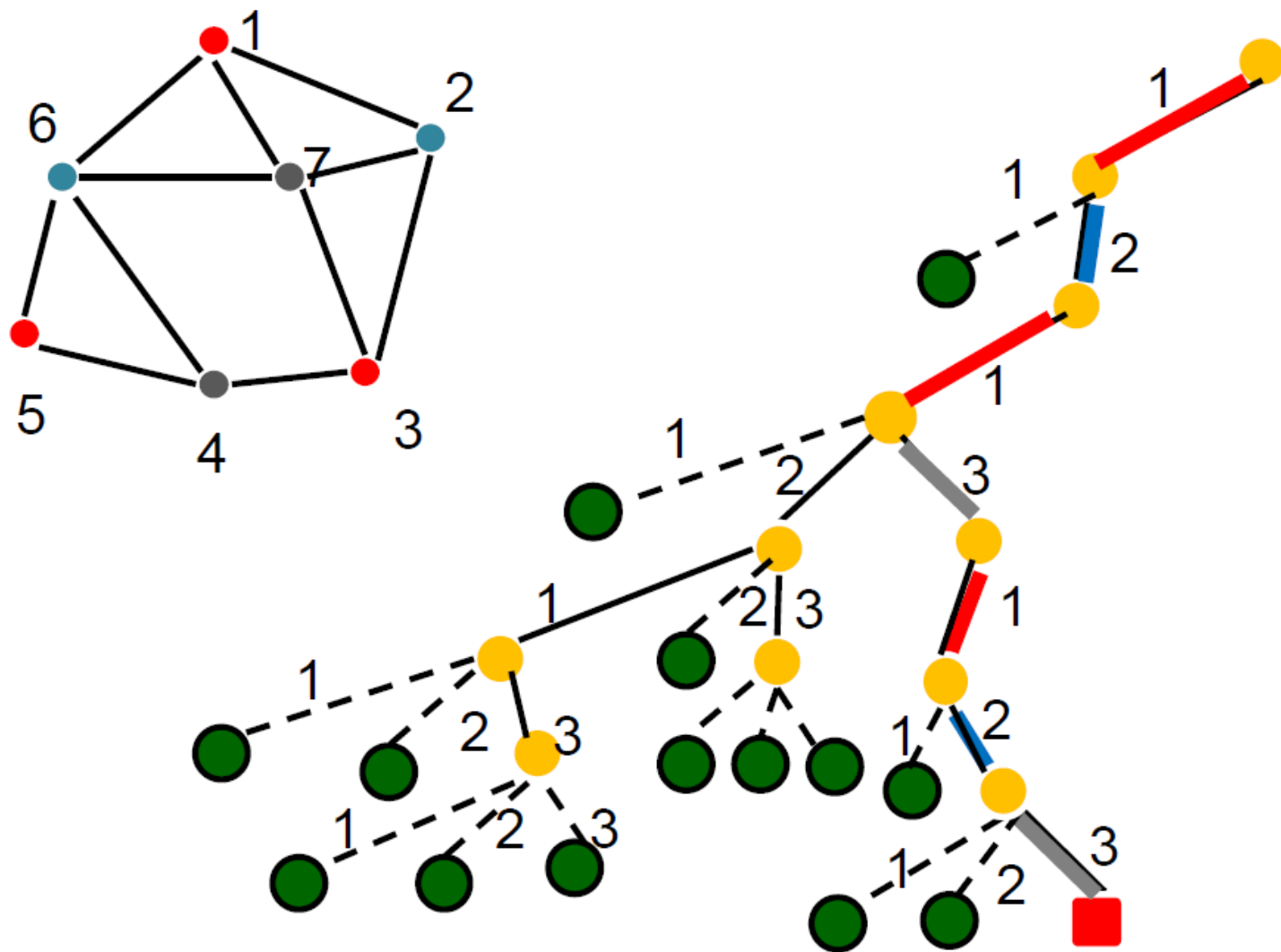
约束条件：在结点 $\langle x_1, x_2, \dots, x_k \rangle$ 处，
顶点 $k+1$ 的邻接表中结点已用过的颜色
不能再使用

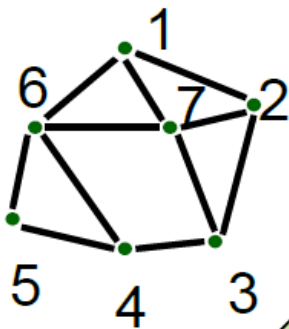
如果邻接表中结点已用过 m 种颜色，
则结点 $k+1$ 没法着色，从该结点回溯
到其父结点。满足多米诺性质

搜索策略：深度优先

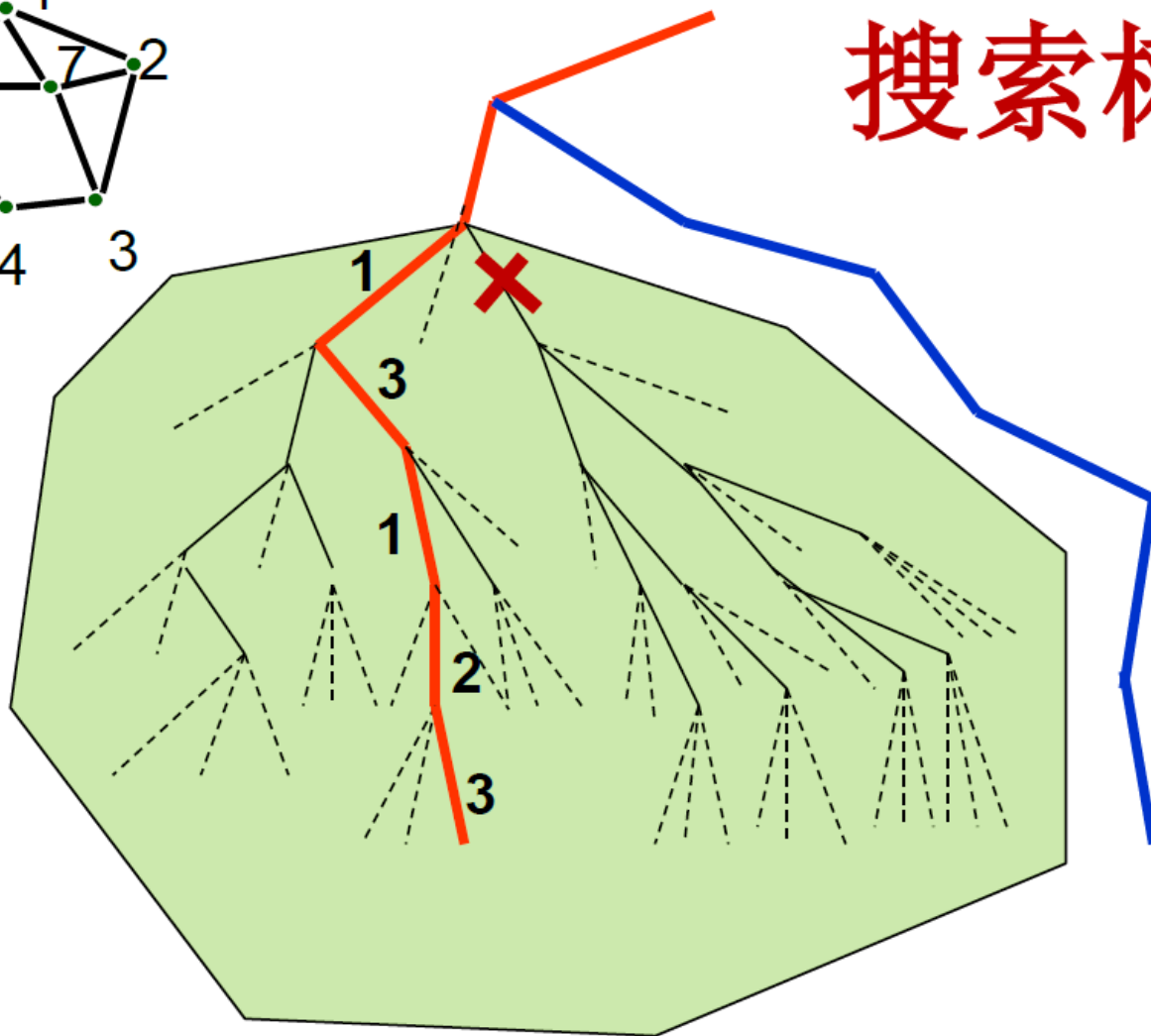
时间复杂度： $O(n m^n)$

运行实例





搜索树



第一个解向量: $\langle 1, 2, 1, 3, 1, 2, 3 \rangle$

时间复杂度与 改进途径

时间复杂度: $O(nm^n)$

根据对称性, 只需搜索 $1/3$ 的解空间. 当 1 和 2 确定, 即 $\langle 1, 2 \rangle$ 以后, 只有 1 个解, 在 $\langle 1, 3 \rangle$ 为根的子树中也只有 1 个解. 由于 3 个子树的对称性, 总共 6 个解.

在取定 $\langle 1, 2 \rangle$ 后, 不可扩张成 $\langle 1, 2, 3 \rangle$, 因为 7 和 1, 2, 3 都相邻. 7 没法着色. 可以从打叉的结点回溯, 而不必搜索其子树.

着色问题的应用

会场分配问题:

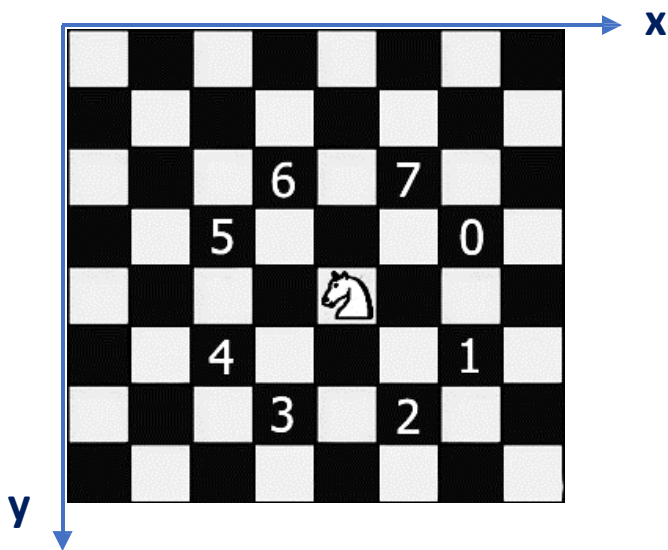
有 n 项活动需要安排, 对于活动 i, j , 如果 i, j 时间冲突, 就说 i 与 j 不相容. 如何分配这些活动, 使得每个会场的活动相容且占用会场数最少?

建模:

活动作为图的顶点, 如果 i, j 不相容, 则在 i 与 j 之间加一条边, 会场标号作为颜色标号. 求图的一种着色方案, 使得使用的颜色数最少.

马踏棋盘问题

- 国际象棋棋盘是 8×8 的方格，现将“马”放在任意指定的方格中，按照走棋规则移动该棋子。要求每个方格只能进入一次，最终走遍棋盘64个方格
- 棋盘可用一个矩阵表示，当“马”位于棋盘上某一位置时，它就有唯一的坐标。根据规则，如果当前坐标是 (x, y) ，那么它的下一跳可能有8个位置，分别是 $(x+2, y-1)$ 、 $(x+2, y+1)$ 、 $(x+1, y+2)$ 、 $(x-1, y+2)$ 、 $(x-2, y+1)$ 、 $(x-2, y-1)$ 、 $(x-1, y-2)$ 、 $(x+1, y-2)$ 。当然坐标不能越界



马踏棋盘问题

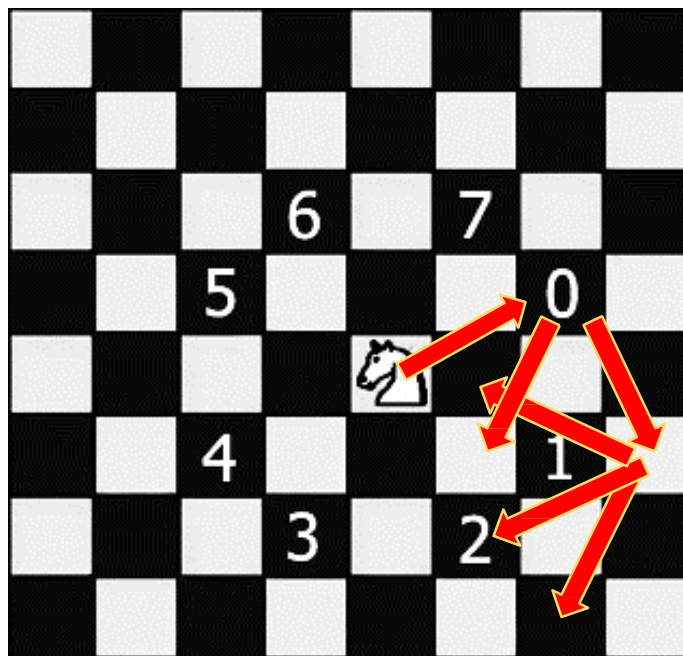
- 马最初的位置标为**1**，它的下一跳的位置标为**2**，再下一跳的位置标为**3**，依次类推，如果马走完棋盘，那么最后在棋盘上标的位置是**64**
- 要求编程解决马踏棋盘问题，最后输出一个**8*8**的矩阵，并用数字**1-64**来标注“马”的移动

27	18	29	32	25	20	3	50
30	33	26	19	2	51	24	21
17	28	31	58	55	22	49	4
34	57	46	1	52	41	54	23
45	16	63	56	59	48	5	40
64	35	60	47	42	53	8	11
15	44	37	62	13	10	39	6
36	61	14	43	38	7	12	9

马踏棋盘问题

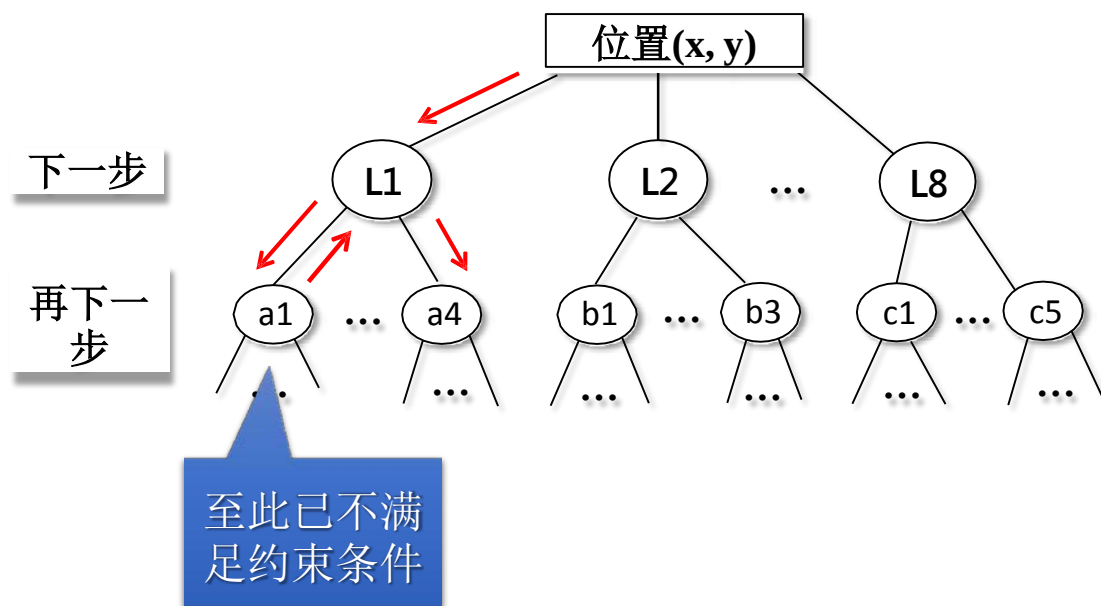
◆ 基本解法：深度优先搜索

从某位置出发，先试探下一个可能位置；进入一个新位置后就从该位置进一步试探下一位置，若遇到不可行位置则回退一步，然后再试探其他可能位置



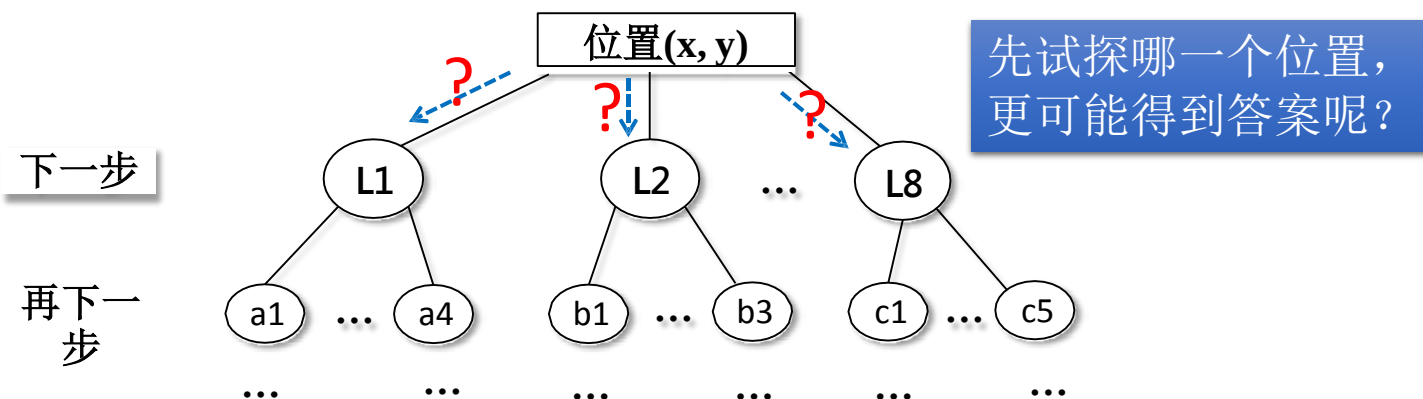
马踏棋盘——深度优先搜索

- ◆ 深度优先搜索求解，相当于在类似下面的树中查询



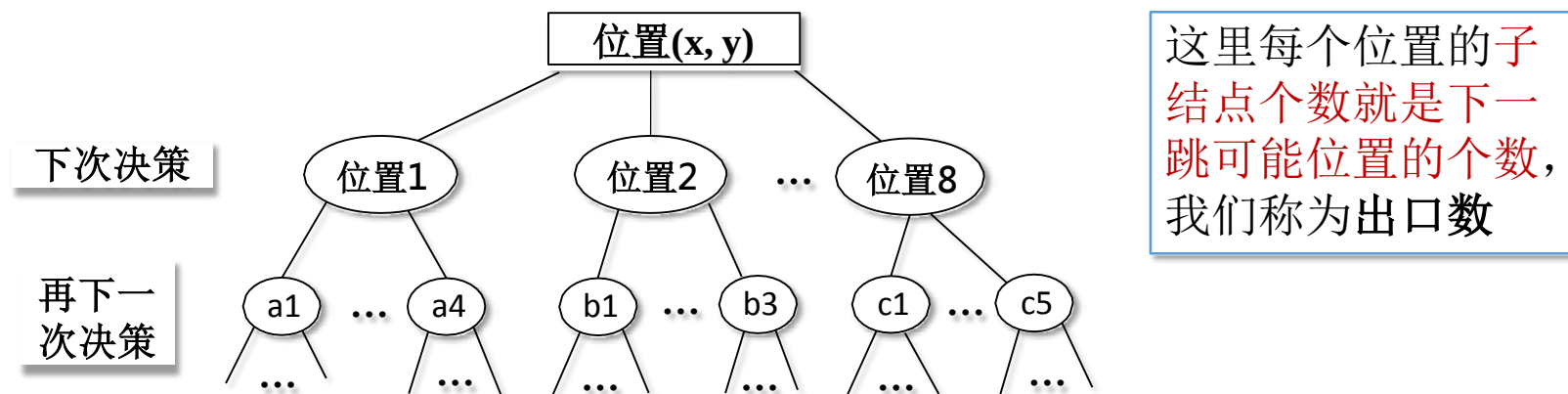
马踏棋盘——深度优先搜索 + 贪心法优化

- ◆ 从 (x,y) 开始，深度优先搜索会依次查询L1、L2、.....、L8一共8个可能位置
- ◆ 如何加入贪心策略？



- ◆ 事实上，先选择L1~L8中哪个位置试探，可利用L1~L8位置的子结点数确定

马踏棋盘——深度优先搜索 + 贪心法优化



贪心策略：在选择下一跳的位置时，总是先选择出口少的那个位置

?

- 如果优先选择出口多的子结点，那么出口少的子结点就会越来越多，很可能出现走不通的‘死’结点。这时只有回退再搜索，会浪费很多时间
- 反之如果优先选择出口少的结点，那出口少的结点就会越来越少，这样成功的机会就更大

搜索树结点数估计

Monte Carlo方法

1. 从根开始，随机选择一条路径，直到不能分支为止，即从 $x_1, x_2, \dots$ ，依次对 x_i 赋值，每个 x_i 的值是从当时的 S_i 中随机选取，直到向量不能扩张为止.
2. 假定搜索树的其他 $|S_i| - 1$ 个分支与以上随机选出的路径一样，计数搜索树的点数.
3. 重复步骤 1 和 2，将结点数进行概率平均.

伪码

Monte Carlo

输入: n 为皇后数, t 为抽样次数

输出: sum , 即 t 次抽样路长平均值

1. $sum \leftarrow 0$

2. for $i \leftarrow 1$ to t do // 取样次数 t

3. $m \leftarrow \text{Estimate}(n)$ // m 为结点数

4. $sum \leftarrow sum + m$

5. $sum \leftarrow sum / t$



一次抽
样结果

一次抽样

m 为本次取样得到的树结点总数

k 为层数

r_2 为上层结点数

r_1 为本层结点数

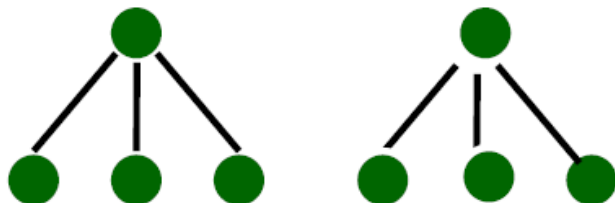
$r_1 = r_2 \cdot \text{分支数}$

n 为树的层数

从树根向下计算,随机选择,直到树叶.

$$r_2 = 2$$

$$r_1 = r_2 \cdot 3 = 6$$

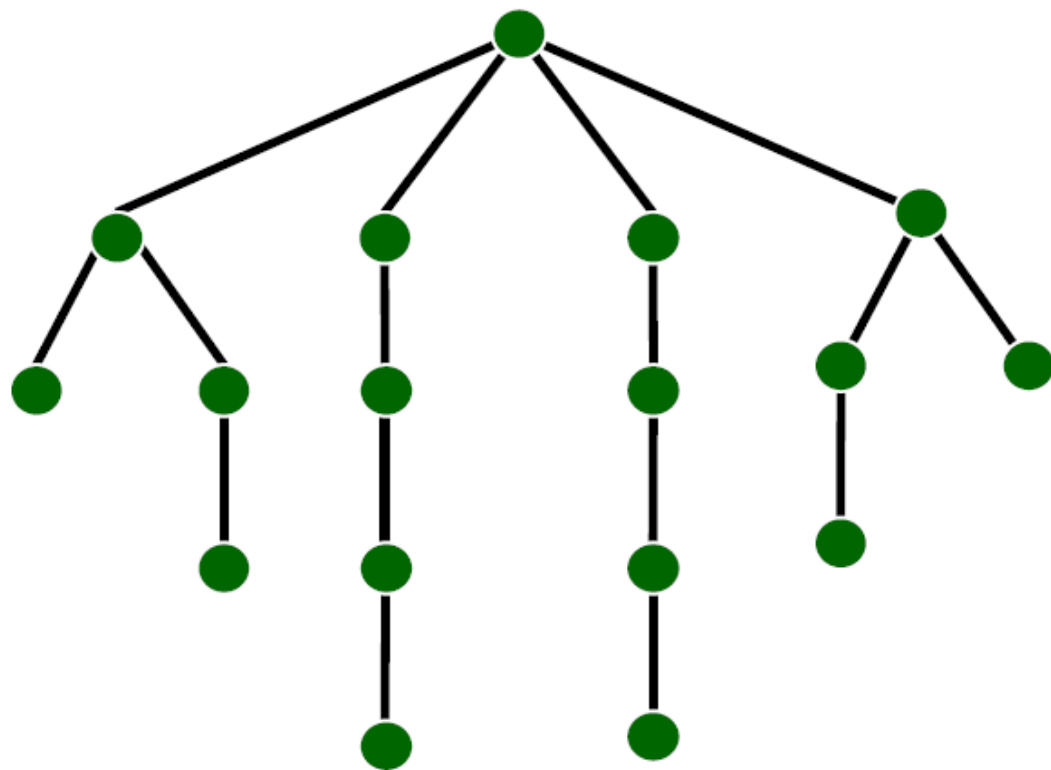


子过程的伪码

算法Estimate(n)

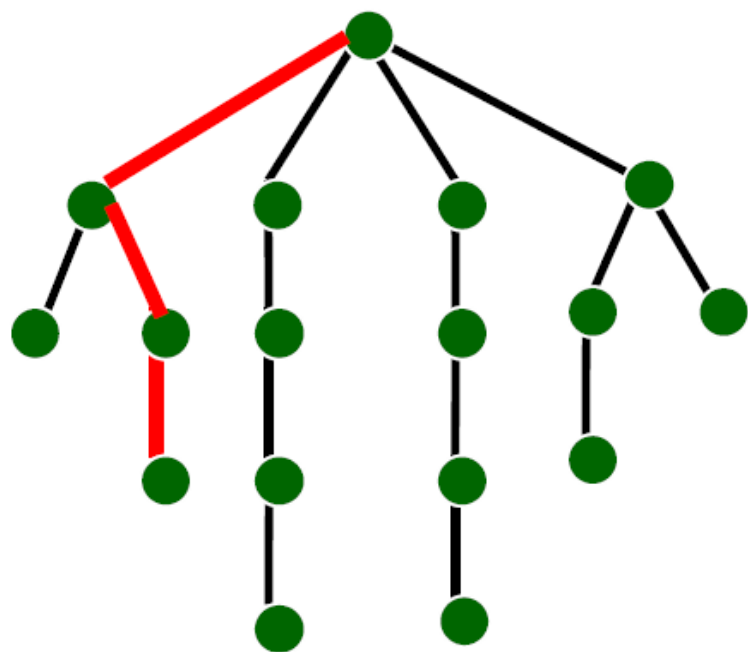
1. $m \leftarrow 1$; $r_2 \leftarrow 1$; $k \leftarrow 1$ // m 为结点总数
2. while $k \leq n$ do
3. if $S_k = \emptyset$ then return m 
4. $r_1 \leftarrow |S_k| * r_2$ // r_1 为扩张后结点总数
5. $m \leftarrow m + r_1$ // r_2 为扩张前结点总数
6. $x_k \leftarrow$ 随机选择 S_k 的元素
7. $r_2 \leftarrow r_1$ 
8. $k \leftarrow k + 1$

4后搜索树遍历的结点

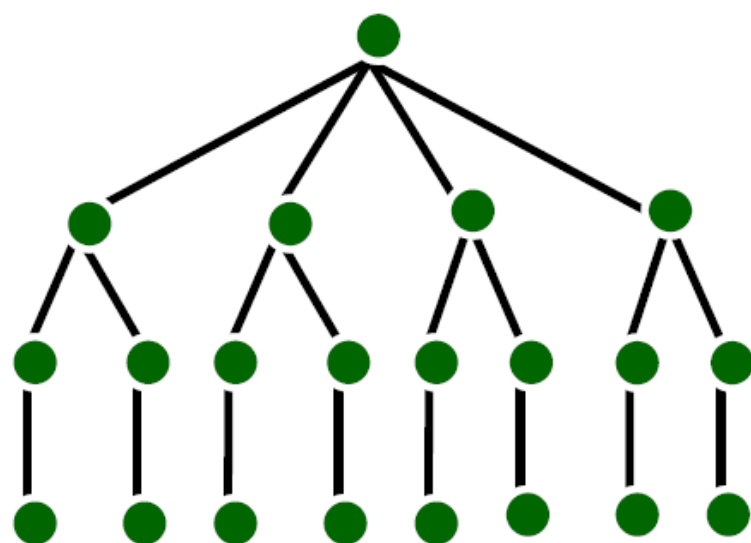


结点数=17

随机选择路径1

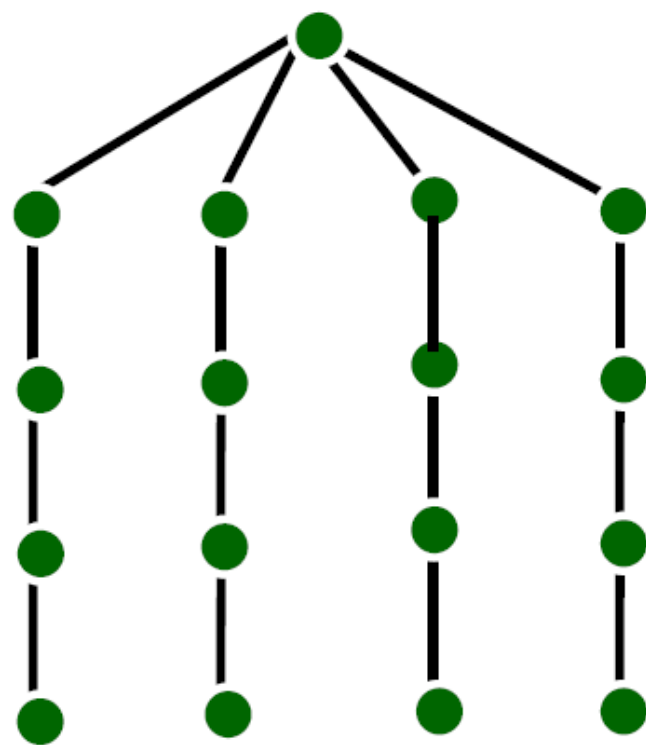
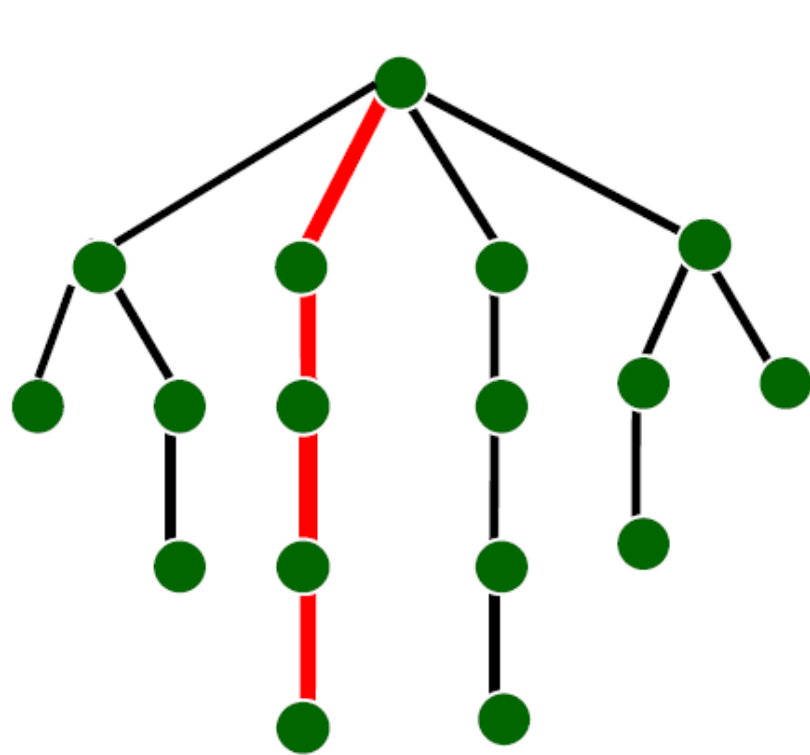


Case1: $\langle 1, 4, 2 \rangle$,



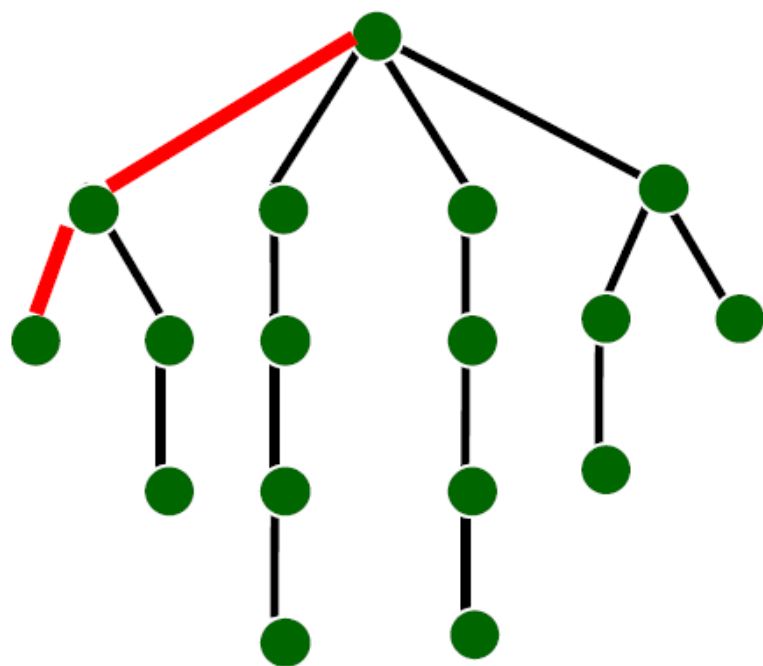
结点数 21

随机选择路径2

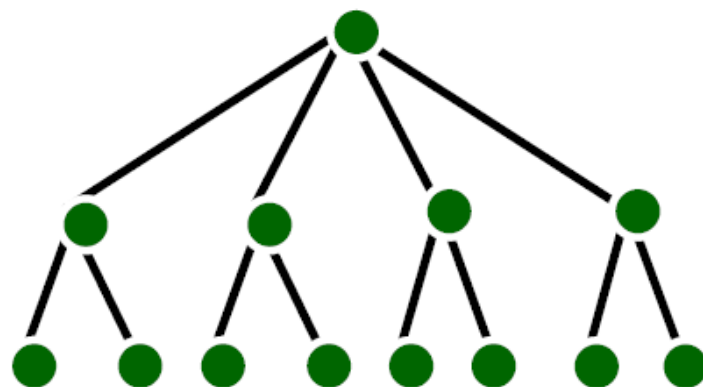


Case1: $\langle 2, 4, 1, 3 \rangle$, 结点数 17

随机选择路径3



Case3: $\langle 1,3 \rangle$,



结点数 13

估计结果

假设 4 次抽样测试:

case1: 1次,

case2: 1次,

case3: 2次,

平均结点数 $= (21 \times 1 + 17 \times 1 + 13 \times 2) / 4 = 16$

搜索空间访问的结点数为 17

影响算法效率的因素

最坏情况下的时间 $W(n) = O(p(n) f(n))$

其中 $p(n)$ 为每个结点时间， $f(n)$ 为结点个数

影响回溯算法效率的因素

搜索树的结构

分支情况：分支均匀否

树的深度

对称程度：对称适合裁减

解的分布

在不同子树中分布解的个数是否均匀

分布深度

约束条件的判断：计算简单

改进途径

根据树分支设计优先策略：

结点少的分支优先，解多的分支优先

利用搜索树的对称性剪裁子树

分解为子问题：

求解时间 $f(n) = c2^n$ ，组合时间 $T = O(2^{n/k})$

如果分解为 k 个子问题，每个子问题大小为 n/k

求解时间为

$$kc^{\frac{n}{k}} + T$$

分支限界法

广度优先搜索是一种依照“由近及远，按层展开”的策略进行的**枚举算法**，也就意味着它需要**遍历整个状态空间图**，导致算法效率不高。给定一个问题的**状态空间**表示，设计搜索算法时需要考虑以下两个事实：

◆并不是所有的分支都包含**可行解**

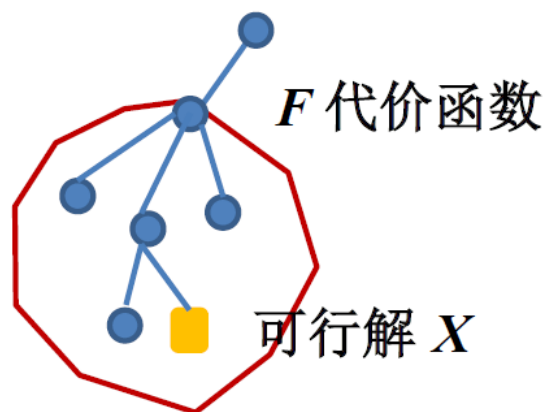
◆并不是所有的分支都包含**最优解**

扩展子结点时，一次性生成它的所有孩子结点，**判断孩子结点是舍弃还是保存？舍弃那些不可能导致可行解或最优解的孩子结点**，其余的结点进入队列中

分支限界算法 = 广度优先搜索 + **剪枝策略**

代价函数

- **计算位置**：搜索树的结点
- **值**：极大化问题是以该点为根的子树所有可行解的值的上界（极小化问题为下界）
- **性质**：对极大化问题父结点代价不小于子结点的代价（极小化问题相反）

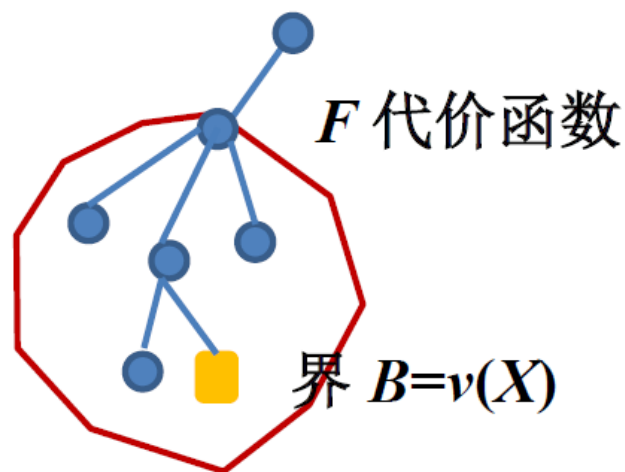


$$F \geq v(X)$$

极大化

界

- **含义**: 当前得到可行解的目标函数的最大值(极小化问题相反)
- **初值**: 极大化问题初值为 0 (极小化问题为最大值)
- **更新**: 得到更好的可行解时



分支限界

停止分支回溯父结点的依据：

1. 不满足约束条件
2. 对于极大化问题，代价函数值小于当前界（对于极小化问题是大于界）

界的更新

对极大化问题，如果一个新的可行解的优化函数值大于 (极小化问题为小于) 当前的界，则把界更新为该可行解的值

实例

背包问题:

4 种物品, 重量 w_i 与价值 v_i 分别为

$$v_1=1, v_2=3, v_3=5, v_4=9$$

$$w_1=2, w_2=3, w_3=4, w_4=7$$

背包重量限制为10

$$\max x_1 + 3x_2 + 5x_3 + 9x_4$$

$$2x_1 + 3x_2 + 4x_3 + 7x_4 \leq 10$$

$$x_i \in \mathbf{N}, i = 1, 2, 3, 4$$

代价函数的设定

- 对结点 $\langle x_1, x_2, \dots, x_k \rangle$ ，估计以该结点为根的子树中可行解的上界。
- 按 v_i/w_i 从大到小排序， $i = 1, 2, \dots, n$
- 代价函数 = 已装入价值 + Δ
 - Δ : 还可继续装入最大价值的上界
 - $\Delta = \text{背包剩余重量} \times v_{k+1}/w_{k+1}$ (可装)
 - $\Delta = 0$ (不可装)

实例：背包问题

背包问题的实例：

$$\max x_1 + 3x_2 + 5x_3 + 9x_4$$

$$2x_1 + 3x_2 + 4x_3 + 7x_4 \leq 10$$

$$x_i \in \mathbf{N}, i = 1, 2, 3, 4$$

对变元重新排序使得

$$\frac{v_i}{w_i} \geq \frac{v_{i+1}}{w_{i+1}}$$

排序后实例 $\max 9x_1 + 5x_2 + 3x_3 + x_4$

$$7x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10$$

$$x_i \in \mathbf{N}, i = 1, 2, 3, 4$$

代价函数与分支策略确定

结点 $\langle x_1, x_2, \dots, x_k \rangle$ 的代价函数

$$\sum_{i=1}^k v_i x_i + (b - \sum_{i=1}^k w_i x_i) \frac{v_{k+1}}{w_{k+1}}$$

若对某个 $j > k$ 有 $b - \sum_{i=1}^k w_i x_i \geq w_j$

$$\sum_{i=1}^k v_i x_i$$

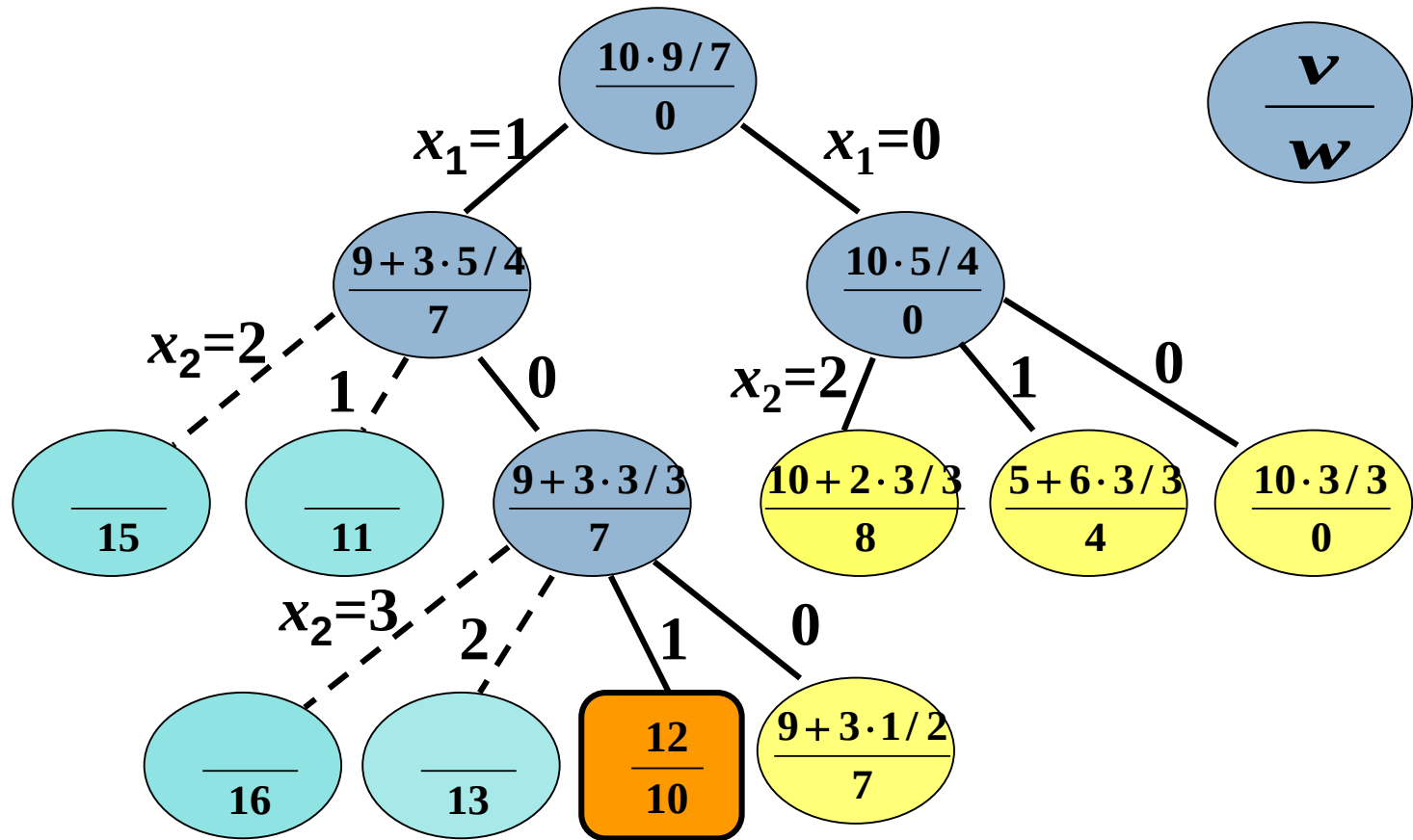
否则

分支策略----深度优先

$$\max 9x_1 + 5x_2 + 3x_3 + x_4$$

$$7x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10, \quad x_i \in \mathbb{N}, i = 1, 2, 3, 4$$

实例



最大团问题

问题: 无向图 $G=<V,E>$, 求 G 的最大团.

G 的子图: $G'=<V',E'>$, 其中 $V'\subseteq V, E'\subseteq E$,

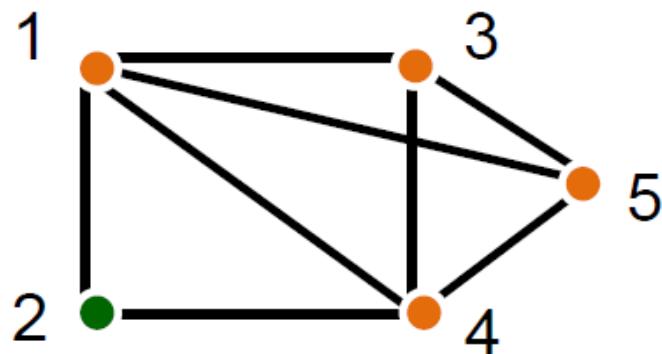
G 的补图: $\overline{G}=<V, E'>$, E' 是 E 关于完全图边集的补集

G 中的团: G 的完全子图

G 的最大团: 顶点数最多的团

实例

最大团: $\{ 1, 3, 4, 5 \}$,



独立集与团

G 的**点独立集**: G 的顶点子集 A , 且
 $\forall u, v \in A, \{u, v\} \notin E.$

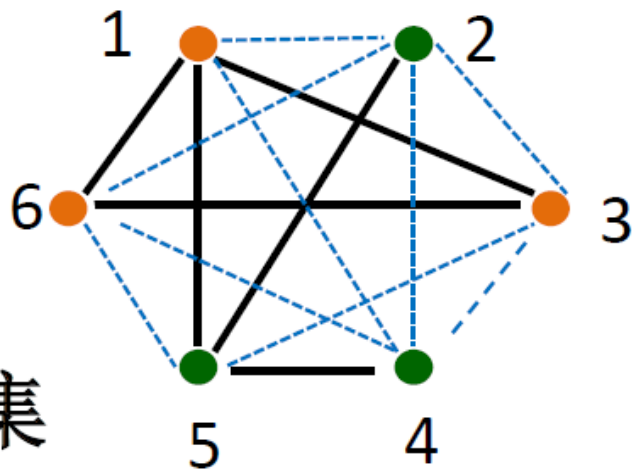
最大点独立集: 顶点最多的点独立集

命题: U 是 G 的最大团当且仅当 U 是 $\overline{G}$ 的最大点独立集.

G 的最大团:

$$U = \{1, 3, 6\}$$

补图 $\overline{G}$ 的最大点独立集



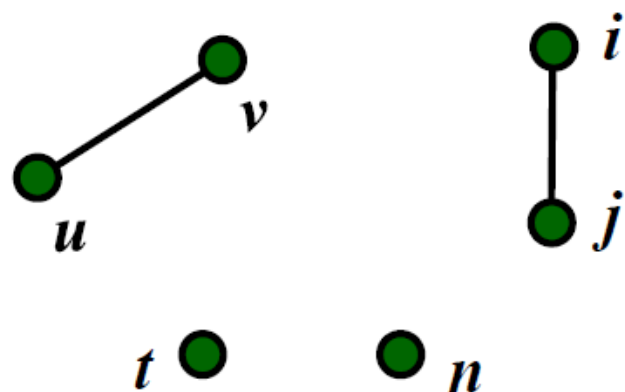
应用

编码, 故障诊断, 计算机视觉, 聚类分析,
经济学, 移动通信, VLSI 电路设计, ...

例子: 噪音使信道传输字符发生混淆

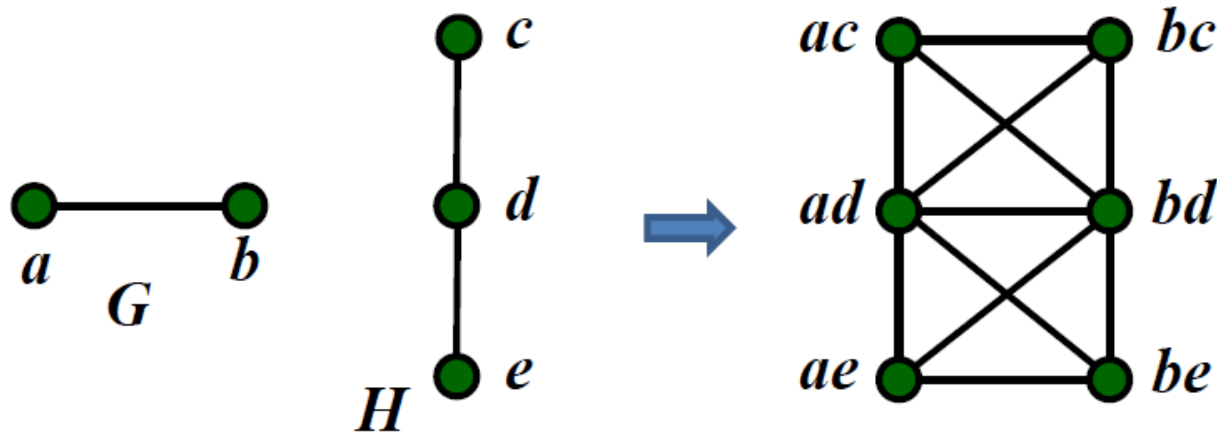
混淆图 $G = \langle V, E \rangle$, V 为有穷字符集,

$\{u, v\} \in E \Leftrightarrow u$ 和 v 易混淆



编码设计

xy 与 uv 混淆 $\Leftrightarrow x$ 与 u 混淆且 y 与 v 混淆
 $\vee x=u$ 且 y 与 v 混淆 $\vee x$ 与 u 混淆且 $y=v$



G 与 H 的正规积

为减少噪音干扰，设计代码应该找混淆图中的最大点独立集

最大团问题

问题： 给定无向图 $G = \langle V, E \rangle$, 其中顶点集 $V = \{ 1, 2, \dots, n \}$, 边集为 E . 求 G 中的最大团.

解： $\langle x_1, x_2, \dots, x_n \rangle$ 为 0-1 向量,
 $x_k=1$ 当且仅当顶点 k 属于最大团.

蛮力算法： 对每个顶点子集, 检查是否构成团, 即其中每对顶点之间是否都有边. 有 2^n 个子集, 至少需要指数时间.

分支限界算法设计

搜索树为子集树.

结点 $\langle x_1, x_2, \dots, x_k \rangle$ 的含义:

已检索顶点 $1, 2, \dots, k$, 其中 $x_i=1$ 表示
顶点 i 在当前的团内, $i = 1, 2, \dots, k$

约束条件: 该顶点与当前团内每个顶
点都有边相连

界: 当前已检索到的极大团的顶点数

代价函数

代价函数：目前的团可能扩张为极大团的顶点数上界

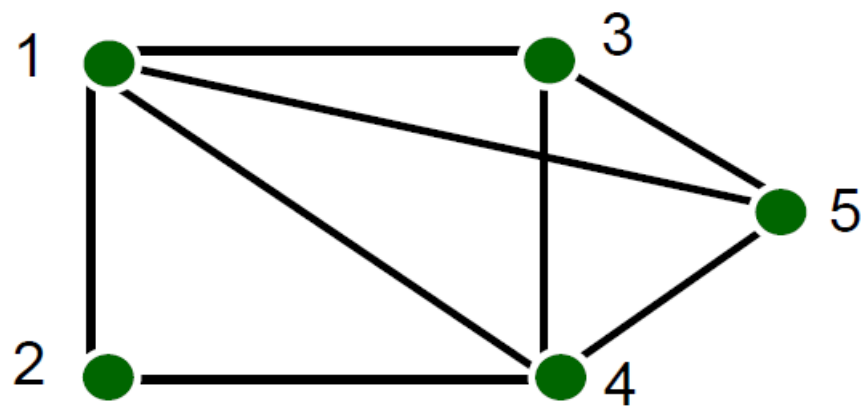
$$F = C_n + n - k$$

其中 C_n 为目前团的顶点数（初始为0）

k 为结点层数

最坏情况下时间： $O(n2^n)$

实例



顶点编号顺序为 1, 2, 3, 4, 5,

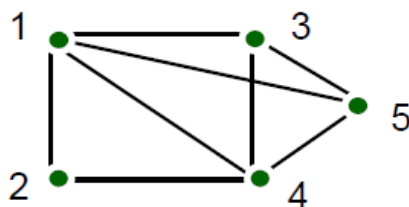
对应 x_1, x_2, x_3, x_4, x_5 ,

$x_i = 1$ 当且仅当 i 在团内

分支规定左子树为 1, 右子树为 0.

B 为界, F 为代价函数值.

搜索树



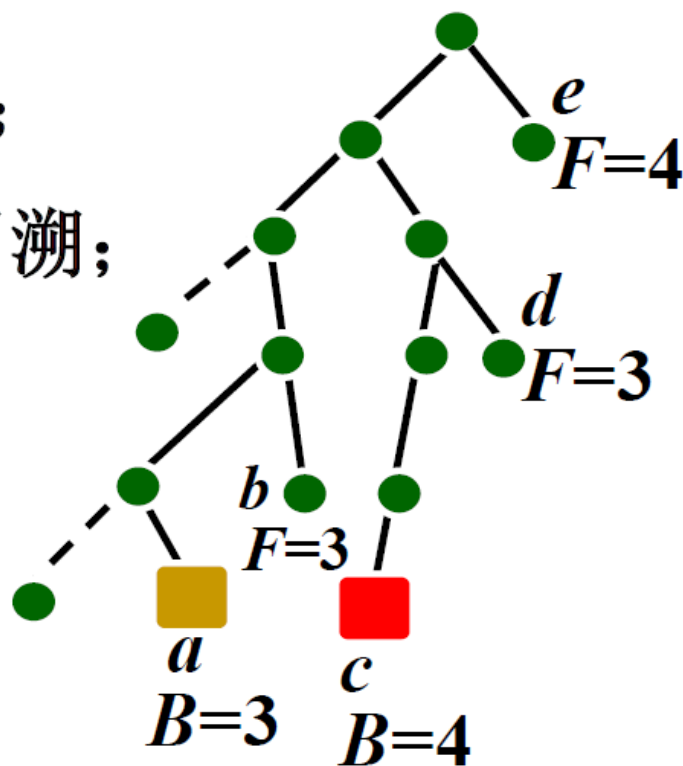
a: 极大团 $\{1,2,4\}$,
顶点数为 3, 界 $B=3$;

b: 代价函数值 $F=3$, 回溯;

c: 极大团 $\{1,3,4,5\}$,
顶点数为 4,
修改界 $B=4$;

d: $F=3$, 不必搜索;

e: $F=4$, 不必搜索.



输出最大团 $\{1,3,4,5\}$, 顶点数为 4.

货郎问题的定义

- 输入

有穷个城市的集合 $C=\{c_1, c_2, \dots, c_n\}$,
距离 $d(c_i, c_j)=d(c_j, c_i) \in \mathbb{Z}^+$, $1 \leq i < j \leq n$

- 解: $1, 2, \dots, n$ 的排列 $k_1, k_2, \dots, k_n$ 使得:

$$\min \left\{ \sum_{i=1}^{n-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_n}, c_{k_1}) \right\}$$

算法设计

解向量为 $\langle 1, i_1, i_2, \dots, i_{n-1} \rangle$, 其中 $i_1, i_2, \dots, i_{n-1}$ 为 $\{2, 3, \dots, n\}$ 的排列.

搜索空间为排列树, 结点 $\langle i_1, i_2, \dots, i_k \rangle$ 表示得到 k 步路线.

约束条件: 令 $B = \{i_1, i_2, \dots, i_k\}$, 则

$$i_{k+1} \in \{2, \dots, n\} - B$$

即每个结点只能访问一次.

代价函数与界

界：当前得到的最短巡回路线长度

代价函数：设顶点 c_i 出发的最短边长度为 l_i , d_j 为选定巡回路线中第 j 段的长度

$$L = \sum_{j=1}^k d_j + l_{i_k} + \sum_{i_j \notin B} l_{i_j}$$

已走过
路径长

剩余长
度下界

代价函数

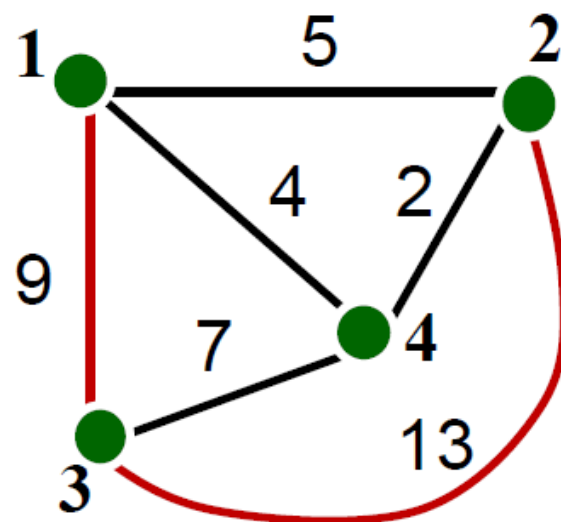
$$L = \sum_{j=1}^k d_j + l_{i_k} + \sum_{i_j \notin B} l_{i_j}$$

部分路线<1,3,2>

$$L=9+13+2+2=26$$

9+13为走过的路径长度

后两项分别为从结点2及4出发的最短边长



实例运行

深度优先遍历搜索树

- 第一个界: $\langle 1, 2, 3, 4 \rangle$, 长度为 29
- 第二个界: $\langle 1, 2, 4, 3 \rangle$, 长度为 23
- 结点 $\langle 1, 3, 2 \rangle$: 代价函数值 $26 > 23$, 不再搜索, 返回 $\langle 1, 3 \rangle$, 右子树向下
- 结点 $\langle 1, 3, 4 \rangle$, 代价函数值 $9 + 7 + 2 + 2 = 20$, 继续, 得到可行解 $\langle 1, 3, 4, 2 \rangle$, 长度 23.
- 回溯到结点 $\langle 1 \rangle$, 沿 $\langle 1, 4 \rangle$ 向下...

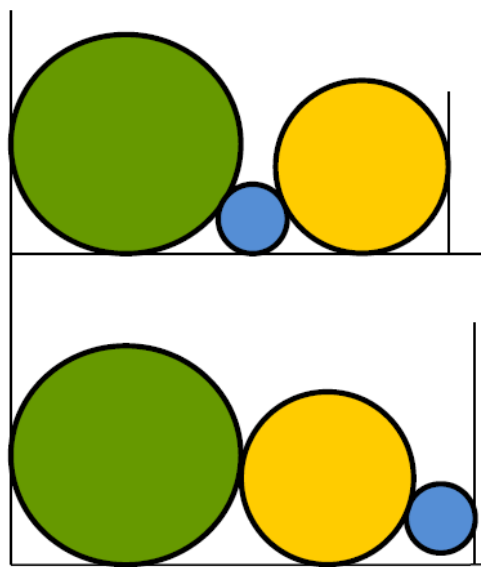
最优解 $\langle 1, 2, 4, 3 \rangle$ 或 $\langle 1, 3, 4, 2 \rangle$, 长度 23

算法分析

- 搜索树的树叶个数: $O((n-1)!)$, 每片树叶对应 1 条路径, 每条路径有 n 个结点.
- 每个结点代价函数计算时间 $O(1)$, 每条路径的计算时间为 $O(n)$
- 最坏情况下算法的时间 $O(n!)$

圆排列问题

给定 n 个圆的半径序列, 各圆与底线相切排列, 假定每个圆占大于1的长度, 求具有最小长度 l_n 的圆的排列顺序.



三个圆,半径给定
两种排列具有不同排列长度, 第一种方式具有更小的排列长度

算法设计：分支限界

解为 $\langle i_1, i_2, \dots, i_n \rangle$ 为 $1, 2, \dots, n$ 的排列
解空间为排列树.

部分解向量 $\langle i_1, i_2, \dots, i_k \rangle$: 表示前 k 个圆
已排好. 令 $B = \{ i_1, i_2, \dots, i_k \}$

下一个圆选择 i_{k+1} .

约束条件: $i_{k+1} \in \{1, 2, \dots, n\} - B$

界: 当前得到的最小圆排列长度

符号说明

k : 算法已经选择了第1— k 个圆

r_k : 第 k 个圆的半径

d_k : 第 $k-1$ 个圆到第 k 个圆的圆心水平距离, $k>1$

x_k : 第 k 个圆的圆心坐标, 规定 $x_1=0$,

l_k : 第 1— k 个圆的排列长度

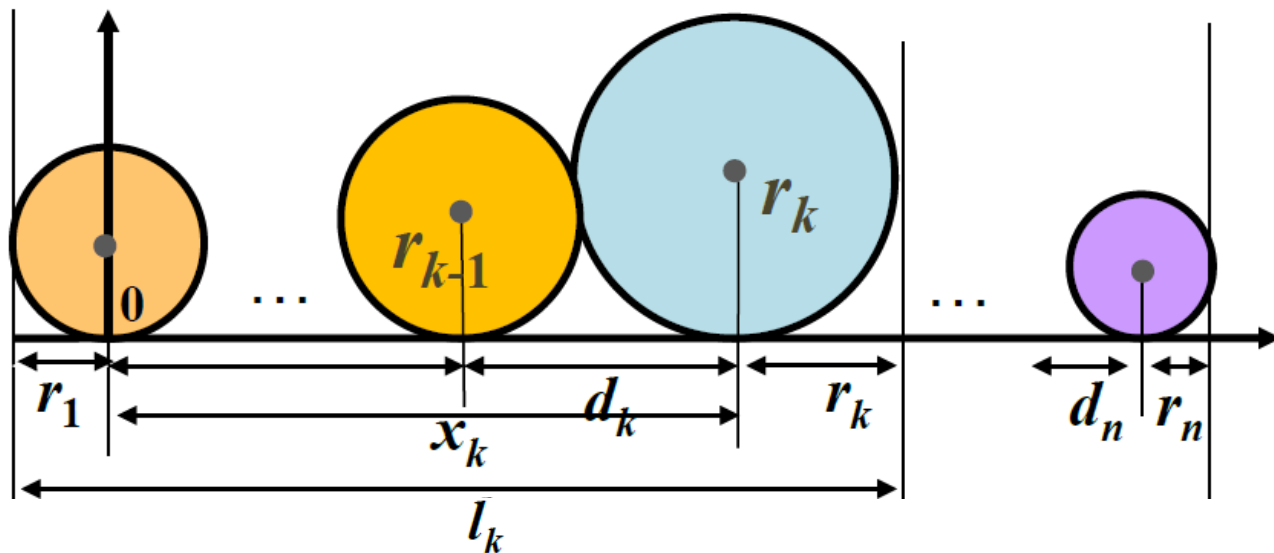
L_k : 放好 1— k 个圆以后, 对应结点的代价函数值 $L_k \leq l_n$

圆排列长度估计

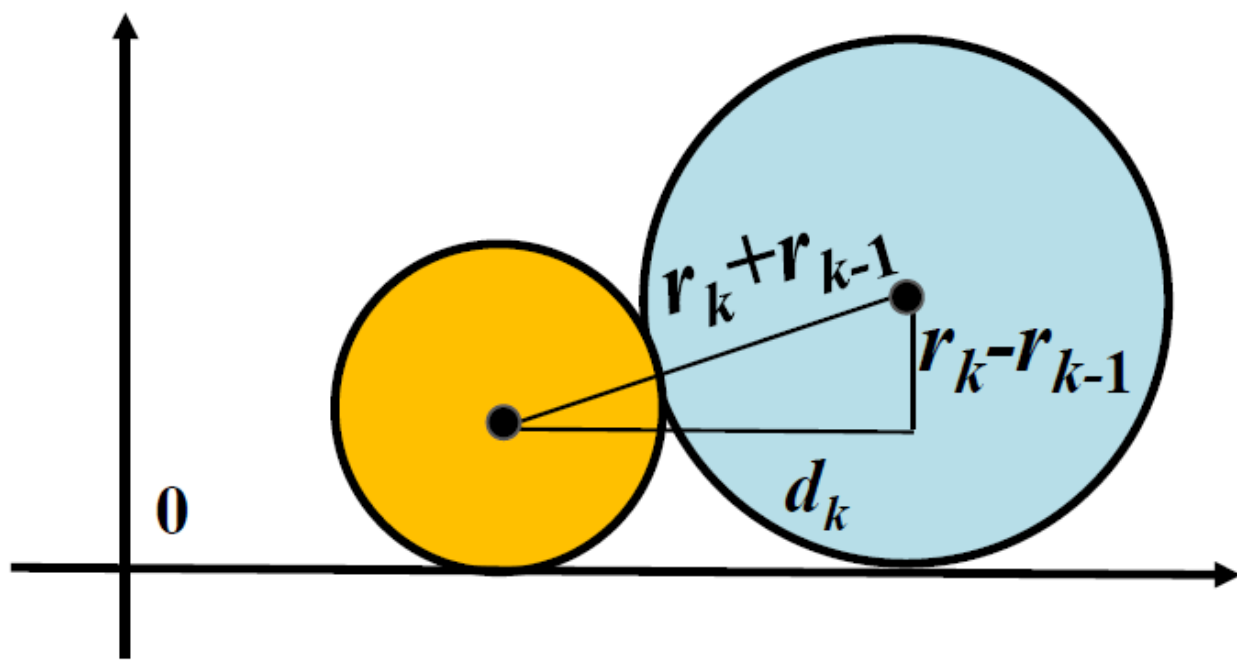
$$x_k = x_{k-1} + d_k$$

部分排列长度 $l_k = x_k + r_k + r_1$

排列长度 $l_n = x_k + d_{k+1} + d_{k+2} + \dots + d_n + r_n + r_1$



有关量的计算



$$\begin{aligned} d_k &= \sqrt{(r_k + r_{k-1})^2 - (r_k - r_{k-1})^2} \\ &= 2\sqrt{r_{k-1}r_k} \end{aligned}$$

代价函数

$$d_k = 2\sqrt{r_{k-1}r_k}$$

排列
长度

$$l_n = x_k + 2\sqrt{r_k r_{k+1}} + 2\sqrt{r_{k+1} r_{k+2}} + \dots + 2\sqrt{r_{n-1} r_n} + r_n + r_1$$

$$\geq x_k + 2(n-k)r + r + r_1$$

下界

$$L_k = x_k + (2n - 2k + 1)r + r_1$$

$$r = \min(r_{i_j}, r_k)$$

$$i_j \in \{1, 2, \dots, n\} - B$$

时间复杂度

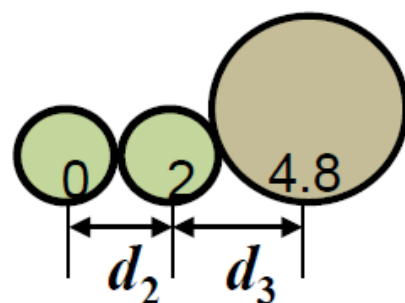
- 搜索树树叶数为 $O(n!)$
- 每条路径代价函数的计算为 $O(n)$
- 最坏情况下算法时间复杂度为
 $O(n \cdot n!) = O((n+1)!)$

实例

输入: $R=\{1,1,2,2,3,5\}$

排列 $\langle 1,2,3,4,5,6 \rangle$

半径排列: 1,1,2,2,3,5



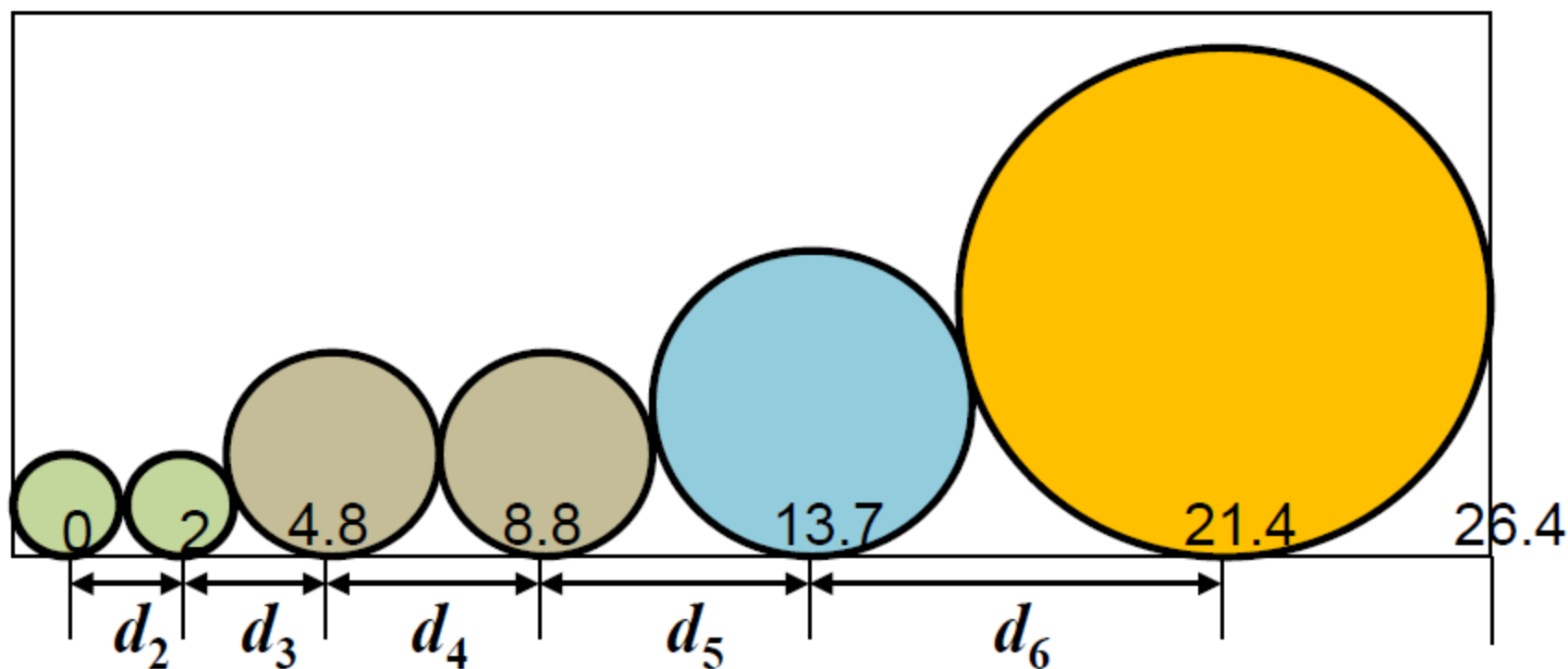
$$L_3 = 7.8 + 12 = 19.8$$

k	r_k	d_k	x_k	l_k	L_k
1	1	0	0	2	12
2	1	2	2	4	12
3	2	2.8	4.8	7.8	19.8
4	2	4	8.8	11.8	19.8
5	3	4.9	13.7	17.7	23.7
6	5	7.7	21.4	27.4	27.4

实例计算

半径排列: 1,1,2,2,3,5, 可行解 $l_6=27.4$

最优解: $\langle 1,3,5,6,4,2 \rangle$, 最短长度26.5



连续邮资问题

问题：给定 n 种不同面值的邮票,每个信封至多贴 m 张, 试给出邮票的最佳设计, 使得从 1 开始, 增量为 1 的连续邮资区间达到最大?

实例： $n = 5, m = 4$.

设计1: 面值 $X_1 = \langle 1, 3, 11, 15, 32 \rangle$,
 邮资连续区间 $\{1, 2, \dots, 70\}$

设计2: 面值 $X_2 = \langle 1, 6, 10, 20, 30 \rangle$,
邮资连续区间 $\{1, 2, 3, 4\}$

算法设计

可行解： $\langle x_1, x_2, \dots, x_n \rangle$, $x_1 = 1$,
 $x_1 < x_2 < \dots < x_n$

搜索策略：深度优先

约束条件：在结点 $\langle x_1, x_2, \dots, x_i \rangle$ 处，
邮资最大连续区间为 $\{1, \dots, r_i\}$, x_{i+1}
的取值范围是

$$\{x_i + 1, \dots, r_i + 1\}$$

若 $x_{i+1} > r_i + 1$, $r_i + 1$ 的邮资将没法支付.

r_i 的计算

$y_i(j)$: 用至多 m 张面值 x_i 的邮票加上 $x_1, x_2, \dots, x_{i-1}$ 面值的邮票贴 j 邮资时的最少邮票数, 则

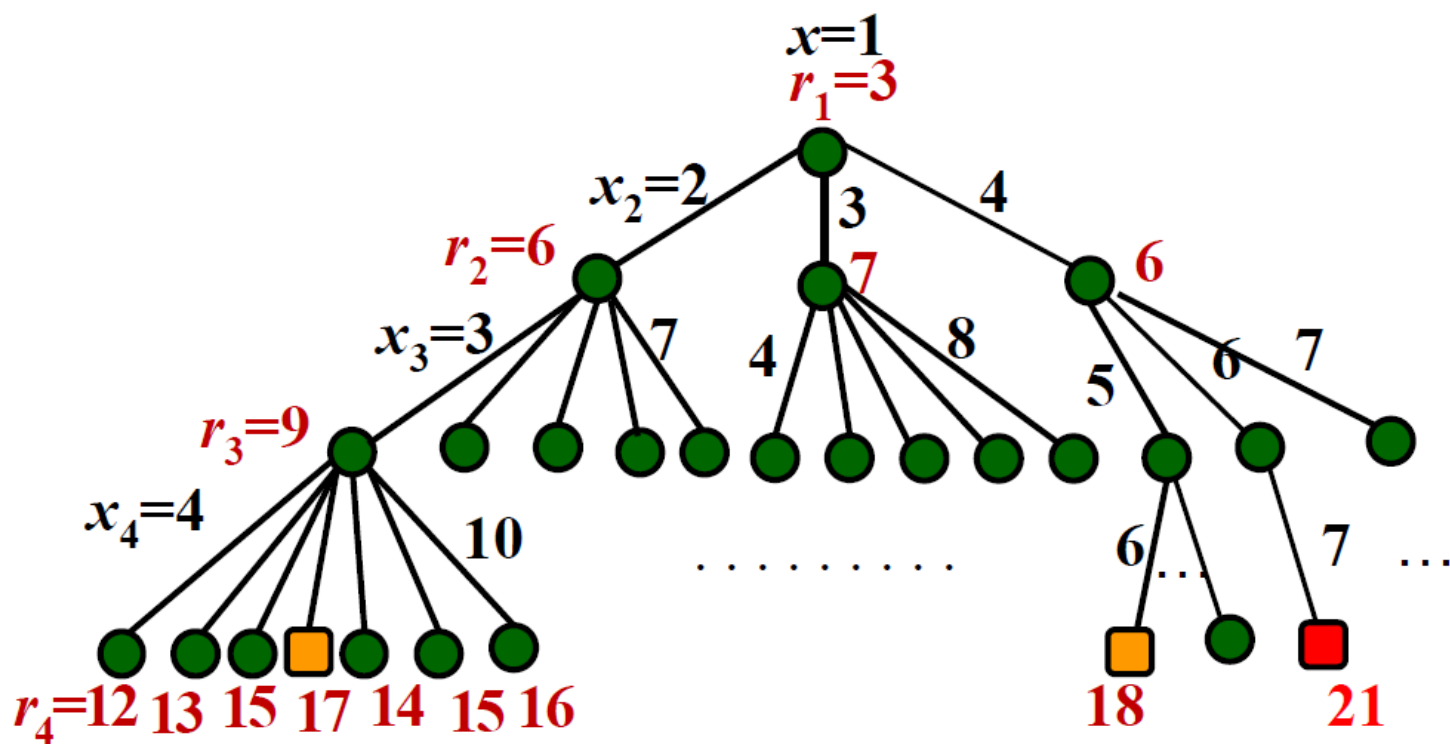
$$y_i(j) = \min_{1 \leq t \leq m} \{t + y_{i-1}(j - t x_i)\}$$

$$y_1(j) = j$$

$$r_i = \min\{j \mid y_i(j) \leq m, y_i(j+1) > m\}$$

界: max , m 张邮票连续付的最大邮资

部分搜索树 $n=4, m=3$



解: $X=<1,4,6,7>$, 最大连续区间 $\{1,\dots,21\}$

回溯算法小结

- (1) 适于求解组合搜索问题及优化问题
- (2) 求解条件：满足多米诺性质
- (3) 解的表示：解向量，求解是不断扩充解向量的过程
- (4) 回溯条件：
 - 搜索问题——约束条件
 - 优化问题——约束条件 + 代价函数
- (5) 分支策略：深度优先、宽度优先、宽深结合、函数优先

小结（续）

(6) 结点状态：白结点,黑结点,灰结点

(7) 算法时间复杂度：

$$W(n) = (p(n) f(n))$$

其中 $p(n)$ 为每个结点的工作量

$f(n)$ 为结点数

最坏情况下时间通常为指数级

平均情况下比蛮力算法好

空间代价小

小结（续）

(8) 降低时间复杂性的主要途径：

- 根据树的分支设计优先策略
结点少分支优先，解多分支优先
- 利用搜索树的对称性裁减子树
- 分解为子问题，若求解时间 $f(n)=c2^n$ ，组合时间 $O(2^{n/k})$ ，分解为 k 个 n/k 规模子问题，则该算法时间为

$$T(n)=kc2^{n/k}+O(2^{n/k})=O(2^{n/k})=o(2^n)$$



启发式搜索算法

集束搜索

B e a m S e a r c h

01

传统搜索的瓶颈与启发式需求

1

传统搜索的局限性

深度优先搜索容易陷入局部最优，广度优先搜索则因内存占用过大而受限。在面对大规模复杂问题时，传统搜索方法往往效率低下，难以在合理时间内找到解决方案。

2

启发式技术的重要性

启发式技术通过利用问题的特定知识来指导搜索过程，能够在不完全探索整个搜索空间的情况下找到近似最优解，显著提高了搜索效率，尤其适用于复杂问题的求解。

启发式技术核心思想



01

启发式技术的定义

启发式技术是一种利用特定标准在多个选项中确定最有效方法以实现预期目标的策略。它优先考虑速度而非系统性和详尽的探索，从而提高了搜索过程的效率。

启发式技术的特点

启发式技术通常能够获得良好的解决方案，而无需像指数时间算法那样耗费大量的计算成本。它尤其适用于以高效的方式解决复杂问题，如旅行商问题。



02



03

启发式技术的应用

启发式技术广泛应用于人工智能领域，如路径规划、机器学习、自然语言处理等，通过减少不必要的搜索路径，快速找到接近最优的解。

集束搜索的定义

集束搜索是一种启发式搜索算法，它通过系统地扩展约束集合内最有希望的节点来导航图。该方法结合了广度优先搜索的元素，在每一层生成所有后继节点，但仅根据启发式值评估并扩展每一层上一定数量的最佳节点。

搜索流程步骤

01

初始化

从根节点开始并生成其后继节点，为搜索过程奠定基础。

02

节点扩展

从当前节点生成后继节点并应用启发式函数对其进行评估，确定每个节点的优先级。

03

选择

根据启发式值选择前 W 个节点，这些节点构成下一层级的探索，确保搜索的高效性。

04

迭代

重复对新层级的节点进行扩展和选择的过程，逐步深入搜索空间，直至达到目标或满足某个条件。

05

终止

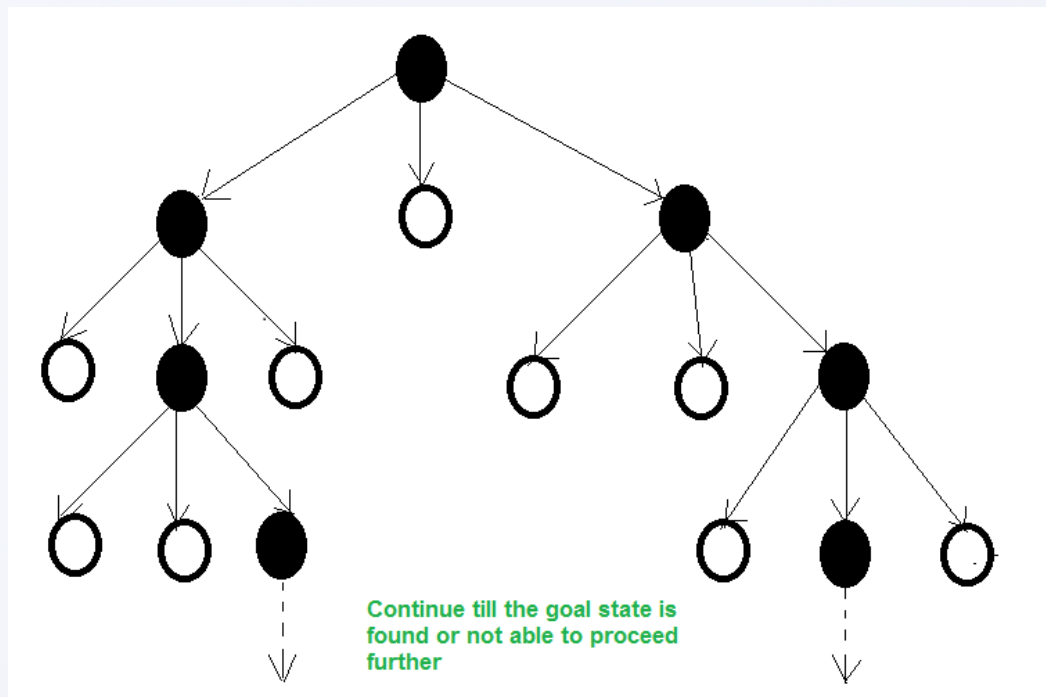
当找到目标或没有更多节点可供扩展时，搜索停止，输出最终结果。

束宽度与分支因子

束宽度 W 定义了每一层级考虑的节点数量，分支因子 B 决定了每个节点的后继数量。在搜索过程中，束宽度 W 直接影响评估的节点数量，从而影响搜索的广度。

示例说明

例如，在一棵搜索树中， $W=2$ ， $B=3$ 。根据启发式值仅选择两个节点在每个级别进行进一步扩展，从而有效控制搜索范围。



优势

效率提升

集束搜索通过限制每层扩展的节点数量，显著提高了搜索效率，能够在大规模搜索空间中快速找到可行解。

灵活性

算法可以通过调整束宽度和启发式函数来适应不同类型的问题，具有很强的灵活性。

内存效率

由于束宽度限制了搜索所需的内存，集束搜索适用于资源受限的环境，降低了内存占用。

局限性与风险



01

次优解风险

由于束宽度有限，集束搜索可能会修剪潜在的最佳路径，导致找到的解并非最优解。



02

启发式依赖

集束搜索的效果高度依赖于启发式函数的质量。如果启发式函数设计不当，可能会导致搜索效率低下或结果不理想。

集束搜索在人工智能中的应用

NLP

在自然语言处理领域，集束搜索广泛应用于机器翻译和语音识别任务中。通过在每一步仅保留概率最高的前W个序列，集束搜索能够在大规模的词或音素序列空间中快速找到最可能的输出，显著降低了解码延迟，同时保持较高的翻译或识别准确率。

机器人路径规划

在机器人路径规划中，集束搜索通过启发式评估路径成本，能够在有限时间内为机器人找到可行且接近最优的行动序列，兼顾实时性与效果。

游戏AI路径规划

在战略游戏AI中，集束搜索帮助智能体在高维动作空间中快速找到最优策略，提升游戏的智能性和趣味性。

贪婪最佳优先搜索

Greedy Best-First Search

01

贪婪最佳优先搜索诞生背景

01

传统搜索的局限

传统无信息搜索算法在面对庞大图结构时，因盲目扩展节点导致效率低下，难以在复杂环境中快速找到目标。

02

贪婪最佳优先搜索的动机

贪婪最佳优先搜索引入启发式函数，估计节点到目标的距离，优先扩展看似最接近目标的节点，从而在速度与路径质量之间做出权衡，提升搜索效率。启发式函数对贪婪最佳优先搜索的效率和成功率起着决定性作用，它需要快速计算、不低估真实代价且与问题结构紧密匹配。

“贪婪” 决策机制

核心思想

贪婪最佳优先搜索每一步仅根据当前启发式值选择局部最优节点，不回顾历史也不预留回溯余地，快速逼近目标。

优势

这种“短视”策略在简单环境中能快速找到看似最优的路径，显著提升搜索速度，适用于对效率要求较高的场景。

风险

然而，由于缺乏全局视野，该算法可能错过全局最优解，陷入局部最优或死胡同，导致搜索失败。

适用场景

适用于启发式函数质量高、环境相对简单且对解的最优性要求不高的场景，如地图导航、游戏寻路等。

执行流程

01

初始化

将起点放入优先队列，初始化已访问集合，为搜索过程做好准备。

02

扩展节点

计算当前节点所有邻居的启发式值，根据启发式值对邻居节点进行排序，选择最有希望的节点进行扩展。

03

迭代搜索

重复选择优先队列中启发式值最小的节点，检查是否为目标节点，若不是则继续扩展其邻居，直至找到目标或队列为空。

优势

速度优势

1

贪婪最佳优先搜索利用启发式信息，通常比无信息搜索算法更快，能在短时间内找到可行路径，适用于对速度要求较高的场景。

2

实现简单

该算法实现简单，易于理解和调试，适合在时间紧迫或资源有限的情况下快速实现搜索功能。

3

节省内存

优先扩展最有希望的节点，减少了不必要的节点扩展和存储，从而节省内存资源，提高搜索效率。

局限剖析

不保证完备

贪婪最佳优先搜索可能陷入局部最优或死胡同，导致无法找到目标节点，不保证搜索的完备性。

不保证最优

由于其贪婪选择特性，该算法可能错过更短的路径，导致找到的解不是最优解，不保证解的最优性。

启发式依赖

算法的效率和成功率高度依赖启发式函数的质量，差的启发式函数可能导致低效搜索甚至搜索失败。

应用

地图导航

在地图导航中，贪婪最佳优先搜索能快速生成从起点到终点的可行路线，为用户提供预览，提升用户体验。

游戏寻路

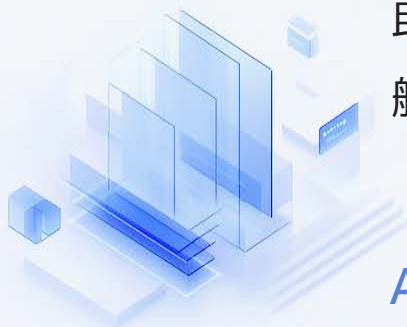
在8-puzzle、15-puzzle等滑块游戏中，该算法能迅速找到将棋盘排列为目标状态的操作序列，提高游戏AI的响应速度。

机器人导航

在机器人未知环境中，贪婪最佳优先搜索能实时规划看似最短的路径，帮助机器人快速避开障碍，实现高效导航。

AI规划

在AI规划任务中，该算法能快速找到可行的解决方案，适用于需要快速响应且可接受次优解的场景。



优化方向



01

与A*结合

将贪婪最佳优先搜索与A*算法结合，引入代价修正，兼顾搜索速度与解的最优性，提升算法性能。



02

动态调整启发式

利用机器学习技术动态调整启发式函数权重，使其能更好地适应不同环境和问题，提高搜索效率。

A*算法

A * a l g o r i t h m

01



01

A*算法的诞生

1968年，Peter Hart、Nils Nilsson和Bertram Raphael在Dijkstra算法基础上提出A*算法，旨在优化路径搜索过程。

A*算法的核心优势



02

A*算法通过引入启发式函数，将实际旅行成本与估计的未来成本相结合，优化搜索过程，减少计算负荷，确保以最少的不必要的探索找到最有效的路径。



03

Dijkstra与A*思想差异

Dijkstra算法均匀地探索起始节点周围的所有可能方向，而A*算法则通过启发式函数 $h(n)$ 引入预测机制，使搜索方向更具目标性，从而在复杂环境中表现出更高的效率。

A*算法的应用场景



04

A*算法广泛应用于电子游戏、网络路由、人工智能等领域，尤其在游戏中的寻路、机器人路径规划、网络数据包传输等场景中发挥重要作用。

$f(n)=g(n)+h(n)$ 拆解

$g(n)$ 的定义

$g(n)$ 表示从起始节点到当前节点的实际累积成本，是路径搜索过程中的已知成本部分，可通过已探索路径直接计算得出。

$h(n)$ 的定义

$h(n)$ 是从当前节点到目标节点的启发式估计成本，它通过特定的启发式函数计算得出，为路径搜索提供方向性指导。

$f(n)$ 的作用

$f(n)=g(n)+h(n)$ 是A*算法的核心公式， $f(n)$ 值越小，表示该节点越有可能位于最短路径上，从而指导算法优先探索这些节点。

流程步骤

01

初始化

将初始节点添加到开放集，计算其 $f(n)$ 值，并初始化相关数据结构，为后续搜索做好准备。

02

循环

在每次迭代中，从开放集中选择 $f(n)$ 值最小的节点进行扩展，扩展此节点并检查其邻居。

03

目标检查

如果正在处理的节点是目标节点，则搜索终止，算法返回到目标的路径。

04

节点扩展

如果找到一条更好的路径，即更低的 $f(n)$ ，然后将邻居添加到开放集或更新其值。

05

重复

此过程持续进行，直到找到目标或开放集为空（即没有解决方案）。

好启发式的两大准则

可接受性

好的启发式函数应满足可接受性，即 $h(n)$ 永远不会高估从当前节点到目标节点的真实成本，这保证了A*算法能够找到最短路径。

一致性

一致性要求启发式函数给出的估计值不会出现太大的跳跃，即对于任何节点及其下一步，到达目标的成本不应大于进入下一步再到达目标的成本。

三种经典距离度量

曼哈顿距离

曼哈顿距离适用于基于网格的环境，其中运动仅限于水平和垂直方向，计算公式为 $|x1-x2|+|y1-y2|$ ，常用于游戏中的寻路问题。

欧几里得距离

欧几里得距离表示两点之间的直线距离，适用于允许在任何方向上移动的情况，计算公式为 $\sqrt{(x2-x1)^2+(y2-y1)^2}$ 。

切比雪夫距离

切比雪夫距离适用于允许对角线移动的情况，计算公式为 $\max(|x2-x1|,|y2-y1|)$ ，在某些特殊场景中具有独特优势。

优势与局限

A*算法的优势

A*算法具有保证最优性、完备性、灵活性和较高效率等优势，能够适应多种复杂场景的路径规划需求。

启发式函数的重要性

启发式函数的质量直接影响A*算法的效率和性能，选择合适的启发式函数是优化算法的关键。

A*算法的局限

A*算法的局限性包括高内存占用、对启发式函数的敏感性以及在复杂或密集图中的计算开销较大。

A*算法的适用场景

A*算法适用于资源允许且启发式函数得当的场景，在游戏、机器人、自动驾驶等领域具有广泛的应用前景。

掌握A*迈向智能决策

1

A*算法的核心

A*算法通过
 $f(n)=g(n)+h(n)$ 公式结
合实际成本与启发式预
测，实现高效路径搜索
，是智能决策的重要工
具。

2

A*算法的未来

随着算力提升和启发式
函数的不断优化，A*算
法将在游戏、机器人、
自动驾驶等领域持续进
化，为智能决策提供更
强大的支持。

3

掌握A*算法的意义

掌握A*算法，意味着掌
握了智能决策的关键技
术，为解决复杂路径规
划问题提供了有效的解
决方案。

AO*算法

A O * a l g o r i t h m

01

AO*算法定位与场景

1

AO*算法简介

AO*算法，即'与或星'算法，是A*算法的扩展，专门用于处理复杂决策问题。它能够同时评估多条路径，适用于包含与或节点的问题。

2

传统A*算法局限

传统A*算法仅追踪单一路径，对于复杂决策问题效率较低。例如在购车决策中，需要同时考虑资金和贷款等多个因素，A*算法难以有效处理。

3

AO*算法优势

AO*算法通过同时处理多条路径，能够有效解决复杂决策问题。它在购车决策等场景中表现出色，能够快速找到最优解决方案。

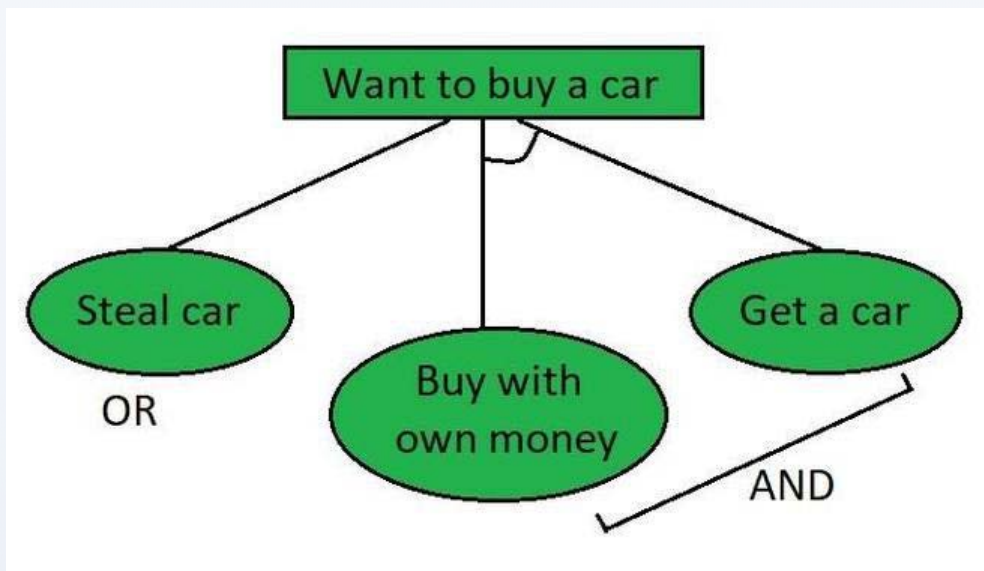
与节点与或节点语义

与节点定义

与节点表示所有子节点必须全部达成。在成本计算中，与节点的成本取子节点的最大值，确保所有条件都得到满足。

或节点定义

或节点表示只需任一子节点达成即可。在成本计算中，或节点的成本取子节点的最小值，提供多种选择策略。



$f(n)=C(n)+h(n)$ 成本模型

01

C(n)定义

C(n)表示从起点到当前节点的真实累积成本，反映了路径的实际开销。

02

h(n)定义

h(n)是启发式估计的剩余成本，用于预测从当前节点到目标节点的潜在开销。

03

与节点成本计算

对于与节点，C(n)取所有子节点成本的最大值，确保所有子任务都完成。

04

或节点成本计算

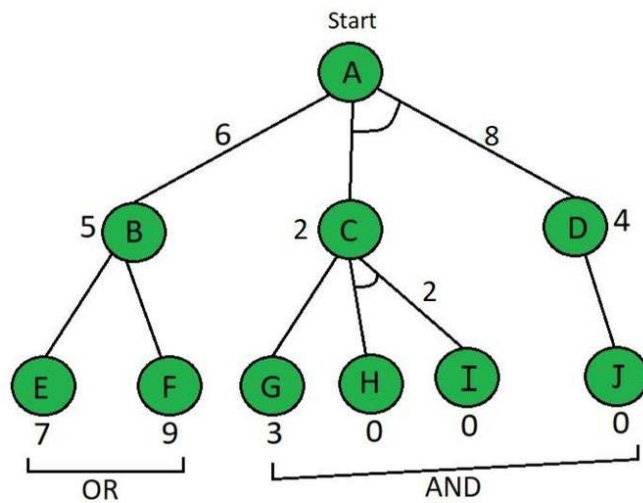
对于或节点，C(n)取所有子节点成本的最小值，选择最优的子任务。

搜索流程概述

AO*算法从根节点出发，按f(n)值升序扩展节点。遇到与节点时递归求解所有子节点，遇到或节点时选择最小f(n)的子节点。叶节点返回启发式值后，自底向上更新父节点成本，动态调整搜索方向。

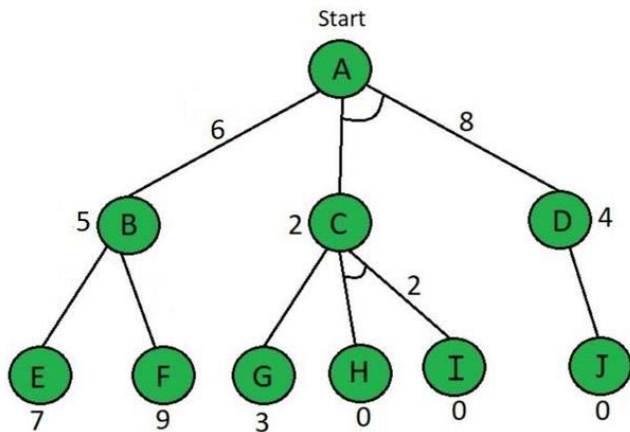
Aspect	<i>A Algorithm*</i>	<i>AO Algorithm*</i>
Search Type	Best-first search	Best-first search
Type of Search	Informed search using heuristics	Informed search using heuristics
Solution Optimality	Always gives the optimal solution	Does not guarantee an optimal solution
Path Exploration	Explores all possible paths	Stops exploring once a solution is found
Memory Usage	Uses more memory	Uses less memory
Endless Loop	May go into an endless loop without proper checks	Cannot go into an endless loop

例子



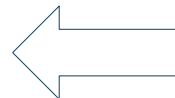
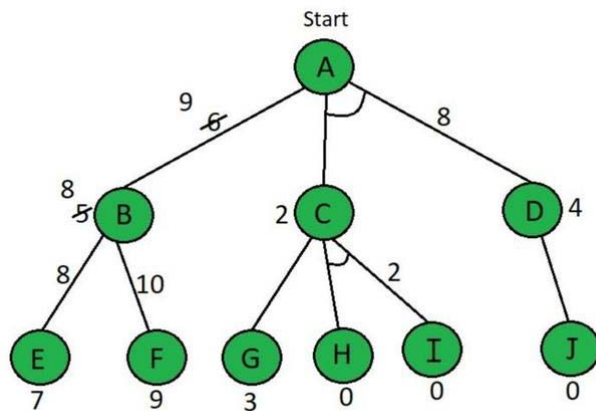
$$\begin{aligned}
 f(A \rightarrow B) &= g(B) + h(B) \\
 &= 1 + 5 \quad (g(n) = 1 \text{ is the default path cost}) \\
 &= 6
 \end{aligned}$$

$$\begin{aligned}
 f(A \rightarrow C + D) &= g(C) + h(C) + g(D) + h(D) \\
 &= 1 + 2 + 1 + 4 \quad (C \text{ \& } D \text{ are AND nodes}) \\
 &= 8
 \end{aligned}$$



$$\begin{aligned} f(B \rightarrow E) &= g(E) + h(E) \\ &= 1 + 7 \\ &= 8 \end{aligned}$$

$$\begin{aligned} f(B \rightarrow F) &= g(F) + h(F) \\ &= 1 + 9 \\ &= 10 \end{aligned}$$

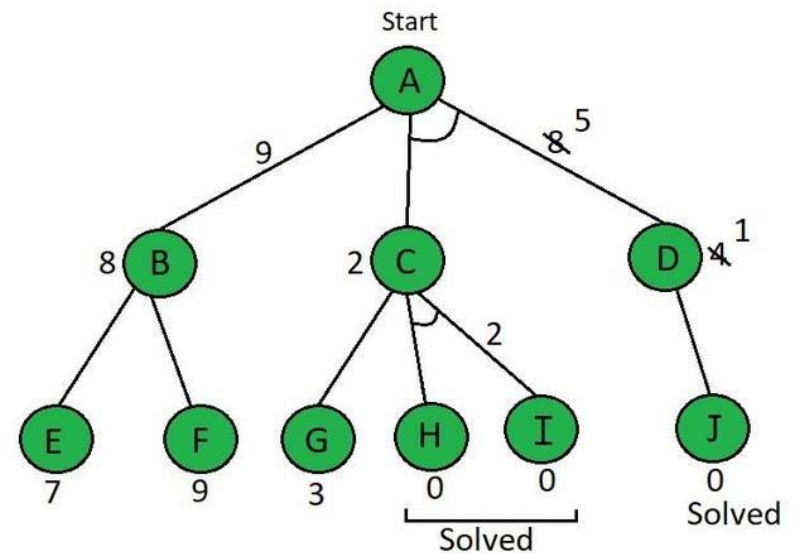


$$\begin{aligned} f(A \rightarrow B) &= g(B) + \text{updated } h(B) \\ &= 1 + 8 \\ &= 9 \end{aligned}$$

$$\begin{aligned}
 f(C \rightarrow G) &= g(G) + h(G) \\
 &= 1 + 3 \\
 &= 4
 \end{aligned}$$

$$\begin{aligned}
 f(C \rightarrow H + I) &= g(H) + h(H) + g(I) + h(I) \\
 &= 1 + 0 + 1 + 0 \quad (H \text{ \& I are AND nodes}) \\
 &= 2
 \end{aligned}$$

$$\begin{aligned}
 f(D \rightarrow J) &= g(J) + h(J) \\
 &= 1 + 0 \\
 &= 1
 \end{aligned}$$



$$\begin{aligned}
 f(A \rightarrow C + D) &= g(C) + h(C) + g(D) + h(D) \\
 &= 1 + 2 + 1 + 1 \\
 &= 5
 \end{aligned}$$

购车决策与或图建模

购车决策场景

购车决策是一个典型的与或问题。根节点为'购车'，下设或节点'现金购买'与'分期付款'。现金路径下再挂与节点'资金到位'与'贷款批准'，体现了任务的依赖关系。

与或图构建

通过给每条边赋单位成本、给叶节点赋启发式值，构建一张可计算的有向无环图（DAG）。这张图既包含策略选择，又包含任务依赖，完美契合AO*算法的能力范围。

。

效率优势与资源瓶颈

效率优势

AO*算法能够一次评估多条路径，内存占用低于穷举搜索，天然适配与或图结构。

启发式质量

启发式函数的质量直接影响算法的速度和效果，高质量的启发式函数能够显著提高效率。

资源瓶颈

当节点过多时，计算量会激增，对内存资源要求较高，可能成为资源瓶颈。

优化方向

合理剪枝与高质量启发式函数是工程落地的关键，能够有效提升算法的性能。

IDA*算法

Iterative Deepening A* algorithm

01

IDA*的背景与动机

1

传统算法的局限

传统A*算法在处理复杂图时，由于需要维护大量节点，导致内存消耗呈指数级增长，难以应用于大规模问题。而深度优先搜索虽省内存，却无法保证找到最优解。

2

IDA*的动机

IDA*算法应运而生，它结合了A*的最优性与深度优先搜索的内存效率，通过迭代加深和启发式评估，既避免了A*的内存爆炸，又克服了DFS的盲目性。

IDA*与A*、DFS的融合

01

继承A*的评估框架

IDA*继承了A*的 $f(n)=g(n)+h(n)$ 评估框架，其中 $g(n)$ 记录从起点到当前节点的实际成本， $h(n)$ 预估从当前节点到目标的成本，二者相加决定节点的优先级。

采用DFS的遍历方式

IDA*采用深度优先搜索的栈式遍历方式，逐层深入探索节点，仅在当前分支中保存节点信息，大大节省了内存空间。

02

03

迭代加深实现平衡

IDA*通过逐轮提高阈值实现迭代加深，在每一轮中仅扩展 f 值小于等于阈值的节点，从而在保证最优性的同时，有效控制了搜索范围和内存占用。



主要特点

一

图遍历算法：IDA* 通过逐步加深搜索深度来探索图。

二

最短路径查找：它使用 DFS 和 A* 技术的组合来有效地找到加权图中的最短路径。

三

可接受的启发式：IDA* 中的启发式函数确保估计成本永远不会超过实际成本，使其成为可接受的启发式。

四

内存效率：作为深度优先搜索算法，IDA* 与传统 A* 算法相比占用的内存要少得多。

五

重点搜索：该算法旨在仅探索最有希望的节点，确保不会陷入不必要的深度。

IDA* 算法流程



1. 初始化

将根节点设置为当前节点并计算其 f 分数。



2. 设置阈值

根据起始节点的 f 分数初始化阈值。



3. 节点扩展

扩展当前节点的子节点并计算它们的 f 分数。



4. 修剪

如果 f 分数超过阈值，则修剪节点并将其存储以供将来探索。



5. 路径返回

一旦找到目标节点，返回从起始节点到目标的路径。



6. 更新阈值

如果没有找到目标，则根据最小剪枝值增加阈值并重复该过程。

迭代加深与剪枝流程

初始化阈值

算法开始时，将根节点的 f 值设为初始阈值，以此作为第一轮搜索的深度限制。

更新阈值

每轮搜索结束后，以本轮中被剪枝节点的最小 f 值作为下一轮的阈值，逐步收紧搜索边界。

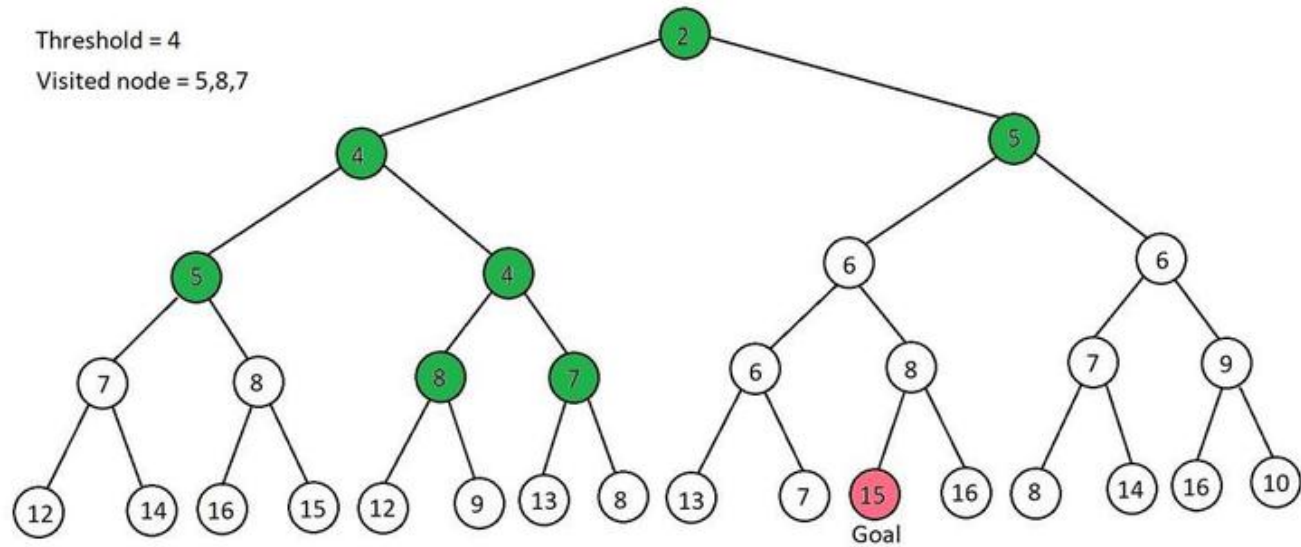
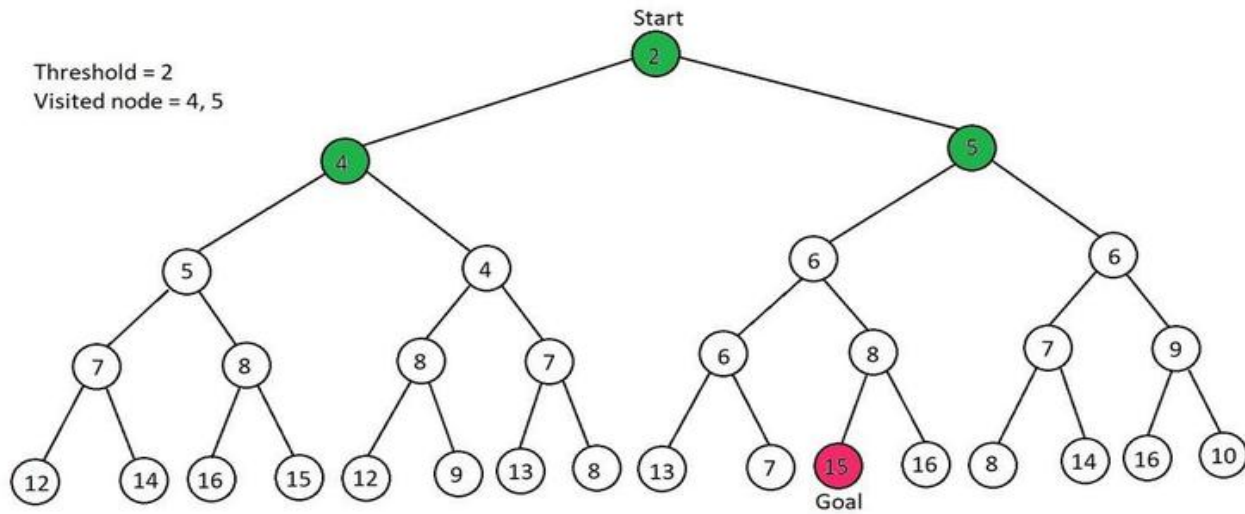
深度优先遍历与剪枝

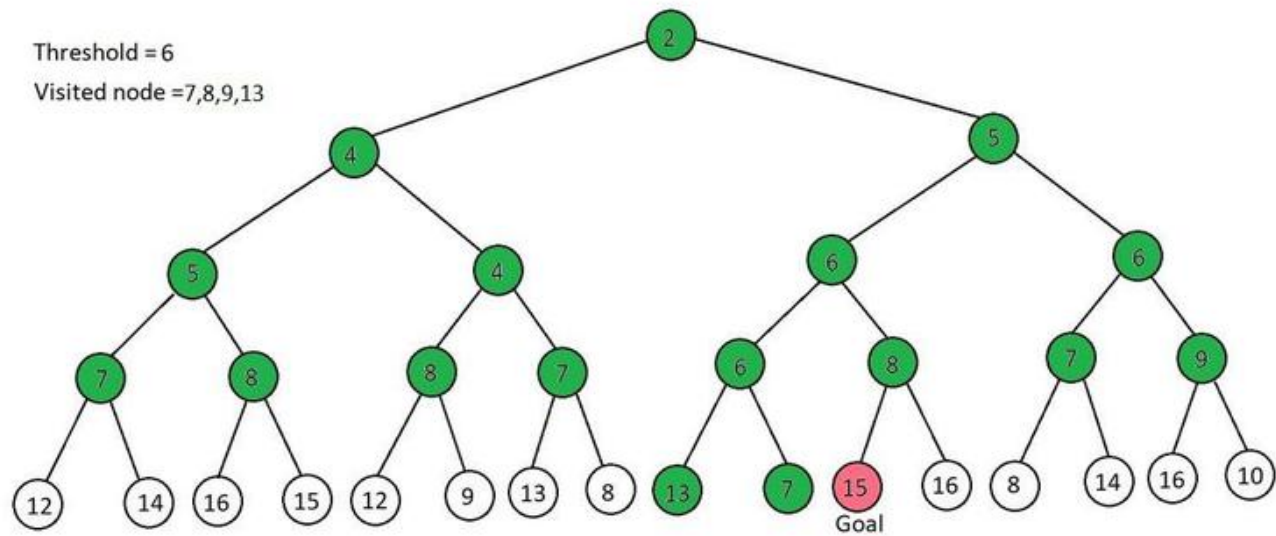
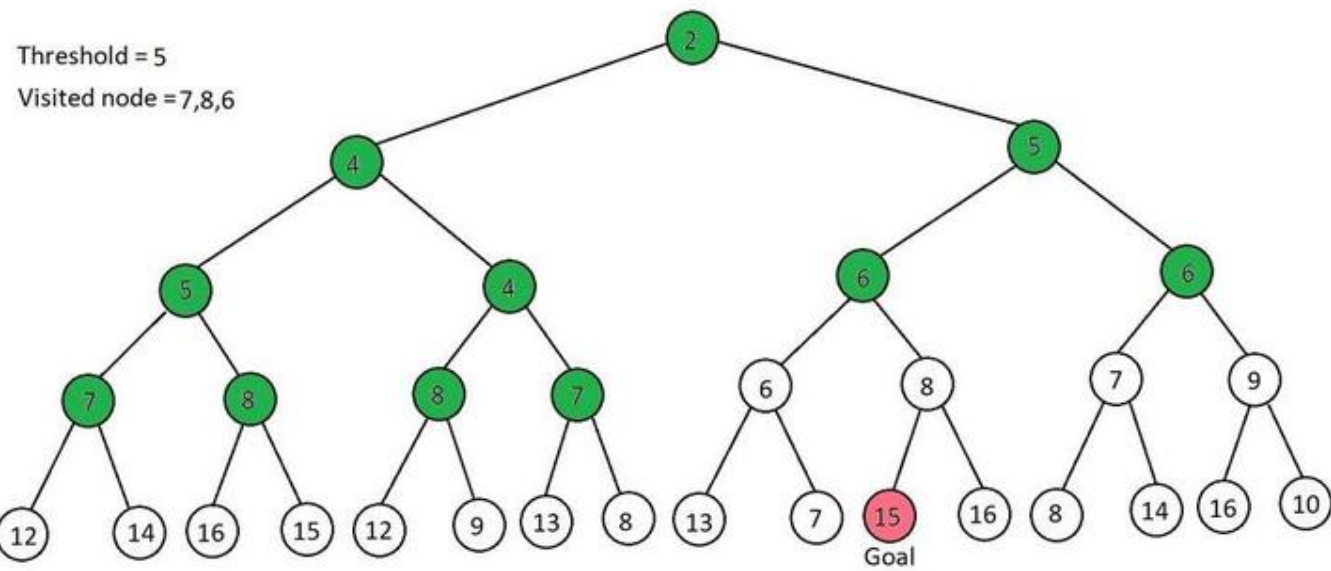
在深度优先遍历过程中，若节点的 f 值超过当前阈值，则对该节点进行剪枝，不再继续扩展其子节点，从而避免了不必要的搜索。

重复迭代直至目标

算法不断重复上述过程，逐轮加深搜索深度，直至找到目标节点为止，整个过程仅需保存当前分支的节点信息，内存占用极低。

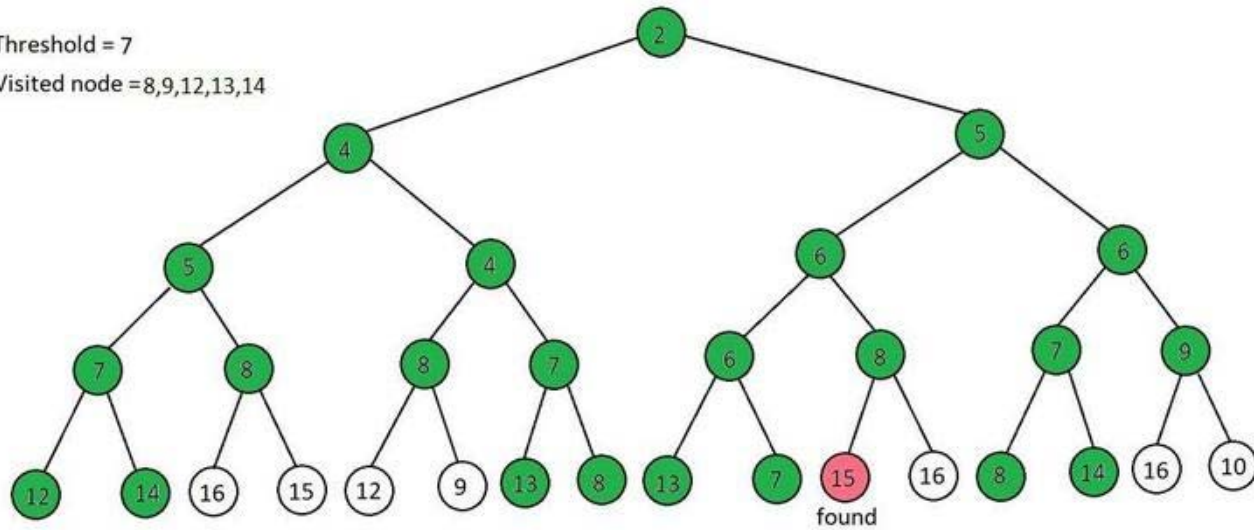
例子





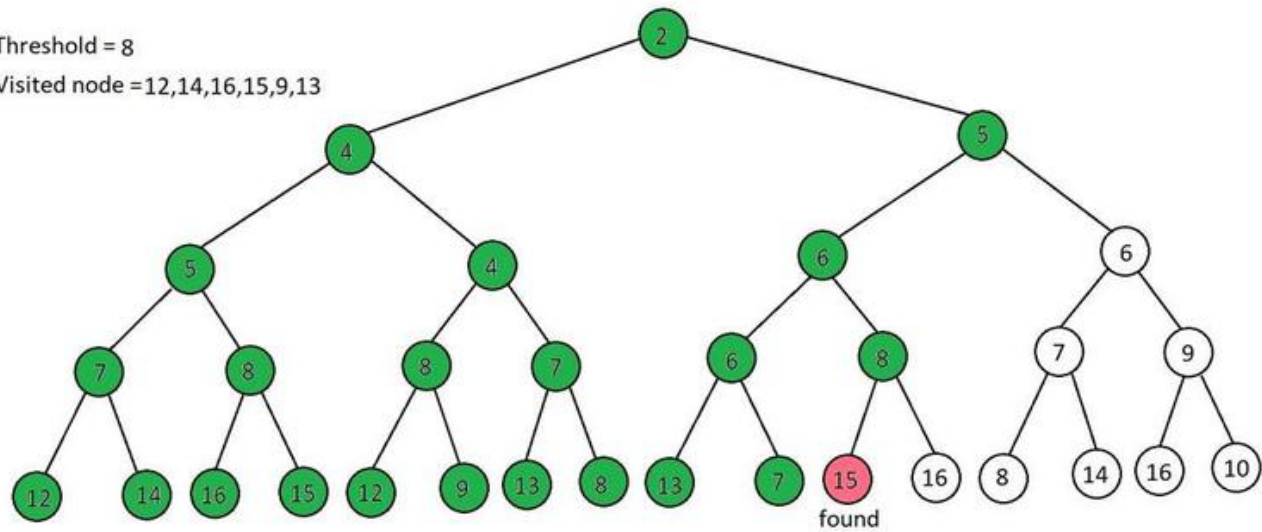
Threshold = 7

Visited node = 8,9,12,13,14



Threshold = 8

Visited node = 12,14,16,15,9,13



IDA*与IDDFS对比

搜索策略

IDDFS在无权图中采用盲目扩展的方式，而IDA*利用启发式函数进行剪枝，搜索更有针对性，效率更高。

最优性保证

IDDFS不保证找到最优解，而IDA*通过可接受的启发式函数 $h(n)$ ，确保了搜索过程的最优性，能够找到成本最低的路径。

节点访问次数

IDDFS可能会重复访问大量节点，导致搜索效率低下，而IDA*通过f值阈值策略，避免了对已访问节点的重复探索，提高了搜索效率。

适用场景

IDDFS适用于无权图或权重相同的图，而IDA*适用于加权图，尤其在权重差异较大时，其优势更为明显。



优势

最优路径保证

IDA*算法能够保证找到从起点到目标的最优路径，不会高估到达目标的成本，确保了搜索结果的准确性。

内存效率高

与A*算法相比，IDA*采用深度优先搜索技术，内存占用大幅减少，仅与搜索深度成正比，适用于内存受限的设备。

可处理大规模问题

IDA*通过启发式剪枝，能够有效处理无限或极大的搜索空间，解决了传统算法在大规模问题面前的无能为力。

启发式函数的作用

IDA*的启发式函数确保了算法的可接受性，优化了搜索性能，使得算法在复杂问题中能够快速找到接近最优的解。

局限剖析

1

节点重复探索

由于IDA*不存储已访问节点，可能会导致对某些节点的重复探索，增加了搜索的时间开销，尤其是在深层搜索中更为明显。

2

速度慢于A*

与A*算法相比，IDA*由于需要逐轮迭代加深搜索，可能会在某些情况下搜索速度稍慢，尤其是在启发式函数不够准确时。

3

计算成本高

IDA*在迭代过程中需要多次计算节点的f值，且随着迭代次数的增加，计算量也会相应增大，对处理器的性能要求较高。

内存受限搜索

Memory-bounded search

01

内存受限搜索的定义与使命

01

定义

内存受限搜索是一种启发式搜索技术，它在严格固定的内存预算内，通过动态管理和修剪搜索数据，高效地找到解决方案。这种技术特别适用于资源有限的设备，如嵌入式系统和实时系统。

02

使命

其使命是在嵌入式、实时或边缘设备中，让AI在资源极度受限的条件下仍能输出可行甚至最优解，确保在有限的内存资源下实现高效的搜索和决策。

与传统搜索的内存对比

1

A*算法

传统A*算法在复杂问题中因需维护开放和关闭列表，导致内存占用呈指数级增长，难以适用于内存受限的场景。

2

BFS算法

BFS算法因需存储整层节点，其内存占用为 $O(b^d)$ ，在深度较大的问题中会迅速耗尽内存资源。

3

DFS算法

DFS算法虽内存占用为 $O(d)$ ，但无法保证找到最短路径，且在某些情况下可能错过最优解。

内存监控与动态修剪

内存监控

算法实时统计开放列表与关闭列表的节点总数，一旦超过预设内存上限，立即触发修剪机制，确保内存占用始终在可控范围内。

动态修剪

按启发式值降序排序后删除后半部分最无希望的节点，通过这种策略在保持搜索质量与释放内存之间实现有效权衡。

节点结构

Node类封装了当前状态、父节点指针及到达该状态的动作，形成轻量级回溯链，确保在内存受限条件下仍能通过启发式引导搜索方向。

8拼图问题建模与合法移动



01

问题建模

将8拼图问题抽象为状态空间搜索，用长度为9的元组表示棋盘，0代表空格，定义moves字典列出每个位置可滑动的邻接索引，清晰定义问题的数据结构。



02

合法移动

通过moves字典，根据空格位置生成所有合法滑动后的新状态，并用Node封装新状态、父节点及对应动作，确保状态转移的合法性。

流 程

1、**定义节点类。** 搜索树中的每个节点都会跟踪当前状态、其父节点（用于重建解决方案路径）以及为达到该状态而采取的操作。

```
class Node:
    def __init__(self, state, parent=None, action=None):
        self.state = state
        self.parent = parent
        self.action = action
```

2、**曼哈顿距离启发式。** 该函数估计达到目标的成本（越低=越近）。

```
def heuristic(state):
    distance = 0
    for i in range(9):
        if state[i] != 0:
            distance += abs(i // 3 - (state[i] - 1) // 3) + abs(i % 3 - (state[i] - 1) % 3)
    return distance
```

3、**内存使用情况统计。** 计算开放列表和封闭列表中的节点数，确保搜索保持在内存限制内。

```
def memory_usage(open_list, closed_list):  
    return len(open_list) + len(closed_list)
```

4、**超出内存限制时修剪打开列表。** 当内存超出时：按启发式对开放列表进行排序（最差的优先），删除下半部分（最没有希望的节点）。

```
def prune_memory(open_list, closed_list):  
    open_list.sort(key=lambda x: heuristic(x.state), reverse=True)  
    open_list[:] = open_list[:len(open_list) // 2]
```

5、**选择当前最佳节点。** 选择看起来最有希望下一步扩展的节点。

```
def select_best_node(open_list):  
    return min(open_list, key=lambda x: heuristic(x.state))
```

6、**定义目标测试。** 检查节点的状态是否符合期望的目标并返回 True。

```
def is_goal(node):  
    return node.state == goal_state
```

7、生成后继状态。 为每个有效移动（滑动空白）生成新节点，每一步都会产生一个新的谜题状态。

```
def generate_successors(node):
    successors = []
    zero_index = node.state.index(0)
    for move in moves[zero_index]:
        new_state = list(node.state)
        new_state[zero_index], new_state[move] = new_state[move], new_state[zero_index]
        successors.append(Node(tuple(new_state), parent=node, action=move))
    return successors
```

8、避免冗余状态。 跳过已经见过的节点。

```
def redundant(successor, open_list, closed_list):
    for node in open_list + closed_list:
        if node.state == successor.state:
            return True
    return False
```

9、内存限制的 A 类搜索例程。 实现内存管理和修剪逻辑。在主循环中重复这些步骤：
如果超出内存限制则进行修剪；如果打开列表为空则停止；展开最佳节点；
如果目标，则返回节点；否则，将节点移至关闭状态，如果不冗余则添加新的后继节点。

```
def MemoryBoundedSearch(initial_state, memory_limit):
    node = Node(initial_state)
    open_list = [node]
    closed_list = []

    while open_list:
        if memory_usage(open_list, closed_list) > memory_limit:
            prune_memory(open_list, closed_list)
        if not open_list:
            return None
        current_node = select_best_node(open_list)
        if is_goal(current_node):
            return current_node
        open_list.remove(current_node)
        closed_list.append(current_node)
        for successor in generate_successors(current_node):
            if not redundant(successor, open_list, closed_list):
                open_list.append(successor)
    return None
```


10、**设置拼图定义。**定义目标状态和合法动作。

```
goal_state = (1, 2, 3, 4, 5, 6, 7, 8, 0)

moves = {
    0: [1, 3],
    1: [0, 2, 4],
    2: [1, 5],
    3: [0, 4, 6],
    4: [1, 3, 5, 7],
    5: [2, 4, 8],
    6: [3, 7],
    7: [4, 6, 8],
    8: [5, 7]
}
```

11、测试并获取结果。 定义 8 个拼图的动作和目标状态；
使用指定的内存限制运行搜索；如果找到则打印解决方案路径。

```
initial_state = (1, 2, 3, 4, 5, 6, 0, 7, 8)

print("Case: Memory Limit = 1")
goal_node = MemoryBoundedSearch(initial_state, memory_limit=1)
if goal_node:
    print("Solution found!")
    while goal_node.parent:
        print("Action:", goal_node.action)
        print("State:", goal_node.state)
        goal_node = goal_node.parent
else:
    print("Memory limit exceeded. No solution found.")

print("\nCase: Memory Limit = 10")
goal_node = MemoryBoundedSearch(initial_state, memory_limit=10)
if goal_node:
    print("Solution found!")
    while goal_node.parent:
        print("Action:", goal_node.action)
        print("State:", goal_node.state)
        goal_node = goal_node.parent
else:
    print("Memory limit exceeded. No solution found.")
```

后继生成与冗余过滤

后继生成

`generate_successors` 函数根据空格位置生成所有合法滑动后的新状态，并用Node封装新状态、父节点及对应动作，为搜索提供候选节点。

冗余过滤

`redundant`函数跳过已在开放或关闭列表中出现的状态，避免重复探索，确保在有限内存中只保留最具潜力的搜索分支。

效率提升

通过冗余过滤，算法避免了不必要的重复计算，显著提高了搜索效率，尤其在内存受限的环境中表现突出。

主循环逻辑

`MemoryBoundedSearch`主循环在每次迭代中先检查内存占用，超限则调用`prune_memory`；随后选择启发式值最小的节点扩展；若为目标立即回溯路径；否则将节点移至关闭列表并追加非冗余后继。

优势

01

内存效率

严格固定内存占用，适用于实时与嵌入式系统，确保在有限资源下仍能高效运行，提升系统稳定性。

02

可行解输出

通过启发式修剪，即使在内存受限条件下，仍能输出可行解，确保任务的基本完成。

03

易于部署

实现逻辑简洁，易于在硬件受限环境中部署与调试，降低开发与维护成本。

局限

计算开销

可能错过最优解

激进修剪可能错过最优解，需要在修剪策略上进行优化，以平衡解的质量与内存占用。

频繁内存检查带来额外计算开销，未来可通过优化检查频率或采用更高效的内存管理策略来缓解。

实现复杂性

实现需精细设计排序与淘汰策略，未来可与机器学习结合预测节点潜力，或利用并行硬件同时维护多条轻量搜索线。

。



局 部 搜 索 算 法

禁忌搜索

T A B U S e a r c h

局部搜索困境与记忆机制

局部搜索局限

传统局部搜索在复杂优化问题中容易陷入循环或局部最优，导致算法反复访问相同解，无法有效前进，限制了解的多样性。

记忆机制引入

禁忌搜索通过引入记忆机制，记录近期访问过的解或移动特征，强制搜索避开这些区域，打破循环，拓展探索空间。

算法改进思路

这种记忆机制使得禁忌搜索能够避免重复探索，从而在有限的计算资源内更高效地寻找全局最优解。

Tabu search (禁忌搜索)

- 从一个初始可行解出发，选择一系列的特定搜索方向（移动）作为试探，选择实现让特定的目标函数值变化最多的移动。
- 为了避免陷入局部最优解，TS搜索中采用了一种灵活的“记忆”技术，对已经进行的优化过程进行记录 and 选择，指导下一步的搜索方向，这就是Tabu表的建立。
- 标记已经解得的局部最优解或求解过程，并在进一步的迭代中避开这些局部最优解或求解过程。局部搜索的缺点在于，太过于对某一局部区域以及其邻域的搜索，导致一叶障目。为了找到全局最优解，禁忌搜索就是对于找到的一部分局部最优解，有意识地避开它，从而或得更多的搜索区域。

Tabu search : 相关概念

□邻域

- 所谓邻域，简单的说即是给定点附近其他点的集合。邻域就是指对当前解进行一个操作(这个操作可以称之为邻域动作)可以得到的所有解的集合。

□邻域动作

- 邻域动作是一个函数，通过这个函数，对当前解 s ，产生其相应的邻居解集合。
- 例如：对于一个bool型问题，其当前解为： $s = 1001$ ，当将邻域动作定义为翻转其中一个bit时，得到的邻居解的集合 $N(s) = \{0001, 1101, 1011, 1000\}$ ，其中 $N(s) \in S$ 。

Tabu search : 相关概念

□ 禁忌表

- 禁忌对象：由于需要避免一些操作的重复进行，就要将一些元素放到禁忌表中以禁止对这些元素进行操作，这些元素就是我们指的禁忌对象（通常指找到的局部最优解）。
- 禁忌长度：禁忌长度是被禁对象不允许选取的迭代次数。一般是给被禁对象 x 一个数（禁忌长度） t ，要求对象 x 在 t 步迭代内被禁，在禁忌表中采用 $\text{tabu}(x)=t$ 记忆，每迭代一步，该项指标做运算 $\text{tabu}(x)=t-1$ ，直到 $\text{tabu}(x)=0$ 时解禁。

□ 候选集合

- 候选集合由邻域中的邻居组成。常规的方法是从邻域中选择若干个目标值或评价值最佳的邻居入选。

Tabu search : 相关概念

□ 评价函数（目标函数）

- 评价函数是候选集合元素选取的一个评价公式，候选集合的元素通过评价函数值来选取。

□ 特赦规则

- 在禁忌搜索算法的迭代过程中，会出现候选集中的全部对象都被禁忌，或有一对象被禁，但若解禁则其目标值将有非常大的下降情况。在这样的情况下，为了达到全局最优，我们会让一些禁忌对象重新可选。这种方法称为特赦，相应的规则称为特赦规则。

禁忌列表存储策略

存储方式

禁忌列表的存储方式有完整解存储和移动特征存储两种，后者通过记录交换位置等抽象信息节省空间。

特赦准则作用

当某禁忌解优于历史全局最优时，即使位于禁忌列表也可被接受，防止因过度禁忌而错失更优解。

权衡影响

列表大小影响探索广度与计算开销，需根据问题规模与内存限制选择合适的存储策略。

复杂度分析

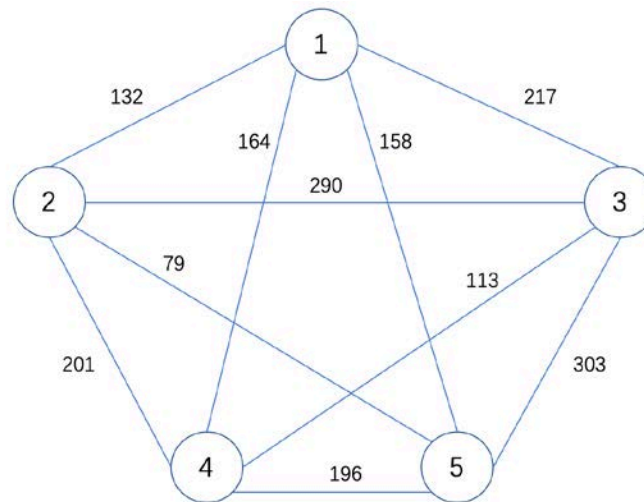
时间复杂度通常为 $O(\text{迭代} \times \text{邻域})$ ，空间复杂度为 $O(\text{禁忌长度})$ ，需权衡解质量与资源消耗。

Tabu search : 算法

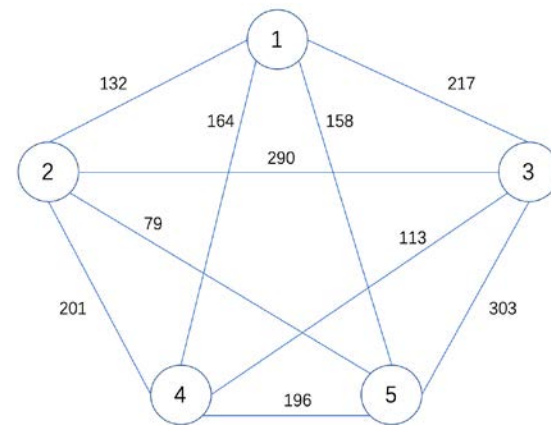
- Step1 : 禁忌表 H =空集 , 并选定一个初始解 x_1 ;
- step2 : 在 x_i 的邻域 $N(x_i)$ 中选择候选集 $Can_N(x_i)$; 在 $Can_N(x_i)$ 中选一个评价值最佳的解 x_{i+1} ,
 - x_{i+1} 不在禁忌表中 , 选为当前解
 - x_{i+1} 在禁忌表中 , 满足破禁条件 , x_{i+1} 为当前解 , 否则从不在禁忌表解中 , 选择一个最优解
- step3 : 重复step2 , 直到满足停止条件

Tabu search : 例子

□求下图的旅行商回路



Tabu search : 例子

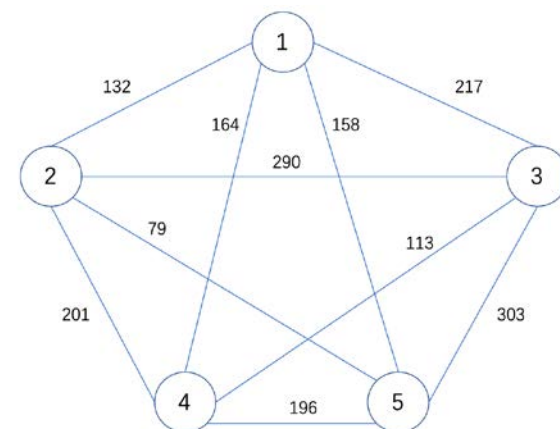


□ 随机选择一个城市序列：21543，路径长度889

交换城市	长度
1↔5	804
1↔5	804
3↔4	907

禁忌表	
1	5↔1
2	
3	

Tabu search : 例子



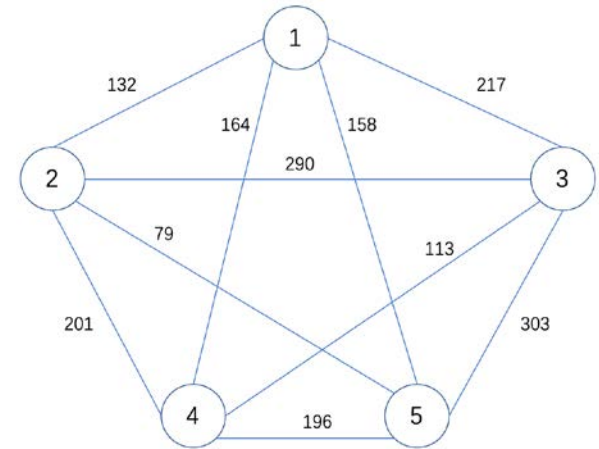
□ 最优访问顺序：25143，最优长度804

□ 当前访问顺序：25143，长度804

交换城市	长度
5↔4	1098
1↔4	928
5↔1	889

禁忌表	
1	4↔1
2	5↔1
3	

Tabu search : 例子



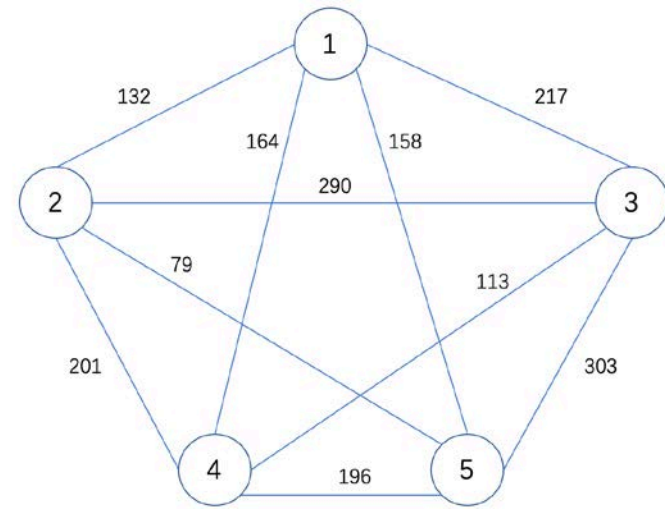
□ 最优访问顺序：25143，最优长度804

□ 当前访问顺序：25413，长度928

交换城市	长度
5↔1	1085
5↔4	1062
5↔3	786

禁忌表	
1	5↔3
2	4↔1
3	5↔1

Tabu search : 例子



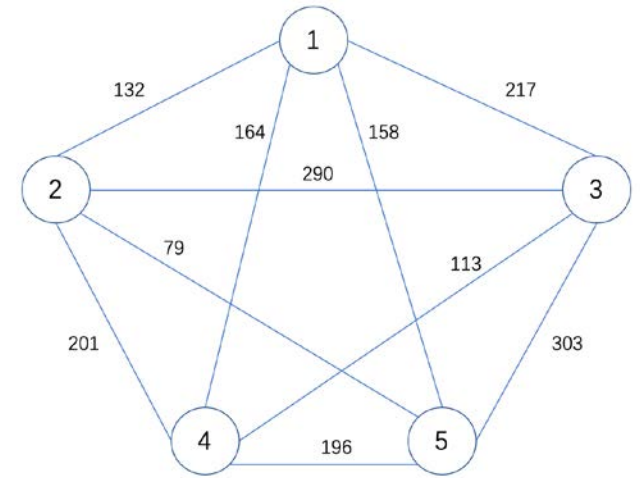
□ 最优访问顺序：23415，最优长度786

□ 当前访问顺序：23415，长度786

交换城市	长度
4↔1	946
3↔4	768
2↔3	1098

禁忌表	
1	3↔4
2	5↔3
3	4↔1

Tabu search : 例子



□ 最优访问顺序：24315，最优长度768

□ 当前访问顺序：24315，长度768

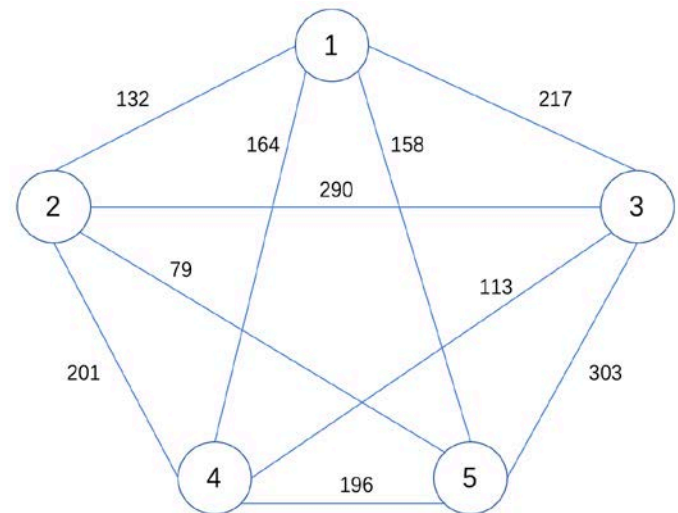
交换城市	长度
4↔5	964
4↔1	737
1↔5	907

破
禁

禁忌表	
1	4↔1
2	3↔4
3	5↔3

Tabu search : 例子

□ 迭代5次最优访问顺序：21345，最优长度737



优势与局限

优势一

通过记忆避免循环，适用于离散与连续混合空间，实现简单易于嵌入现有系统。

局限一

迭代次数与参数敏感，需要根据具体问题
进行精细调优。

优势二

禁忌搜索能够有效避免局部最优，通过记忆机制拓展探索空间。

局限二

最坏复杂度仍可能指数级，对于大规模问题效率有待提升。

爬山算法

H i l l C l i m b i n g

01

爬山法定义与灵感来源



01

爬山法定义

爬山法是一种启发式局部搜索算法，通过不断向邻近更优状态移动来逼近最优解。它在人工智能中被广泛用于优化问题，因其简单高效而备受青睐。



02

灵感来源

其灵感源自自然界‘登高望远’的直观体验，算法仅依赖局部信息即可在庞大解空间中快速定位高峰，为复杂问题提供高效解决方案。

核心思想与迭代框架

迭代过程

核心思想

爬山法的核心在于每一步都选择当前最优的邻近状态，逐步提升目标函数值，直至无法进一步改进。

从任意初始解出发，生成邻近候选，比较评估函数并迁移至更优邻居，这一过程不断重复，直至达到局部最优。

收敛条件

当所有邻近状态都无法提供比当前状态更好的解时，算法收敛，此时可能已到达局部最大值。

简单、最陡与随机爬山

简单爬山

简单爬山法顺序考察邻居，一旦发现首个改进即采纳，快速但可能错过更好解。

最陡爬山

最陡爬山法评估全部邻居，选择最佳改进，计算量大但收敛精度高。

随机爬山

随机爬山法随机挑选邻居，依概率迁移，可跳出局部最优，但收敛速度较慢。

状态空间图与极值陷阱

01

状态空间图

状态空间图直观展示了所有可能状态及其目标函数值，帮助理解算法行为。

02

局部最大值

局部最大值比邻近状态更好，但并非全局最优，可能导致算法停滞。

03

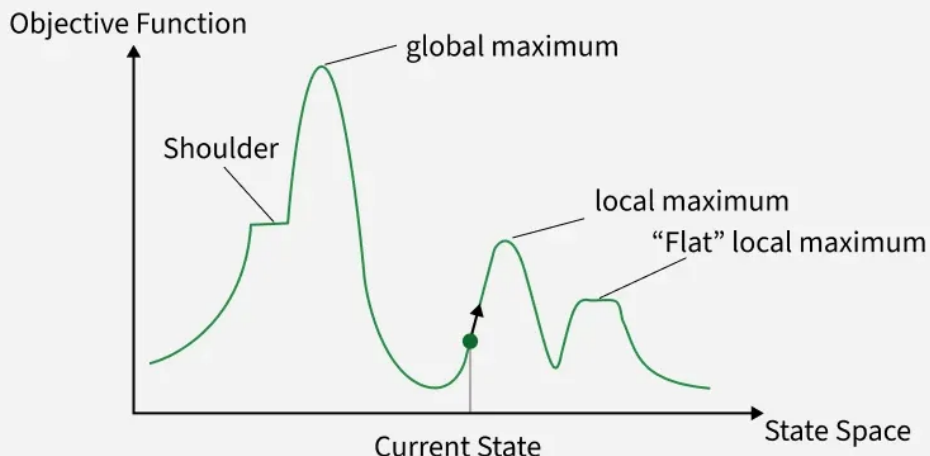
高原

高原区域相邻状态目标值相同，使算法难以决定最佳移动方向。

04

山脊

山脊区域坡度高，算法可能在此震荡，错过附近更好解。



优势

实现简单

爬山法易于编码调试，适合初学者快速上手，降低开发门槛。

适用广泛

对连续或离散优化问题均适用，灵活性高，可快速适配不同领域。

资源高效

内存占用低，仅需存储当前状态与少量邻居，适合资源受限场景。

快速反馈

能快速给出局部最优解，为复杂问题提供即时反馈与初步方案。

局限剖析

局部最优

易陷入局部最优，导致全局次优，需结合全局策略缓解。

初始敏感

对初始解敏感，不同起点可能导致截然不同的结果，需谨慎选择。

地形陷阱

在高原或山脊地形易停滞，需引入随机性或重启策略突破。



模拟退火

Simulated Annealing

01

传统优化困境与退火启示

01

传统优化方法的局限性

传统优化方法如梯度下降和穷举法在面对高维、离散、非凸的复杂搜索空间时，常常陷入局部最优解，难以找到全局最优解。这些问题在实际应用中广泛存在，如芯片布局、物流配送等。

模拟退火的灵感来源

模拟退火算法的灵感来源于冶金学中的退火过程。在高温下，材料的原子可以自由移动并重新排列，随着温度的逐渐降低，材料逐渐冷却并结晶，最终形成稳定的晶体结构。这一过程启发了模拟退火算法的设计。

02

03

模拟退火的核心动机

模拟退火算法通过引入温度参数，控制搜索过程中的随机性，从而在早期大胆探索，后期精细收敛，避免陷入局部最优，逐步逼近全局最优解。

模拟退火

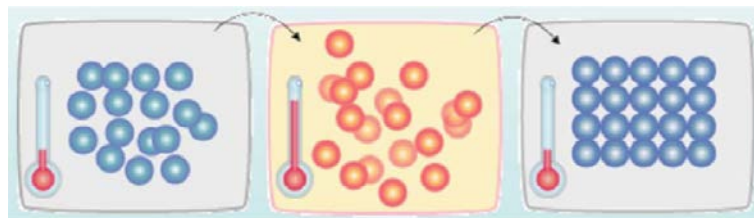
□什么是退火——物理上的由来

- 退火（annealing）现象指物体逐渐降温的物理现象，温度愈低，物体的能量状态会低；够低后，液体开始冷凝与结晶，在结晶状态时，系统的能量状态最低。大自然在缓慢降温（亦即，退火）时，可“找到”最低能量状态：结晶。但是，如果过程过急过快，快速降温（亦称「淬炼」，quenching）时，会导致不是最低能态的非晶形。

模拟退火

□什么是退火——物理上的由来

- 如下图所示，首先（左图）物体处于非晶体状态。我们将固体加温至充分高（中图），再让其徐徐冷却，也就退火（右图）。加温时，固体内部粒子随温升变为无序状，内能增大，而徐徐冷却时粒子渐趋有序，在每个温度都达到平衡态，最后在常温时达到基态，内能减为最小（此时物体以晶体形态呈现）



模拟退火

□ 算法概述

- 若目标函数 f 在第 $i+1$ 步移动后比第 i 步更优，即 $f(Y(i+1)) \leq f(Y(i))$ ，则总是接受该移动；
- 若 $f(Y(i+1)) > f(Y(i))$ （即移动后的解比当前解要差），则以一定的概率接受移动，而且这个概率随着时间推移逐渐降低（逐渐降低才能趋向稳定），从而在后期更倾向于接受更优解。

Metropolis 准则

模拟退火

□ Metroplis 准则

- 温度越高，算法接受新解的概率越高。这个根据一定概率选择是否接受差解的方法叫做 Metropolis 准则

$$p = e^{-\frac{\Delta E}{kt}}$$

温度与能量映射关系

能量映射

在模拟退火中，目标函数值被映射为物理系统的能量。能量越低，解的质量越高。通过这种方式，算法的目标转化为寻找能量最低的稳定状态。

温度的作用

温度参数控制了搜索过程中的随机性。高温时，算法可以接受较大的能量变化，从而进行广泛的探索；低温时，算法趋于稳定，只接受较小的能量变化。

冷却调度策略

指数冷却策略

冷却策略的重要性

冷却策略决定了温度如何随时间降低。合适的冷却策略可以平衡算法的探索能力和收敛速度，避免过早收敛或搜索效率低下。

指数冷却策略 $T \leftarrow \alpha T$ 是最常用的冷却方式之一。其中 α 是降温系数，通常取值在 0.8 到 0.99 之间。这种策略简单且有效，能够逐步降低温度，使算法逐渐收敛。

冷却参数的调整

α 的取值对算法性能有重要影响。较大的 α 值使冷却过程更缓慢，有助于更充分的探索；较小的 α 值则加快冷却速度，提高算法的收敛速度。

流程

1

初始化

初始化温度和随机解。选择一个初始解作为起点，并设置一个足够高的初始温度，以便在搜索初期进行广泛的探索。

2

邻域扰动

在当前温度下，对当前解进行扰动，生成邻域解。扰动方式需根据具体问题设计，如连续变量的微移或离散变量的交换。

3

概率接受

根据Metropolis准则，以概率 $e^{(-\Delta E/T)}$ 接受新解。如果新解更优，则直接接受；如果新解较差，则有一定概率被接受。

4

降温

按照预设的冷却策略降低温度。温度的降低使得接受劣解的概率逐渐减小，算法逐渐趋于稳定。

5

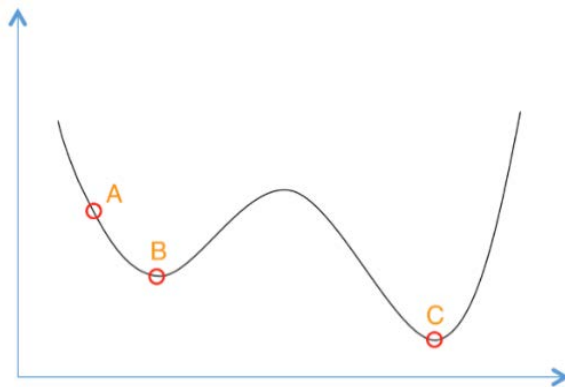
终止条件

当温度降至最低温度或达到预设的迭代次数时，算法终止。此时，当前解即为算法找到的近似最优解。

模拟退火

□ 贪心算法

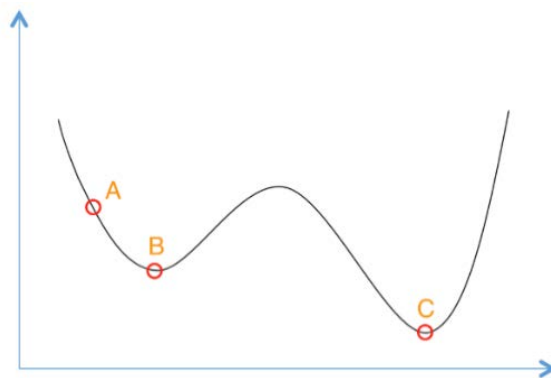
- A点开始试探，如果函数值继续减少，那么试探过程就会继续。而当到达点B时，显然我们的探求过程就结束了找不到全局最优解



模拟退火

□模拟退火

- Greedy算法，但是加入随机因素。以一定的概率来接受一个比当前解要差的解，因此有可能会跳出这个局部的最优解，达到全局的最优解。
- 以上图为例，模拟退火算法在搜索到局部最优解B后，会以一定的概率接受向右继续移动。也许经过几次这样的不是局部最优的移动后会到达B 和C之间的峰点，于是就跳出了局部最小值B



邻域扰动设计

01

邻域函数的重要性

邻域函数的设计对模拟退火算法的性能至关重要。合适的邻域函数可以确保算法在解空间中进行有效的探索。

02

连续变量的扰动

对于连续变量，可以通过在当前值基础上添加小的随机扰动来生成邻域解，例如在当前值附近随机移动一小段距离。

03

离散变量的扰动

对于离散变量，可以通过交换、逆序或随机选择等操作来生成邻域解，例如在序列中交换两个元素的位置。

04

布局问题的扰动

在布局问题中，可以通过移动、旋转或交换模块的位置来生成邻域解，从而探索不同的布局配置。

关键参数设定

初始温度

初始温度应足够高，以便在搜索初期接受大多数邻域解，从而进行广泛的探索。通常设置为1或更高。

终止温度

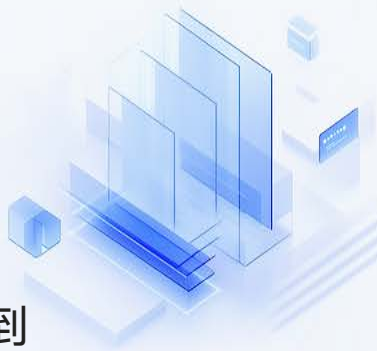
终止温度应接近零，表示算法冷却到足够低的温度，此时算法趋于稳定，不再接受劣解。通常设置为 10^{-4} 或更低。

降温系数

降温系数 α 控制温度的下降速度。较大的 α 值使冷却过程更缓慢，有助于更充分的探索；较小的 α 值则加快冷却速度，提高算法的收敛速度。

每温迭代次数

每温迭代次数决定了在每个温度下进行多少次邻域扰动和接受操作。这个参数需要根据问题的复杂度和计算资源进行平衡。



模拟退火：旅行商问题

□旅行商问题

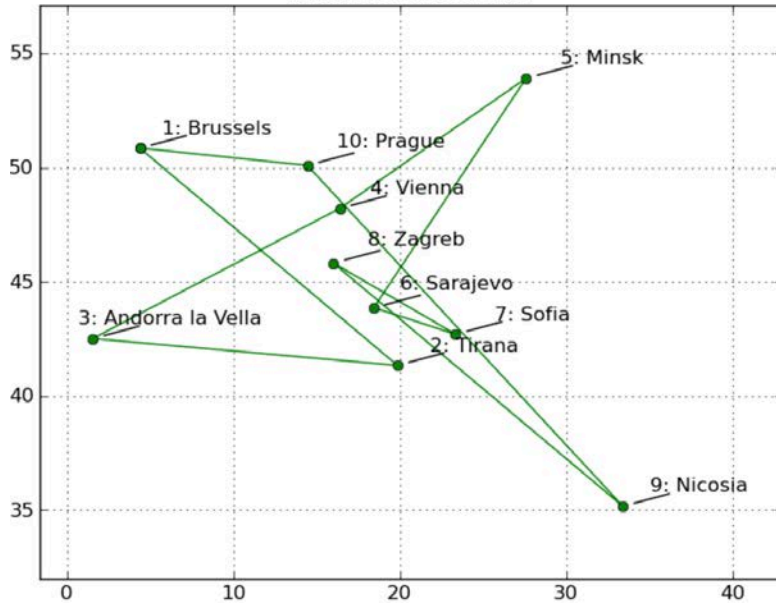
- 输入：旅行图、初始温度、最小温度、降温系数
- 输出：旅行回路

□算法

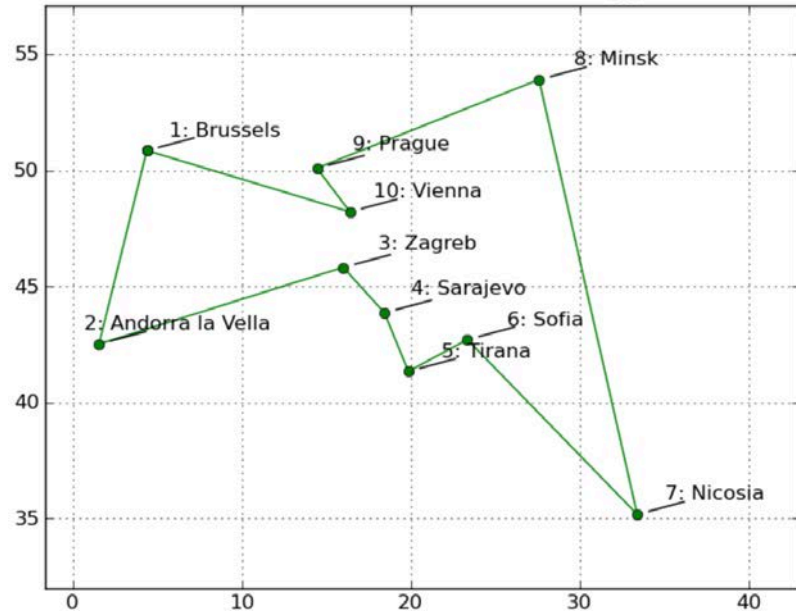
- 随机选择城市顺序
- 在第 i 步的基础上，随机交换两个（或多个）城市的顺序（移动一步）
- 如果第 $i+1$ 步的代价更小，则作为当前哈密顿回路，否则依据Metropolis准则定义的概率作为当前回路，重复执行此步骤 n 次
- 按照降温系数降低温度，重复以上步骤直到迭代次数达到预设值或者温度下降到预设值

模拟退火：旅行商问题

Initial tour on 10 cities
Distance: 12699 km



Optimal tour on 10 cities
Distance: 8588 km on 1 iteration(s)



全局搜索优势

无需梯度信息

模拟退火算法不依赖梯度信息，适用于目标函数不可导或梯度难以计算的问题，如离散优化问题。

跳出局部最优

通过概率接受机制，算法可以在搜索过程中跳出局部最优，探索更广泛的解空间，从而更接近全局最优。

实现简单

模拟退火算法的实现相对简单，只需定义目标函数、邻域函数和冷却策略即可。

适用范围广

该算法适用于离散和连续混合的优化问题，如芯片布局、物流配送路径规划等。

收敛速度与调参挑战



收敛速度慢

模拟退火算法的收敛速度相对较慢，尤其是在冷却过程较为缓慢时，需要较多的迭代次数才能找到较好的解。



参数敏感

算法的性能对初始温度、降温系数、每温迭代次数等参数较为敏感，需要根据具体问题进行精细调整。



邻域设计依赖高

邻域函数的设计对算法性能影响较大，需要根据具体问题精心设计，以确保算法能够有效探索解空间。

混合算法与未来方向

混合算法

将模拟退火与遗传算法、粒子群优化等其他优化算法结合，可以发挥各自的优势，提高全局搜索能力和收敛速度。

自适应冷却策略

开发自适应冷却策略，根据搜索过程中的动态信息自动调整温度，可以进一步提高算法的性能。

并行退火

利用并行计算技术，同时运行多个退火过程，可以加速算法的收敛，提高解的质量。

未来展望

模拟退火算法在复杂优化问题中具有广阔的应用前景，未来可结合更多领域知识和先进技术，不断拓展其应用范围和性能。

遗传算法

Genetic Algorithms

01

从自然选择到计算进化

01

遗传算法的灵感来源

遗传算法借鉴了达尔文的自然选择理论，通过模拟生物进化过程来解决优化问题。它将自然界的适者生存机制引入计算领域，为复杂问题的求解提供了全新的视角。

02

算法的基本思想

遗传算法通过种群进化来搜索最优解。每个个体代表一个可能的解决方案，通过选择、交叉和变异操作，不断优化种群的适应度，最终收敛到全局最优解。

核心思想

01

种群竞争资源

在遗传算法中，种群中的个体需要竞争有限的资源，如繁殖机会。适应度较高的个体更有可能生存下来并繁衍后代，从而推动种群进化。

02

高适应度个体优先繁殖

遗传算法根据个体的适应度来分配繁殖概率。适应度高的个体有更高的机会参与交配，产生后代，从而将优良基因传递给下一代。

03

基因跨代传播

通过交叉和变异操作，个体的基因可以在代际之间传播。这使得优良基因能够在种群中扩散，提高整个种群的适应度。

04

新一代整体更优

每一代种群都比前一代更适应环境。随着进化过程的推进，种群的整体适应度不断提高，最终收敛到最优解。

适应度函数设计原则

01

适应度函数的作用

适应度函数用于量化个体的优劣程度，是遗传算法中评价个体的重要标准。它决定了个体在种群中的生存和繁殖概率。

设计原则

适应度函数的设计需要满足可计算、。只有这样，才能有效地引导种群向最优解进化。可区分和与目标一致的原则

02

03

对算法的影响

适应度函数的设计直接影响遗传算法的收敛速度和结果质量。一个好的适应度函数可以使算法更快地找到全局最优解。

选择算子：优者胜出

选择方法

选择算子的功能

选择算子根据个体的适应度来选择参与交配的个体。适应度高的个体有更高的概率被选中，从而将优良基因传递给下一代。

常见的选择方法有轮盘赌选择、排序选择和锦标赛选择等。不同的选择方法对种群的多样性有不同的影响。

对种群多样性的影响

选择算子的强度需要适中。过强的选择压力可能导致种群过早收敛，而过弱的选择压力则会使进化过程缓慢。

交叉算子与变异算子

交叉方法

交叉算子的作用

交叉算子模拟生物的交配过程，通过交换两个父代个体的部分基因，生成新的子代个体。这有助于快速组合优良基因，加速进化过程。

常见的交叉方法有单点交叉、多点交叉和均匀交叉等。不同的交叉方法对子代的基因组合有不同的影响。

变异算子的作用

变异算子通过随机改变个体的某些基因，引入新的遗传变异。这有助于保持种群的多样性，防止算法陷入局部最优。

初始化与迭代框架

算法的初始化过程

遗传算法从随机生成初始种群开始，每个个体都代表一个可能的解决方案。然后计算每个个体的适应度，为后续的进化过程做准备。

迭代进化框架

遗传算法通过选择、交叉、变异和替换等操作，不断迭代进化。每一代种群都比前一代更适应环境，最终收敛到最优解。

收敛判定与终止策略

收敛判定标准

遗传算法可以通过多种方式判断是否收敛，如达到最大迭代次数、适应度达到阈值或种群多样性降低等。

终止策略

当算法满足收敛条件时，可以终止进化过程。此时，种群中的最优个体即为问题的近似最优解。

早熟收敛的应对

如果算法过早收敛到局部最优解，可以通过调整参数或引入多样性机制来避免早熟收敛，提高算法的全局搜索能力。

遗传算法：流程

Algorithm 95 遗传算法

- 1: **Input:** 适应度函数 f , 种群个体数目 n , 个体编码长度 m , 最大迭代次数 t_{max}
种群差异阈值 t_{th} , 交叉概率 p_c , 变异概率 p_m ;
- 2: **Output:** 最优解;
- 3: **初始化:** $t \leftarrow 0$, 按照编码随机生成 n 的个体 $\mathcal{X}^0 = \{x_1^0, x_2^0, \dots, x_n^0\}$;
- 4: **repeat**
- 5: /* 选择流程 */
- 6: $\mathcal{X}^{t+1} \leftarrow$ 在 $\mathcal{X}^t$ 中按照轮盘赌算法选择 n 个个体;
- 7: /* 单点交叉流程 */
- 8: **for** $i=1$ to $n-1$ **do**
- 9: **if** $p_c \geq \text{Random}[0, 1]$ **then**
- 10: 随机选择一个交叉点: $c = \text{Random}[1, \dots, m]$;
- 11: 对 x_i^{t+1} 和 x_{i+1}^{t+1} 在 c 点进行单点交叉;
- 12: **end if**
- 13: **end for**

遗传算法：流程

```
14:  /* 变异流程 */
15:  for  $\mathcal{X}^{t+1}$  中的每一个个体  $x_i^{t+1}$  do
16:      for j=1 to  $m$  do
17:          if  $p_m \geq \text{Random}[0, 1]$  then
18:              对基因  $x_i^{t+1}.j$  进行变异（如果是 0, 1 基因，进行翻转即可）
19:          end if
20:      end for
21:  end for
22:   $t++$ ;
23: until  $t = t_{max}$  或者  $|\mathcal{X}^t - \mathcal{X}^{t-1}| \leq t_{th}$ 
24: 选择  $\mathcal{X}^t$  中的最优个体  $x^{opt}$ ;
25: return 对个体  $x^{opt}$  进行解码;
```

遗传算法：应用

求解函数的最大/最小值

$$\max \quad x \cdot \sin(10\pi \cdot x) + 5$$

$$s.t. \quad 2 \geq x \geq -2$$

要求的精度是小数点后 5 位

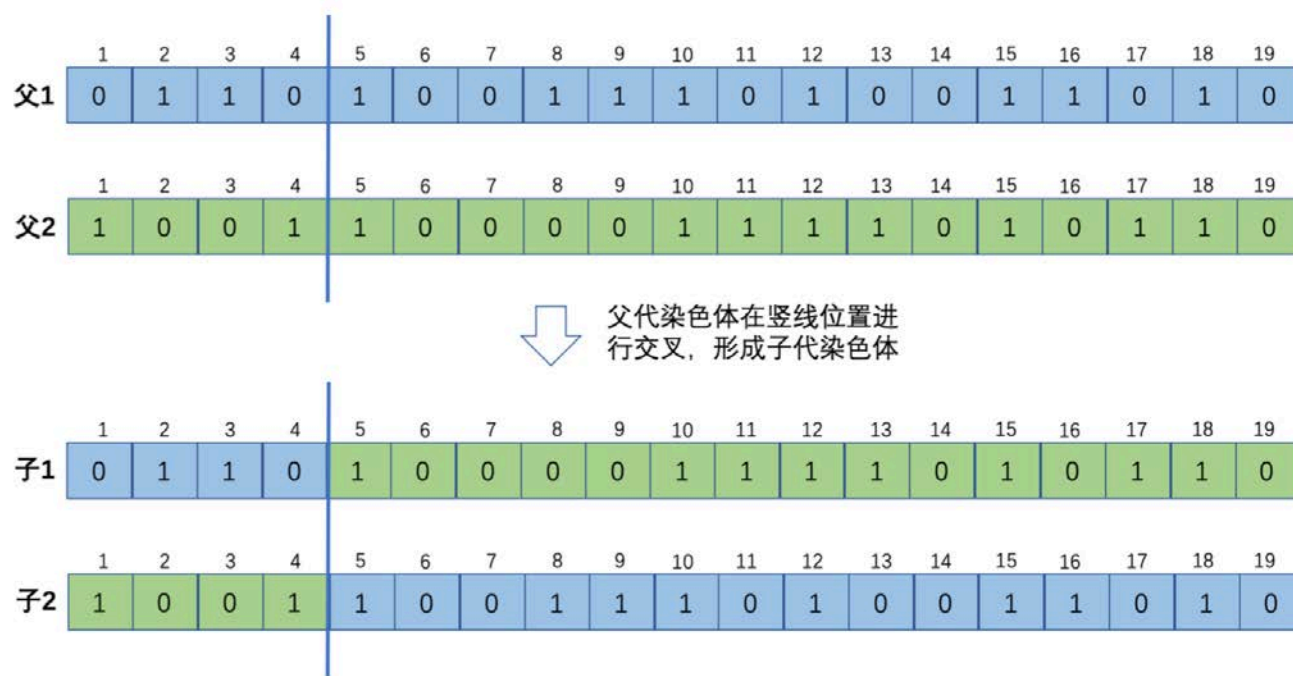
遗传算法：应用

将区间 $[-2, 2]$ 划分 4×10^5 等份，因为

$2^{18} < 4 \times 10^5 < 2^{19}$ ，所以编码的长度应该设置为 $m = 19$ 比特

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	1	1	0	1	0	0	1	1	1	0	1	0	0	1	1	0	1	0

遗传算法：应用



父代个体 1（父 1）代表的解为 0.1673，父代个体 2（父 2）代表的解为 1.12278，通过单点交叉后生成子代个体 1（子 1）代表的解为 0.13974，子代个体 2（子 2）代表的解为 1.15034。

遗传算法：旅行商问题

个体

旅行商问题的个体定义是比较直观的，如有 10 个城市，将个城市从 1 到 10 进行编号，每个编号就是个体的基因，所以一个个体就是 1 到 10 的一个排序，代表了城市的访问顺序，

	1	2	3	4	5	6	7	8	9	10
个体1	4	8	7	2	5	1	10	3	6	9
个体2	8	2	5	9	1	6	3	10	7	4

遗传算法：旅行商问题

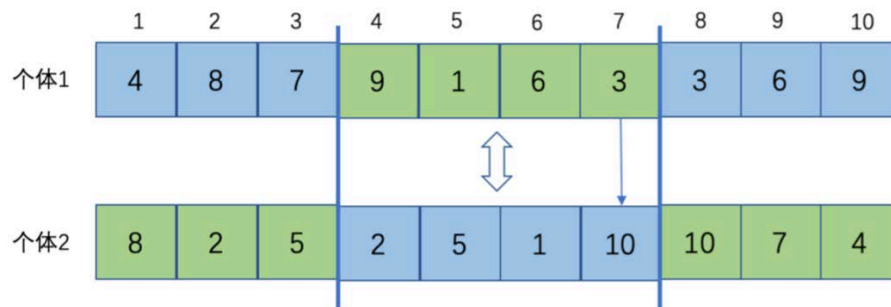
交叉

	1	2	3	4	5	6	7	8	9	10
个体1	4	8	7	9	1	6	3	10	7	4
个体2	8	2	5	2	5	1	10	3	6	9

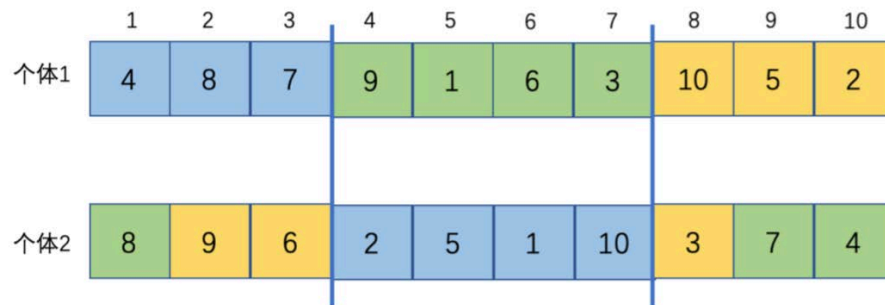
可能出现非法解

遗传算法：旅行商问题

部分匹配交叉 (Partially Mapped Crossover)



(a)



(b)

遗传算法：旅行商问题

顺序交叉 (Order Crossover)

	1	2	3	4	5	6	7	8	9	10
个体1	4	8	7	2	5	1	10	3	6	9

	1	2	3	4	5	6	7	8	9	10
个体2	8	2	5	9	1	6	3	10	7	4

↓ 顺序交叉

	1	2	3	4	5	6	7	8	9	10
个体1				9	1	6	3			

	1	2	3	4	5	6	7	8	9	10
个体2				2	5	1	10			

	1	2	3	4	5	6	7	8	9	10
个体1	2	5	10	9	1	6	3	4	8	7

	1	2	3	4	5	6	7	8	9	10
个体2	9	6	3	2	5	1	10	7	4	8

遗传算法：旅行商问题

变异 $m_1 = 3$ 和 $m_2 = 7$

1	2	<u>3</u>	4	5	6	<u>7</u>	8	9	10
4	8	7	9	1	6	3	10	5	2

未变异前的个体

1	2	<u>3</u>	4	5	6	<u>7</u>	8	9	10
4	8	3	9	1	6	7	10	5	2

方法一变异后的个体

1	2	<u>3</u>	4	5	6	<u>7</u>	8	9	10
4	8	7	6	1	9	3	10	5	2

方法二变异后的个体

蚁群优化算法

Ant Colony Optimization

01

蚁群觅食行为启发算法

蚂蚁觅食的随机探索

蚂蚁最初在巢穴周围随机寻找食物，这种无序的探索为后续路径选择奠定了基础。它们通过不断尝试，开辟了从巢穴到食物源的多条路径，为群体智慧的汇聚提供了可能。

信息素的正反馈机制

蚂蚁在返回巢穴时会在路径上释放信息素，信息素浓度越高，后续蚂蚁选择该路径的概率越大。这种正反馈机制使得优质路径逐渐凸显，群体逐渐达成共识。

路径优化的自然选择

随着时间推移，较短路径上的信息素因蚂蚁频繁往返而不断积累，而较长路径上的信息素因蒸发而逐渐减少。最终，整个蚁群会倾向于选择最短路径，实现路径优化。



从生物到算法的映射

蚂蚁与搜索代理的对应

在蚁群优化算法中，每只蚂蚁被抽象为一个搜索代理，它们在解空间中随机移动，通过信息素交流来寻找最优解，这种群体协作方式与自然界的蚂蚁觅食行为高度相似。

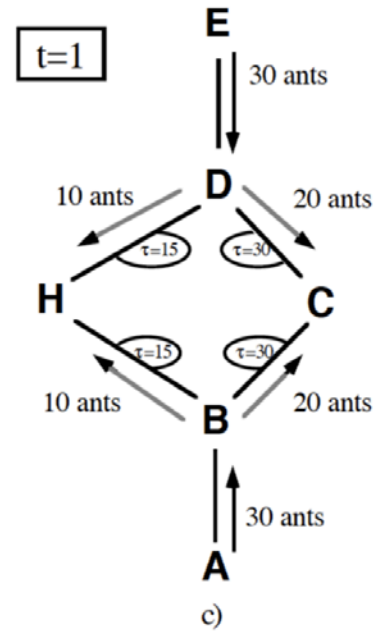
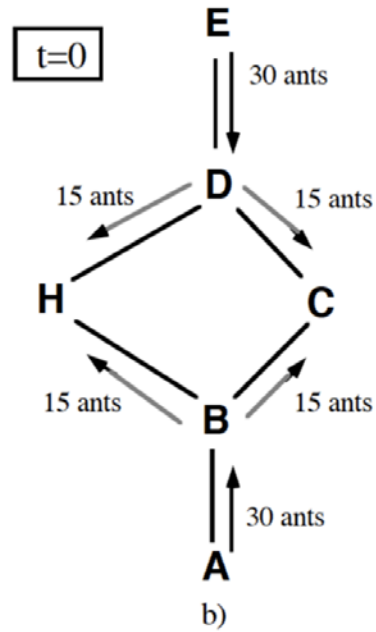
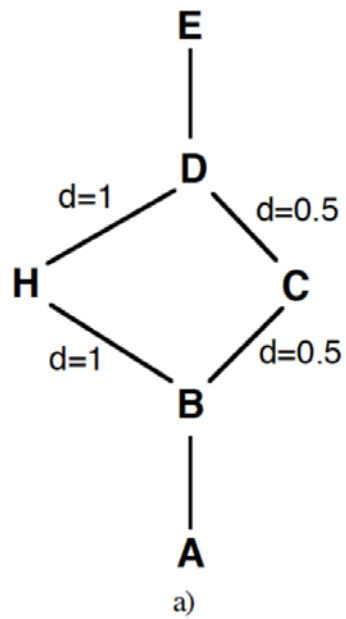
信息素与边权重的关联

算法中的信息素对应图论中的边权重，信息素浓度越高，表示该路径越优质。通过不断更新信息素，算法能够动态调整路径选择，从而逼近全局最优解。

信息素更新的强化与蒸发

蚁群优化算法中的信息素更新分为强化和蒸发两个阶段。强化阶段根据路径长度增加信息素，越短路径增量越大；蒸发阶段以一定系数衰减信息素，防止早熟收敛。这种动态平衡机制使得算法能够在探索 and 开发之间找到平衡。

蚁群算法：例子



概率选择机制公式

概率选择公式

蚂蚁选择路径的概率公式为

$P_i = R_i / (R_1 + R_2)$ ，信息素浓度 R_i 越高，被选择的概率越大。这种概率机制使得算法能够根据信息素浓度动态调整路径选择。

信息素浓度的影响

信息素浓度直接影响蚂蚁选择路径的概率。浓度越高，路径越容易被选中，从而形成正反馈机制，使得优质路径逐渐凸显。

概率机制的动态性

概率选择机制使得算法能够在探索 and 开发之间动态调整。通过不断更新信息素浓度，算法能够动态调整路径选择，从而逼近全局最优解。

全局最优的涌现

通过概率选择机制，算法能够在多次迭代中逐渐汇聚到全局最优解。这种群体智慧的涌现使得蚁群优化算法在解决复杂优化问题时表现出色。

蚁群算法

□ 路径选择概率
$$p_i = \frac{\tau_i^\alpha \eta_i^\beta}{\sum_{j=1}^m \tau_j^\alpha \eta_j^\beta}$$

其中 τ_i 表示第 i 条路径的信息素浓度； η_i 表示第 i 条路径的路径信息，通常用此路径长度的倒数来描述 $\eta_i = \frac{1}{d_i}$ ，表示路径越短，选择此路径的概率越大； α 和 β 这两个参数用于对控制信息素浓度和路径信息在选择概率中的贡献度。

$$\tau_i = (1 - \rho)\tau_i + \Delta\tau_i$$

其中 ρ 代表信息素的挥发率， $\Delta\tau_i$ 表示新增加的信息素。

蚁群算法：基本蚁群算法

基本蚁群算法是针对旅行商问题提出的

$$p_{ij}^k(t) = \begin{cases} \frac{(\tau_{ij}(t))^\alpha (\eta_{ij}(t))^\beta}{\sum_{l \in \text{未被访问的城市}} (\tau_{il}(t))^\alpha (\eta_{il}(t))^\beta} & j \in \text{未被访问的城市} \\ 0 & otherwise \end{cases}$$

城市 i 和城市 j 间路径的信息素更新为：

$$\tau_{ij}(t+n) = (1-\rho)\tau_{ij}(t) + \Delta\tau_{ij}$$

蚁群算法：基本蚁群算法

$$\tau_{ij}(t+n) = (1-\rho)\tau_{ij}(t) + \Delta\tau_{ij}$$

$$\Delta\tau_{ij} = \sum_{k=1}^m \Delta\tau_{ij}^k$$

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{L^k} & \text{if 蚂蚁经过城市 } i \text{ 和 } j \text{ 路径} \\ 0 & \text{otherwise} \end{cases}$$

其中， Q 为常量， L^k 为第 k 只蚂蚁在此次更新中经由的旅行商回路总长。

蚁群算法：蚁群系统

□对基本蚁群算法的改进

➤在路径选择中采用了利用 (exploitation) 和探索

随机数 $rand \in [0, 1]$ ，当这个随机数 $rand \leq q_0$ 时，蚂蚁选择目前的最优路径

$$l_{ij} = \arg \max_{j \in \text{还未被访问的城市}} \left\{ \tau_{ij}(t) (\eta_{ij}(t))^\beta \right\}$$

当 $rand > q_0$ 时，系统采用了探索策略，

即如同基本蚁群算法，按照概率的方式选择路径。

蚁群算法：蚁群系统

□对基本蚁群算法的改进

- 信息素的更新采用了全局更新和局部更新相结合的方案
局部更新是所有蚂蚁完成一次移动 后执行

$$\tau_{ij} = (1 - \rho)\tau_{ij} + \rho\Delta\tau_{ij}$$

$$\text{另 } \Delta\tau_{ij} = \gamma \max_k \tau_{jk}$$

$$\tau_{ij} = \tau_{ij} + \rho(\gamma \max_k \tau_{jk} - \tau_{ij})$$

蚁群算法：蚁群系统

□对基本蚁群算法的改进

➤信息素的更新采用了全局更新和局部更新相结合的方案

全局更新是 在所有的蚂蚁完成了旅行商回路后执行，全局更新仅仅对目前的最优回路上的路径(边)进行信息素的添加。

$$\tau_{ij} = (1 - \alpha)\tau_{ij} + \alpha\Delta\tau_{ij}$$

$$\Delta\tau_{ij} = \begin{cases} 1/L^{opt} & \text{if } l_{ij} \in L^{opt} \\ 0 & \text{otherwise} \end{cases}$$

初始化与参数设定



01

算法初始化

蚁群优化算法在启动前需要进行初始化，包括蚂蚁数量、信息素初始值、蒸发率 ν 、强化系数 K 等参数。这些参数的合理设置对算法性能至关重要。



02

参数设定的重要性

参数设定直接影响算法的探索与开发平衡。合理的参数配置可以避免过早收敛或过度随机，从而提高算法的收敛速度和解的质量。

流程

生成蚂蚁种群

在每次迭代中，首先生成蚂蚁种群。这些蚂蚁将在解空间中随机移动，通过信息素交流来寻找最优解。

计算适应度值

每只蚂蚁在完成一次搜索后，计算其适应度值。适应度值越高，表示该路径越优质。

更新信息素

根据蚂蚁的搜索结果更新信息素。信息素的更新将影响后续蚂蚁的选择，从而推动算法向最优解逼近。

蚁群优化算法的优势与局限

蚁群优化算法具有分布式搜索、鲁棒性强等优势，但同时也存在易早熟收敛、参数敏感等局限。在实际应用中，需要根据具体问题合理调整参数，以充分发挥算法的优势。

群体智能未来趋势

01

蚁群优化算法的回顾

蚁群优化算法以信息素为记忆、以概率为探索、以迭代为收敛，成功模拟蚂蚁群体智慧解决复杂优化问题。

算法的应用场景

蚁群优化算法在旅行商问题、通信路由、车辆调度等领域表现出色，其分布式搜索和鲁棒性强的特点使其能够有效解决复杂优化问题。

02

03

未来发展趋势

未来，蚁群优化算法将结合机器学习自适应参数调整和并行硬件加速，进一步提高算法的性能和应用范围。

粒子群优化

Particle Swarm Optimization

01

传统优化困境与PSO诞生

粒子群优化的动机

传统优化算法的局限性

传统优化算法在处理复杂、高维、非线性问题时，常因局部最优解、对初值敏感以及难以处理未知搜索空间而受限。这些问题限制了其在实际应用中的广泛性。

粒子群优化算法（PSO）应运而生，旨在借鉴自然界中群体行为的智慧，通过模拟鸟类协同觅食的模式，克服传统算法的局限性，实现全局优化。

群体智慧的优势

PSO通过个体与群体信息共享，无需梯度信息，即可在复杂环境中高效寻优，展现出强大的全局搜索能力和适应性。

群体行为启发与核心思想

群体行为的启发

PSO算法的灵感来源于自然界中鸟类和鱼类的群体行为，这些群体通过简单的局部交互展现出复杂的全局行为，为优化问题提供了新的思路。

核心思想阐述

PSO将每个个体视为一个粒子，在搜索空间中飞行，其速度和位置受个体历史最佳位置和群体历史最佳位置的引导，实现全局优化。

粒子状态与数据结构

粒子群的组成

粒子群由多个粒子组成，每个粒子都维护着自己的位置、速度、个体最佳位置及对应的适应度值，共同构成群体的搜索能力。

数据结构设计

每个粒子的数据结构包括当前位置、速度、个体最佳位置等信息，群体则维护全局最佳位置和适应度，为算法迭代提供基础。

群体信息共享

通过个体与群体信息的共享，粒子群能够在搜索过程中不断调整方向，向更优解靠近，展现出群体智慧的力量。

速度与位置更新公式

速度更新公式为 $v = w \cdot v + c1 \cdot r1 \cdot (pbest - x) + c2 \cdot r2 \cdot (gbest - x)$ ，位置更新公式为 $x = x + v$ ，其中 w 是惯性权重， $c1$ 和 $c2$ 分别是个体认知系数和社会系数， $r1$ 和 $r2$ 是随机因子， $pbest$ 是个体历史最佳位置， $gbest$ 是群体历史最佳位置。

迭代循环四步操作

01

计算适应度

在每轮迭代中，首先计算所有粒子的适应度值，评估它们在当前搜索空间中的位置优劣。

02

更新个体最佳

根据适应度值更新每个粒子的个体历史最佳位置，记录个体在搜索过程中的最优解。

03

更新群体最佳

比较所有粒子的适应度，更新群体的历史最佳位置和适应度值，为群体提供全局最优方向。

04

调整速度与位置

根据速度和位置更新公式，调整每个粒子的速度和位置，并进行边界裁剪，确保粒子保持在有效搜索空间内。

优势总结

无需梯度信息

PSO算法无需依赖梯度信息，适用于非线性、不可导的复杂优化问题，拓宽了其应用范围。

对变量缩放不敏感

算法对变量的缩放具有良好的鲁棒性，无需对数据进行复杂的预处理，简化了优化流程。

参数少易调优

PSO的参数相对较少，易于调整和优化，降低了算法的使用门槛，便于快速上手。

全局搜索能力强

凭借群体智慧，PSO在全局搜索方面表现出色，能够有效避免陷入局部最优解，提高优化质量。

易于并行化

算法的迭代过程易于并行化处理，可充分利用现代计算资源，加速优化进程，提升效率。

局限

收敛缓慢

在精细搜索阶段，PSO的收敛速度较慢，可能导致优化过程耗时较长，影响实际应用的效率。

局部搜索能力弱

PSO在局部搜索方面的能力相对较弱，容易出现早熟收敛或震荡现象，降低了优化精度。

函数优化与参数调优

连续函数极值求解

1

PSO能够快速定位连续函数的极值点，在数学优化领域具有广泛的应用前景，为复杂函数的求解提供了高效工具。

神经网络权重优化

2

在神经网络训练中，PSO可用于优化网络权重，提高模型的收敛速度和泛化能力，改善学习效果。

3

SVM超参数搜索

对于支持向量机等机器学习模型，PSO能够高效搜索超参数，提升模型性能，优化分类或回归效果。



对抗搜索算法

Mini-Max 算法

Mini-Max Algorithm

01

对抗搜索定义与典型场景

1

对抗搜索的定义

在多智能体环境中，各智能体目标相互冲突，对抗搜索算法通过搜索博弈树来寻找最优策略，以确保在最坏情况下仍能获得最佳收益。

2

典型应用场景

对抗搜索广泛应用于国际象棋、井字棋、黑白棋等双人零和博弈场景，其中一方的收益即为另一方的损失，算法需为玩家锁定最优策略。

两大玩家角色解析

Max玩家：最大化自身效用

Max玩家代表AI自身，其目标是在对手采取最优反击策略的情况下，选择能够带来最高效用的动作。在每一步决策中，Max玩家都会评估所有可能的动作，并选择其中效用值最高的动作，以确保在对抗中获得最大优势。

Min玩家：最小化对手效用

Min玩家代表对手，其目标是在Max玩家采取最优策略的情况下，选择能够给Max玩家带来最低效用的动作。Min玩家通过评估Max玩家的可能动作，选择最能削弱Max玩家优势的动作，从而在对抗中取得主动。

递归流程

生成游戏树

首先，算法需要生成一个树结构，表示当前游戏状态的所有可能动作。每个节点代表一个游戏状态，每条边代表一种可能的移动。这个过程是递归的，从当前状态开始，逐步展开所有可能的后续状态，直至达到游戏的终端状态。

评估终端状态

接下来，为游戏树的终端节点分配效用值。这些值代表游戏的结果，如赢、输或平局。终端状态的效用值通常是预先定义好的，根据游戏规则直接赋予每个终端节点。

向上传播效用值

从终端节点开始，通过树向上传播效用值。如果轮到最大化玩家，则从子节点中选择最大值；如果轮到最小化玩家，则从子节点中选择最小值。这个过程一直持续到根节点。

选择最佳举措

在游戏树的根部，最大化玩家根据传播上来的效用值，选择导致效用值最高的举动。这个举动就是当前最优的决策，算法返回这个动作作为最终结果。

递归公式与终端赋值



Max玩家的递归公式

Max玩家的递归公式为
$$\text{Max}(s) = \max_{a \in \text{Actions}(s)} \text{Min}(\text{Result}(s, a))$$
，其中 s 表示当前状态， a 表示状态 s 的所有可能动作， $\text{Result}(s, a)$ 表示采取动作 a 后的结果状态。这个公式体现了Max玩家在选择动作时，会考虑对手的最优反应。



Min玩家的递归公式

Min玩家的递归公式为
$$\text{Min}(s) = \min_{a \in \text{Actions}(s)} \text{Max}(\text{Result}(s, a))$$
，与Max玩家的公式类似，但目标相反。Min玩家在选择动作时，会尽量降低Max玩家的效用值，从而达到最小化对手收益的目的。

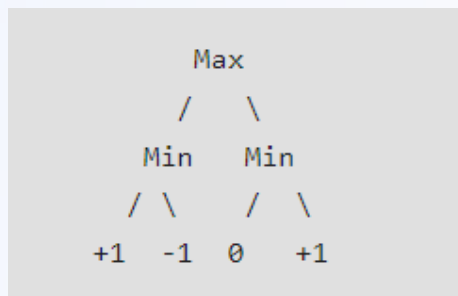


终端状态的效用值赋值

对于终端状态，效用值直接根据游戏结果赋予。如果Max玩家赢了，效用值为+1；如果是平局，效用值为0；如果Min玩家赢了，效用值为-1。这些值是算法决策的基础。

简化博弈树构建

01



构建两层博弈树

以两层博弈树为例，Max根节点下有两个Min子节点，每个Min节点再展开两个终端叶节点。这种简化结构便于理解算法的基本逻辑，同时展示了博弈树的层次结构和信息流向。

02

终端效用值分布

为终端叶节点赋予效用值+1、-1、0、+1，这些值代表了游戏在这些状态下的结果。通过这些值，算法可以评估不同路径的优劣，从而做出最优决策。

回溯计算与最优选择

01

Min层回溯计算

在Min层，算法分别取子节点的最小值，得到-1与0。这一过程体现了Min玩家的策略，即选择能给Max玩家带来最低效用的动作。

02

Max层最优选择

在Max层，算法取最大值0作为根节点的决策结果。这表明在当前情况下，Max玩家的最佳选择是导致效用值为0的动作，从而确保在对抗中获得最大优势。

伪代码与递归实现

1

minimax函数定义

定义minimax函数，参数包括当前状态、剩余深度及当前玩家标识。这个函数是算法的核心，通过递归调用自身，实现了从当前状态到终端状态的效用值评估。

2

终端或深度为0时返回效用

当到达终端状态或搜索深度为0时，函数直接返回当前状态的效用值。这一机制确保了算法在有限的搜索深度内能够得出决策结果。

3

Max与Min轮的评估逻辑

在Max轮，算法遍历所有可能的动作，取最大评估值；在Min轮，取最小评估值。这种交替评估逻辑体现了双方的对抗策略，确保了算法能够找到最优解。

与MCTS及RL对比

与MCTS的对比

极小最大算法通过穷举固定深度内的所有可能动作，确保在该范围内做出最优决策，但随着深度和分支因子的增加，计算复杂度呈指数增长。相比之下，蒙特卡洛树搜索（MCTS）通过随机采样和统计分析来评估动作，更适合高分支因子的游戏。

与强化学习的对比

强化学习（RL）通过与环境的交互来学习最优策略，适用于动态和不确定的环境，能够根据新的经验不断调整策略。而极小最大算法依赖于预计算的效用值和完整的博弈树，适用于状态空间可控的确定性对抗游戏。

适用场景差异

极小最大算法适用于国际象棋等具有明确效用值和可控树大小的游戏；MCTS在复杂、概率性的游戏中表现出色；强化学习则广泛应用于需要自适应、实时决策的场景，如机器人和自动驾驶。

算法优势

最优决策保证

在有限的搜索深度内，极小最大算法能够保证做出最优决策。它通过评估所有可能的动作及其结果，确保在对抗环境中选择最有利的行动方案，为AI提供了强大的决策能力。

逻辑清晰易实现

该算法的逻辑简单明了，易于理解和实现。它通过递归评估和传播效用值，形成了一套清晰的决策流程，使得算法在教学和实际应用中都具有较高的可接受度。

适用于确定性对抗游戏

在状态空间可控的确定性对抗游戏中，如国际象棋、井字棋等，极小最大算法能够有效地模拟双方的最优策略，为AI提供决策支持，是这些领域的经典算法之一。



局限与挑战剖析

计算复杂度高

1

随着博弈树的深度和分支因子的增加，极小最大算法需要评估的节点数量呈指数级增长，导致计算复杂度急剧上升。这使得算法在处理深度较大或分支因子较高的游戏时效率较低。

深度限制问题

2

为了控制计算量，算法通常会限制搜索深度。然而，这可能导致错过关键的远期走法，从而做出次优决策。如何在深度和计算可行性之间取得平衡是一个重要挑战。

3

对不确定环境的适应性差

极小最大算法假设每个动作的结果是确定的，这在现实世界中可能不成立。在不确定或概率性的环境中，算法需要扩展为Expectimax等变体，以适应复杂的情况。

Minimax算法原理与瓶颈



Minimax算法原理

Minimax算法通过递归方式评估博弈树，Max节点取子节点最大值，Min节点取最小值，层层回溯至根节点以确定最优策略。



时间复杂度问题

Minimax算法的时间复杂度为 $O(b^d)$ ，随着分支因子 b 和深度 d 的增加，计算量呈指数级增长，导致复杂游戏难以实时决策。



优化需求

为了在复杂游戏中实现高效决策，需要对Minimax算法进行优化，以减少不必要的节点评估，提高搜索效率。

剪枝触发条件与流程

Max层剪枝条件

在Max层遍历时，若某子节点返回值使 $\text{Alpha} \geq \text{Beta}$ ，则后续兄弟节点无需再探，直接触发剪枝。

Min层剪枝条件

在Min层遍历时，若 $\text{Beta} \leq \text{Alpha}$ ，则直接触发剪枝，无需再评估后续兄弟节点。

剪枝流程总结

一旦发现当前路径无法撼动已确认的边界，即果断放弃后续分支，从而显著减少节点评估量，提高搜索效率。

Alpha与Beta值的定义

Alpha值代表Max玩家当前可确保的最低得分底线，Beta值代表Min玩家当前可确保的最高得分天花板，两者随搜索动态收紧，Alpha只升不降，Beta只降不升。

Alpha-Beta剪枝原理

剪枝条件

Alpha-Beta剪枝的核心是维护两个界限： α （Max玩家的最优值）和 β （Min玩家的最优值）。当 $\alpha \geq \beta$ 时，剪掉剩余分支，因为这些分支不会影响最终决策。

剪枝效果

通过剪枝，算法可以跳过大量不影响决策的节点，从而显著减少计算量，提高搜索效率。这种优化在深度较大的博弈树中尤其有效，使算法能够在更短的时间内做出决策。

Alpha-Beta递归核心剖析

递归函数参数

1

alpha_beta_pruning函数包含depth控制递归深度，maximizing_player布尔值区分Max与Min层，为剪枝提供关键参数。

Max层递归逻辑

2

Max层循环子节点更新alpha值，若某子节点返回值使 $\text{Alpha} \geq \text{Beta}$ ，则触发beta截断，避免无意义的搜索。

Min层递归逻辑

3

Min层对称地更新beta值，若 $\text{Beta} \leq \text{Alpha}$ ，则触发alpha截断，减少不必要的节点评估。

剪枝带来的双重优势

01

效率提升

与朴素Minimax相比，Alpha-Beta剪枝理想排序下节点评估量由指数级降为约 $O(b^{(d/2)})$ ，使深层搜索成为可能，显著提高搜索效率。

02

最优性保持

剪枝仅跳过无关分支，最终决策与穷举完全一致，既提速又不牺牲正确性，是算法优化的黄金标准。

局限与启发式依赖

复杂度问题

尽管剪枝减少了节点评估量，但对于非常大的博弈树，复杂度仍随深度指数增长，超大博弈树依旧吃力。

启发式依赖

剪枝效果高度依赖节点求值顺序，若最佳分支排在最后，剪枝率骤降，需与启发式深度结合以逼近理想剪枝效果。

期望最大搜索

Expectimax Search

01

从Minimax到Expectimax

1

Minimax算法的局限

Minimax算法在确定性对抗游戏中表现出色，但在面对随机事件时显得力不从心。例如，在涉及掷骰子或抽牌的游戏中，Minimax无法有效处理结果的不确定性，这限制了其应用范围。

2

Expectimax算法的诞生

为了解决Minimax在随机环境中的不足，Expectimax算法应运而生。它通过引入机会节点，将概率计算纳入决策过程，使AI能够在不确定性中做出基于期望效用的最优决策。

三大节点类型

最大节点 (Max)

最大节点代表AI决策者，其目标是最大化预期效用。在游戏或决策过程中，AI通过评估各种可能的行动，选择能够带来最高期望收益的动作。

最小节点 (Min)

最小节点通常代表对手，在对抗性游戏中，对手的目标是最小化AI的效用。通过假设对手会做出最优反应，AI可以提前规划应对策略。

机会节点 (Chance)

机会节点用于表示随机事件，如掷骰子或抽牌。在这些节点上，算法根据各种可能结果的概率计算期望效用，而不是假设最坏情况。



递归评估流程

终端状态判断

当搜索到达终端状态或达到预设的深度限制时，算法返回当前状态的效用值。这是递归的终止条件，确保算法不会无限深入。

最小节点处理

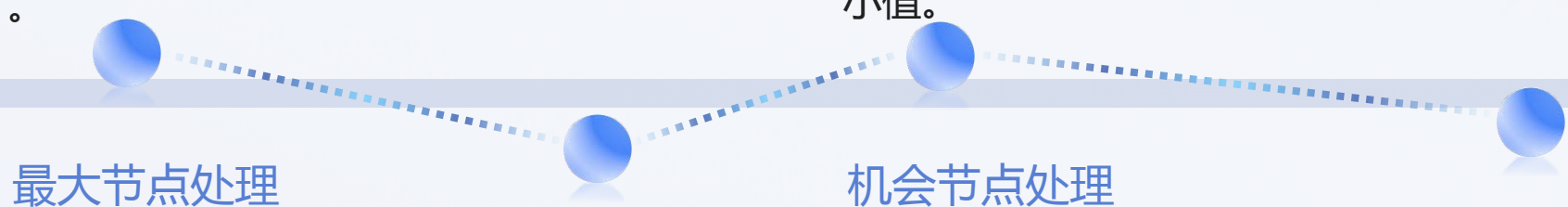
在最小节点，算法假设对手会做出最优反应，因此选择使AI效用最小化的动作。通过递归计算每个动作的期望效用，选择最小值。

最大节点处理

在最大节点，算法遍历所有可能的动作，递归调用自身计算每个动作的期望效用，并选择期望效用最高的动作。

机会节点处理

在机会节点，算法根据每个可能结果的概率加权求和，计算出该节点的期望效用。概率的准确性直接影响决策的合理性。



期望效用计算

1

Move B的期望计算

对于Move B，算法计算每个可能结果的概率加权和。掷出1或2得10分，概率为1/3；掷出3或4得0分，概率为1/3；掷出5或6得6分，概率为1/3。

2

期望效用结果

Move B的期望效用为 $(1/3 * 10) + (1/3 * 0) + (1/3 * 6) = 5.33$ 。相比之下，Move A的确定效用为5分。

3

决策结果

由于Move B的期望效用高于Move A，根据Expectimax算法，AI会选择Move B。

游戏场景描述

在一个简单的骰子游戏中，AI需要在两个动作之间选择：Move A是一个确定性动作，总是获得5分。Move B是一个随机动作，根据掷骰子的结果，可能获得10分、0分或6分。

与Minimax及剪枝对比

Minimax算法

Minimax算法适用于确定性对抗游戏，假设对手总是做出最优反应，以最小化AI的效用。

Expectimax算法

Expectimax算法适用于包含随机事件的游戏，通过计算期望效用，而不是假设最坏情况，来做出决策。

Alpha-Beta剪枝

Alpha-Beta剪枝是Minimax的优化技术，通过剪枝减少节点评估。但由于Expectimax中的机会节点引入概率，无法直接应用剪枝。

随机博弈中的AI

2048游戏

西洋双陆棋

在西洋双陆棋中，掷骰子的结果直接影响棋子的移动。Expectimax算法通过计算不同走法的期望胜率，帮助AI做出最优决策。

在2048游戏中，新方块的出现是随机的。AI通过Expectimax算法评估不同移动策略的期望得分，从而选择最优的合并策略。

扑克游戏

在扑克游戏中，牌的抽取是随机的。

Expectimax算法帮助AI评估不同手牌的期望价值，从而做出合理的下注或弃牌决策。

现实世界决策



机器人路径规划

在机器人路径规划中，动作结果可能受到传感器噪声或环境不确定性的干扰。Expectimax算法通过计算期望代价，帮助机器人选择最安全的路径。



金融投资决策

在金融投资中，市场条件的不确定性要求投资者考虑各种可能的未来情况。Expectimax算法通过评估不同投资策略的期望收益，帮助投资者做出最优决策。



自动驾驶决策

在自动驾驶中，面对感知不确定的情况，Expectimax算法评估不同驾驶方案的期望风险，帮助自动驾驶系统做出安全决策。

算法局限与风险



01

计算复杂度

Expectimax算法在机会节点处的分支因子可能很大，导致计算量随深度指数增长，性能受限。

概率模型的准确性

算法的决策质量依赖于机会节点概率的准确建模。错误的概率估计可能导致决策不理想。



02



03

终端效用函数设计

终端效用函数的设计至关重要，不合理的效用函数可能导致期望计算失真，影响决策的合理性。

蒙特卡洛树搜索

Monte Carlo Tree Search

01

MCTS诞生背景与动机

01

传统算法瓶颈

在围棋等复杂游戏中，传统Minimax算法因决策空间巨大而面临指数级复杂度挑战，难以在有限时间内找到最优解。

02

MCTS的创新思路

蒙特卡洛树搜索通过随机模拟逐步构建搜索树，以统计抽样代替穷举，从而在有限计算预算内逼近最优决策，有效突破传统算法瓶颈。

UCB公式

蒙特卡洛树搜索 (MCTS) 中的UCB (Upper Confidence Bound) 公式用于平衡探索 (exploration) 和利用 (exploitation)，其核心在于通过自然对数函数 $\ln(N)$ 调整探索强度。公式为：

$$UCB = w_i/n_i + C \cdot \sqrt{(\ln(N)/n_i)}$$

公式解析

- 第一部分 (w_i/n_i)：表示节点i的平均奖励，反映当前已知收益，用于利用已知信息。
- 第二部分 ($C \cdot \sqrt{(\ln(N)/n_i)}$)：用于探索未知潜力较大的节点，随着父节点访问次数N增加，探索强度逐渐减弱但保持增长趋势。

$\ln(N)$ 的作用

1. 平衡探索与利用： $\ln(N)$ 随父节点访问次数N增长缓慢，避免后期过度探索，使算法更专注于高收益节点。
2. 渐进收敛：随着N增大，探索项逐渐趋近于0，最终算法趋向纯利用策略。

该公式通过调节参数C (超参数) 可控制探索与利用的平衡度，广泛应用于强化学习、博弈树决策等领域。

探索与利用的平衡思想

探索与利用的定义

探索是指尝试新的、未被充分研究的决策路径，而利用是指在已知的高胜率路径上进行决策。

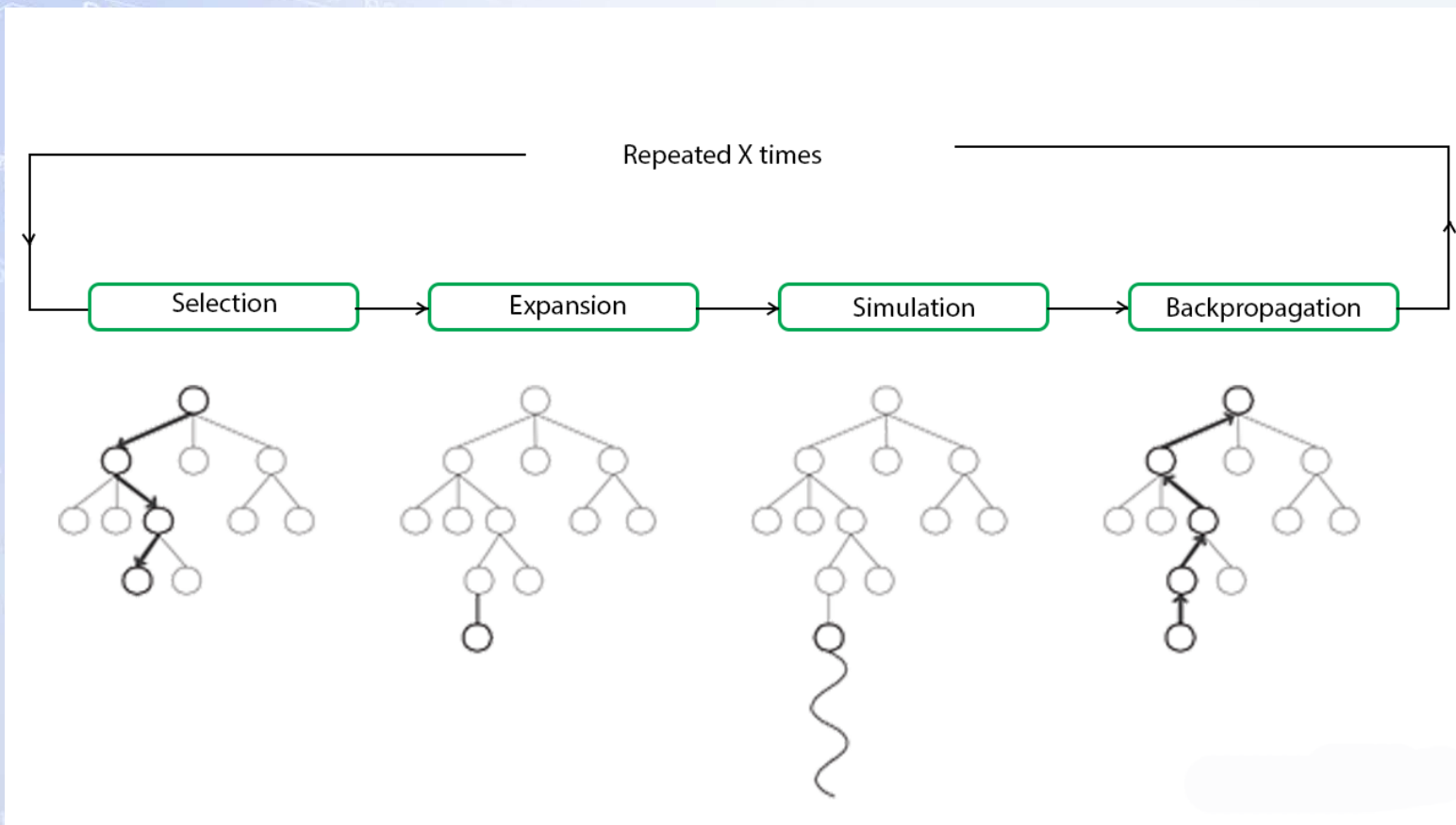
UCB公式的角色

UCB公式通过结合平均胜率和探索项，量化了探索与利用的平衡，为节点选择提供了科学依据。

动态优化决策

在每次迭代中，MCTS根据UCT公式动态调整探索与利用的比例，持续优化决策质量。

流程





01

选择阶段

从根节点出发，沿子树逐层使用Upper Confidence Bounds公式挑选置信上限最高的节点，直至到达未完全展开的叶节点。

扩展阶段



02

在叶节点处随机选择一个未尝试动作生成新子节点，为后续模拟提供新的起点。



03

模拟阶段

从新子节点开始执行随机走子直至终局，记录胜负结果，为反向传播提供数据。

反向传播流程



04

将模拟结果沿路径回传，更新所有祖先节点的访问次数与累计胜率。统计量通过增量更新机制逐步收敛到真实胜率，为后续决策提供更准确的参考。通过多次迭代，算法逐步逼近最优策略，提高决策的准确性。

围棋与神经架构搜索

围棋中的应用

在AlphaGo中，MCTS通过百万级模拟结合价值网络实现超人类水平，展现了其在复杂决策中的强大能力。

神经架构搜索

在神经架构搜索中，MCTS通过随机网络采样优化模型结构，为深度学习模型设计提供高效解决方案。

算法优势与局限剖析

1

算法优势

MCTS无需领域知识即可通过随机模拟逼近最优，天然并行且易于实现，适用于多种复杂决策场景。

2

局限性

高方差导致结果不稳定，战术盲区可能错过短期机会，探索常数需人工调参，影响算法的适用性。

3

改进方向

结合领域启发式进行改进，提高算法在特定领域的决策效率和准确性。